Scalable Training of Mixture-of-Experts Models with Megatron Core

Technical Report

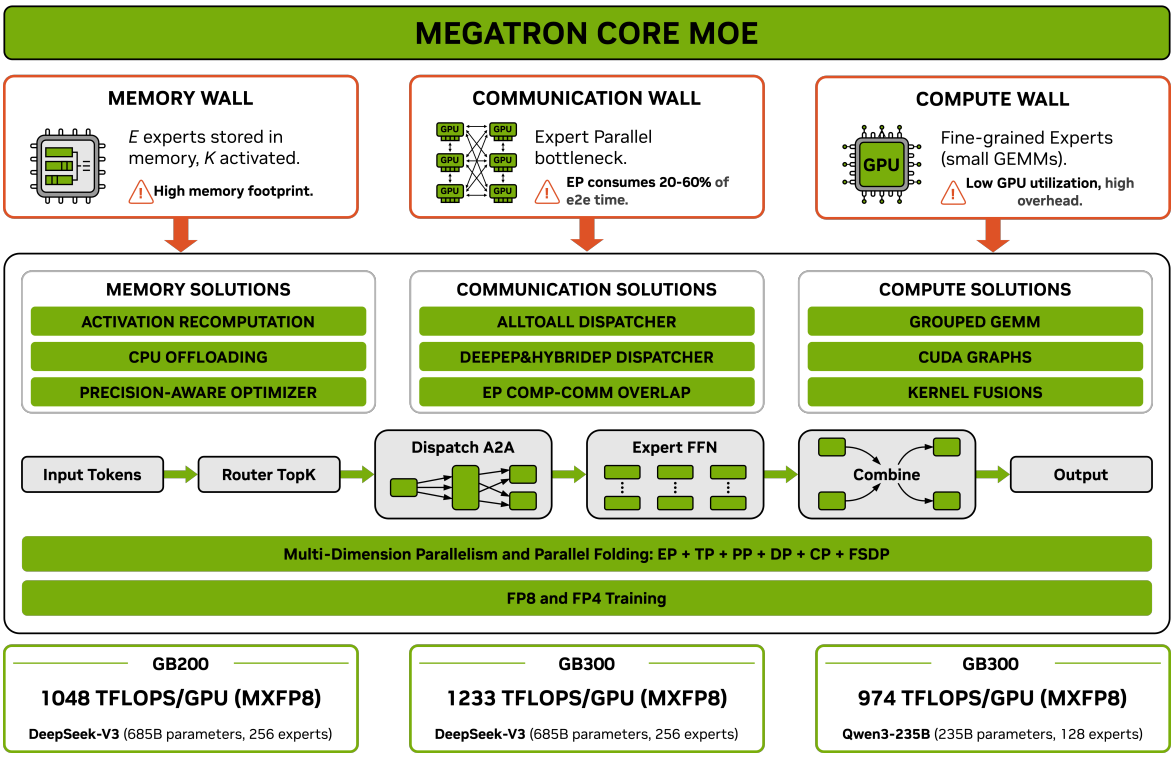
NVIDIA[1]

Abstract

Scaling Mixture-of-Experts (MoE) training introduces systems challenges absent in dense models. Because each token activates only a subset of experts, this sparsity allows total parameters to grow much faster than per-token computation, creating coupled constraints across memory, communication, and computation. Optimizing one dimension often shifts pressure to another, demanding co-design across the full system stack.

We address these challenges for MoE training through integrated optimizations spanning memory (fine-grained recomputation, offloading, etc.), communication (optimized dispatchers, overlapping, etc.), and computation (Grouped GEMM, fusions, CUDA Graphs, etc.). The framework also provides Parallel Folding for flexible multi-dimensional parallelism, low-precision training support for FP8 and NVFP4, and efficient long-context training. On NVIDIA GB300 and GB200, it achieves 1,233/1,048 TFLOPS/GPU for DeepSeek-V3-685B and 974/919 TFLOPS/GPU for Qwen3-235B. As a performant, scalable, and production-ready open-source solution, it has been used across academia and industry for training MoE models ranging from billions to trillions of parameters on clusters scaling up to thousands of GPUs.

This report explains how these techniques work, their trade-offs, and their interactions at the systems level, providing practical guidance for scaling MoE models with Megatron Core.



[1]For the complete list of authors, please refer to the Contributions and Acknowledgments section. Corresponding authors: {zijiey, juney, jiajiey}@nvidia.com.

Contents

1. Introduction

Training dense transformers at scale requires computation that grows linearly with model size [1, 2]. Mixture of Experts (MoE) models follow a different scaling pattern: by routing each token to a selected subset of expert networks rather than activating all parameters, per-token computation grows sub-linearly with model size [3, 4]. Recent MoE models have demonstrated order-of-magnitude reductions in training cost relative to quality-matched dense models [5, 6, 7].

However, training MoE at scale introduces systems challenges that dense-model frameworks were not designed for. This report presents Megatron-Core MoE, the MoE training stack within Megatron-Core [8], covering the architecture, parallelism strategies, and system optimizations required to train trillion-parameter-class MoE models at high throughput.

1.1. Mixture of Experts

A Mixture of Experts (MoE) model augments a standard neural network with a collection of specialized sub-networks called *experts*, together with a lightweight *router* (or gating network) that dynamically selects which experts process each input [3, 9, 10]. In the context of Transformer-based language models [11], MoE layers typically replace the dense Feed-Forward Network (FFN) blocks: instead of a single FFN applied to all tokens, an MoE layer contains multiple FFN experts, and each token is routed to a small subset (e.g., top-k) of these experts based on learned routing weights.

Formally, given an input token representation $\mathbf{x}$, the router computes a probability distribution over E experts:

$$\mathbf{p}(\mathbf{x}) = \text{Softmax}(\mathbf{W}_r \mathbf{x})$$

The output of the MoE layer is then computed as a weighted combination of the selected experts' outputs:

$$\text{MoE}(\mathbf{x}) = \sum_{i \in \text{TopK}(\mathbf{p}(\mathbf{x}))} p_i(\mathbf{x}) \cdot E_i(\mathbf{x})$$

where E_i denotes the i-th expert network. This architecture offers three key advantages: model capacity can grow independently of computational cost by adding more experts (*scalable capacity*); only a fraction of parameters activate per token, reducing FLOPs relative to a dense model of equivalent size (*computational efficiency*); and different experts can specialize on different input types (*specialization*). Appendix A provides a notation reference for all symbols and abbreviations used in this report.

While the concept dates back to the early 1990s [9, 10], the integration of MoE with modern Transformers has driven renewed interest. GShard pioneered distributed MoE training at scale, introducing Expert Parallelism and load-balancing auxiliary losses [12]. Switch Transformer demonstrated that MoE could scale to trillion parameters while maintaining training stability [4]. GLaM showed that MoE could match dense model quality at a fraction of the training cost [13]. Frameworks including Tutel [14] and DeepSpeed-MoE [15] further advanced MoE training systems.

MoE adoption has accelerated across research and industry. Mixtral-8x7B showed that open-weight MoE models can match proprietary dense models while reducing inference cost [5]. DeepSeek-V2 and DeepSeek-V3 extended this with fine-grained expert architectures, using hundreds of small experts to maximize the capacity-to-compute ratio [16, 17, 6]. NVIDIA's Nemotron-3 family [18] adopts a hybrid Mamba-Transformer MoE architecture with LatentMoE [19], trained at scale using Megatron-Core. Scaling law studies confirm that fine-grained MoE achieves favorable compute-optimal trade-offs [20, 21], accelerating this trend—while amplifying the systems challenges it creates.

1.2. Challenges in Training Large-Scale MoE Models

As models push toward hundreds of experts with smaller individual capacity, the systems challenges of MoE training grow in proportion. These challenges stem from a single root cause: MoE's **sparsity**—which manifests as a *Parameter-Compute Mismatch* (total parameters far exceeding active computation, this section) and a *Dense-Sparse Mismatch* (attention and MoE layers requiring conflicting parallelism configurations, Section 3).

The core asymmetry comes from sparsity. In a dense transformer, every parameter participates in every training step. A model with N_{total} parameters requires roughly $6N_{\text{total}}$ FLOPs per token (forward and backward combined), so parameters and per-token computation scale in lockstep. Splitting the model across more GPUs usually also splits computation in the same proportion, keeping each GPU busy enough that communication overhead stays small.

For MoE, sparsity causes this coupling to break: only K of E total experts activate per token, so per-token computation is roughly $6N_{\text{active}}$ rather than $6N_{\text{total}}$, where N_{active} scales with K while N_{total} scales with E, and $K \ll E$. This creates a fundamental parameter-compute mismatch: compared with a dense model matched on active parameters per token, an MoE model has far more total parameters, often by an order of magnitude. DeepSeek-V3 illustrates this concretely: 685B total parameters but only 37B active per token, an 18$\times$ gap.

With so little computation per token, model partitioning requires more care than for dense models—naively sharding expert matrices (as Tensor Parallelism would) fragments already-small computations, making them even less efficient. Because MoE experts are independent networks, the natural strategy is *Expert Parallelism* (EP): placing different experts on different GPUs, preserving full-size expert GEMMs. EP introduces all-to-all communication to route tokens to their assigned GPUs (Section 3 details why EP is preferred and how Parallel Folding addresses the resulting challenges). This parameter-compute mismatch and EP's communication demands together create three tightly coupled challenges—the *Three Walls*—that constrain every MoE training step:

The Memory Wall. All E experts' parameters, gradients, and optimizer states must reside in memory during training, even though only K activate per token. This creates memory pressure far exceeding that of a dense model with comparable per-token compute [4, 7]. Reducing this pressure requires spending elsewhere: distributing parameters across more devices costs *communication bandwidth*; recomputing activations instead of storing them costs *extra computation*; offloading to host memory costs *PCIe bandwidth*. Dynamic routing further complicates matters: uneven token distributions cause unpredictable memory spikes when some experts receive disproportionate load [12, 22].

The Communication Wall. EP requires all-to-all collectives to dispatch tokens to their assigned experts and collect results [12]. The per-GPU send volume in each all-to-all is approximately $T \cdot K \cdot h \cdot \frac{EP-1}{EP}$, where T is the local token count, K the top-k, and h the hidden dimension; a full dispatch-and-combine cycle doubles this. As EP grows, this volume saturates but the communication increasingly moves from high-bandwidth intra-node links (e.g., NVLink) to narrower inter-node interconnects, where available bandwidth drops by an order of magnitude [23]. The sparse activation pattern, meanwhile, provides limited computation to overlap with this communication. In architectures like DeepSeek-V3, where experts span multiple nodes, unoptimized all-to-all can consume up to 60% of total training time.

The Compute Efficiency Wall. MoE introduces computational inefficiencies absent in dense models:

- **Small GEMMs.** Fine-grained experts produce many small matrix multiplications that underutilize GPU compute units [24]. In our measurements, GEMMs account for ~70% of execution time in Llama-3 405B (dense) but under 50% in DeepSeek-V3 (MoE). The remainder is consumed by operations that scale with tensor count rather than FLOP count.
- **Routing and permutation overhead.** Token routing and permutation, absent in dense models, add ~9% to layer execution time even after optimization.

- **Load imbalance.** Dynamic routing assigns uneven token counts to experts, leaving some overloaded while others sit idle, wasting compute capacity [22].
- **Host overhead.** MoE launches more kernels for the same amount of FLOPs because of sparsity and routing, and each launch carries fixed host-side cost, these add up and leave the GPU idle between kernels. In dropless MoE, dynamic tensor shapes further require costly host-device synchronization.

These three walls are tightly coupled: optimizing one often shifts pressure to another. Increasing batch size improves GEMM utilization but amplifies memory pressure and communication volume. CUDA Graphs eliminate host overhead but require static tensor shapes, conflicting with dropless routing. Grouping tokens across experts improves compute efficiency but complicates load balancing. Section 3 provides the detailed parallelism analysis underlying EP and Parallel Folding; Section 4 presents Megatron-Core's integrated approach to addressing all three walls while managing their interactions.

1.3. Megatron-Core MoE

Built within Megatron-Core, a PyTorch-based library for large-scale transformer training [8, 25], this MoE training stack tackles all three walls simultaneously:

Multi-Dimensional Parallelism. Expert Parallelism (EP) integrates with tensor, pipeline, sequence, and data parallelism. MoE Parallel Folding [26] decouples attention and MoE layer configurations, breaking the traditional EP $\leq$ DP constraint and enabling configurations tailored to specific model architectures and hardware topologies.

Memory Optimizations. Fine-grained activation recomputation, memory-efficient permutation, precision-aware optimizers, and activation offloading reduce memory footprint without sacrificing throughput [27, 28]. Comprehensive reduced-precision training (FP8 and FP4) support in expert GEMMs, activations, and communication further reduces activation storage while maintaining convergence through selective precision strategies.

Communication Optimizations. High-performance token dispatchers (DeepEP, HybridEP) maximize bandwidth utilization. Communication-computation overlap hides all-to-all latency behind expert computation.

Compute Optimizations. Grouped GEMM kernels, kernel fusion, CUDA Graphs, and sync-free execution address the computational fragmentation inherent in fine-grained MoE architectures.

Production Features. Load balancing strategies, token dropping with capacity control, distributed optimizer and FSDP support, distributed checkpointing with flexible resharding, and upcycling from dense checkpoints enable deployment at scale. The MoE stack's modular design enables rapid experimentation, while its production-grade optimizations support training from research prototypes to trillion-parameter models [29]. This report explains what the stack provides, why key design decisions were made, and how they address MoE training challenges, with practical guidance for configuration and tuning.

1.4. Structure of This Paper

The remainder of this report follows a progression from architecture to optimization to evaluation:

- **Section 2: Megatron-Core MoE Architecture.** Introduces Megatron-Core MoE's design in two parts: the internal design of the MoE layer itself (router, token dispatcher, experts) and the four-stage forward pass (route, dispatch, compute, combine), followed by the external design covering integration with the transformer model, parallel process group organization, and optimizer handling for expert parameters.
- **Section 3: Scaling MoE with Parallel Folding and Multi-Dimensional Parallelism.** Examines how MoE's sparsity breaks the parallelism assumptions of dense training, why Expert Parallelism (EP) is needed alongside traditional strategies, and how the resulting dense-sparse mismatch between attention

and MoE layers is resolved by MoE Parallel Folding, which decouples their parallelism configurations for flexible, efficient mapping at trillion-parameter scale.

- **Section 4: Breaking the Memory, Communication, and Compute Efficiency Walls.** Presents Megatron-Core MoE's solutions to the three fundamental barriers: the Memory Wall (activation management, recomputation strategies, offloading, distributed parameter storage), the Communication Wall (optimized dispatchers including DeepEP and HybridEP, communication-computation overlap), and the Compute Efficiency Wall (Grouped GEMM, kernel fusion, CUDA Graphs, sync-free execution for dropless MoE).
- **Section 5: Reduced-Precision Training in FP8/FP4 for MoE.** Covers reduced-precision training as a cross-cutting optimization that simultaneously impacts all three walls by reducing activation memory, halving communication volume, and accelerating Tensor Core GEMMs, while presenting strategies for selective precision to maintain training stability.
- **Section 6: Long-Context MoE Training.** Examines how long-context scenarios (16K to 64K+ tokens) fundamentally shift the optimization landscape as attention computation dominates, and presents techniques for managing activation memory growth through Context Parallelism and Tensor Parallelism scaling.
- **Section 7: Production Features.** Describes operational features for production training: load balancing and token dropping for stable training, distributed checkpointing for parallelism-agnostic resharding, upcycling from dense checkpoints, and integration with multi-token prediction.
- **Section 8: Performance Evaluation.** Validates the framework's effectiveness through empirical benchmarks on DeepSeek-V3 and Qwen3-235B across GB200 and H100 platforms, demonstrating the impact of the full optimization stack.
- **Section 9: Performance Best Practices with DeepSeek-V3 Case Study.** Provides a systematic workflow for identifying optimal parallelism configurations, validated through a detailed DeepSeek-V3 case study that demonstrates how the optimizations work together to achieve state-of-the-art performance.
- **Section 10: Megatron-Core MoE in Reinforcement Learning.** Addresses the emerging RL post-training paradigm, covering challenges unique to RL workloads (variable sequence lengths, memory offloading, online weight export), Megatron-Bridge integration with popular RL frameworks, and RL-specific optimizations including packed sequence support, dynamic context parallelism, and router replay.

2. Megatron-Core MoE Architecture

This section presents Megatron-Core's MoE implementation architecture in two parts. We first describe the **internal design** of the MoE layer itself: its modular components (router, token dispatcher, experts) and the four-stage forward pass that transforms input tokens into output representations. We then examine the **external design**: how parallel process groups are organized to support distributed execution and how the optimizer handles expert parameters differently from dense layers.

2.1. MoE Layer Architecture and Forward Pass

An MoE layer replaces the dense feed-forward network (FFN) in a transformer block with a collection of expert FFNs, only a subset of which process each token. Megatron-Core implements this through three modular components (a *router* for token-to-expert assignment, a *token dispatcher* for cross-GPU communication, and *experts* for computation) connected through the four-stage forward pass shown in figure 1. This separation of concerns enables independent optimization: the router can be fused into CUDA Graphs without affecting dispatcher logic, dispatchers can be swapped between all-to-all and DeepEP without modifying expert computation, and expert implementations can use different GEMM backends transparently.

2.1.1. Forward Pass: Route, Dispatch, Compute, Combine

The MoE layer processes input tokens through four sequential stages:

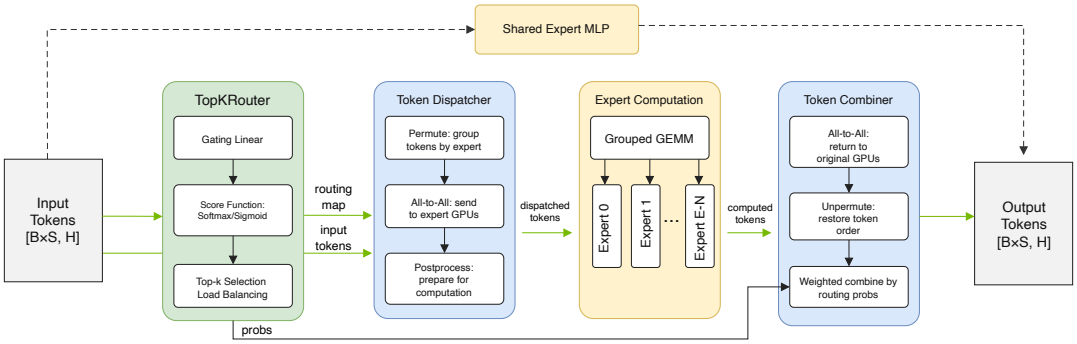


Figure 1: Data flow through an MoE layer: Route, Dispatch, Compute, and Combine stages.

Stage 1: Route. The `TopKRouter` determines which experts process each token. A learned linear projection maps each token's hidden state to E logits (one per expert), a score function (softmax or sigmoid) converts logits to probabilities, and top-k selection identifies the highest-scoring experts for each token. The router outputs two tensors: `probs` containing the routing weights, and `routing_map`, a boolean mask indicating token-expert assignments. For numerical stability with many experts, the router can operate in FP32 via `--moe-router-dtype fp32`.

Stage 2: Dispatch. The token dispatcher prepares tokens for cross-GPU communication. It first permutes tokens so that all tokens destined for the same expert are contiguous; this permutation is essential for efficient dense GEMM on the expert side. The dispatcher then moves tokens to the GPUs hosting their assigned experts using one of three backends: AllGather (simple but memory-intensive), all-to-all (standard NCCL-based), or Flex (supporting optimized backends like DeepEP and HybridEP).

Stage 3: Expert Computation. Each GPU executes its local experts on the received tokens. All local experts run in a single Grouped GEMM call via `TEGroupedMLP`, maximizing GPU utilization even when individual expert workloads are small.

Stage 4: Combine. The inverse communication returns processed tokens to their original GPUs, followed by unpermutation to restore the original sequence order. If a shared expert is configured, its output (optionally computed in parallel with routed experts) is added at this stage.

2.1.2. Router: Token-to-Expert Assignment

The router transforms a global token batch into expert-specific workloads through two operations [3]:

1. **Gating:** A linear projection $\mathbf{W}_r \in \mathbb{R}^{h \times E}$ maps each token's hidden state $\mathbf{x} \in \mathbb{R}^h$ to logits $\mathbf{l} = \mathbf{W}_r^\top \mathbf{x} \in \mathbb{R}^E$.
2. **Top-k Selection:** A score function converts logits to probabilities: softmax ($p_i = e^{l_i} / \sum_j e^{l_j}$) or sigmoid ($p_i = \sigma(l_i) / \sum_j \sigma(l_j)$, used by DeepSeek-V3 [6]). The top-k experts with highest probabilities are selected per token.

Uneven expert utilization degrades both training efficiency and model quality. As shown in figure 2, Megatron-Core supports multiple load balancing strategies; these are discussed in Section 7.

2.1.3. Token Dispatcher: Communication Abstraction

The dispatcher manages token movement between GPUs through a six-phase pipeline: `dispatch_preprocess` → `token_dispatch` → `dispatch_postprocess` (forward), and `combine_preprocess` → `token_combine` →

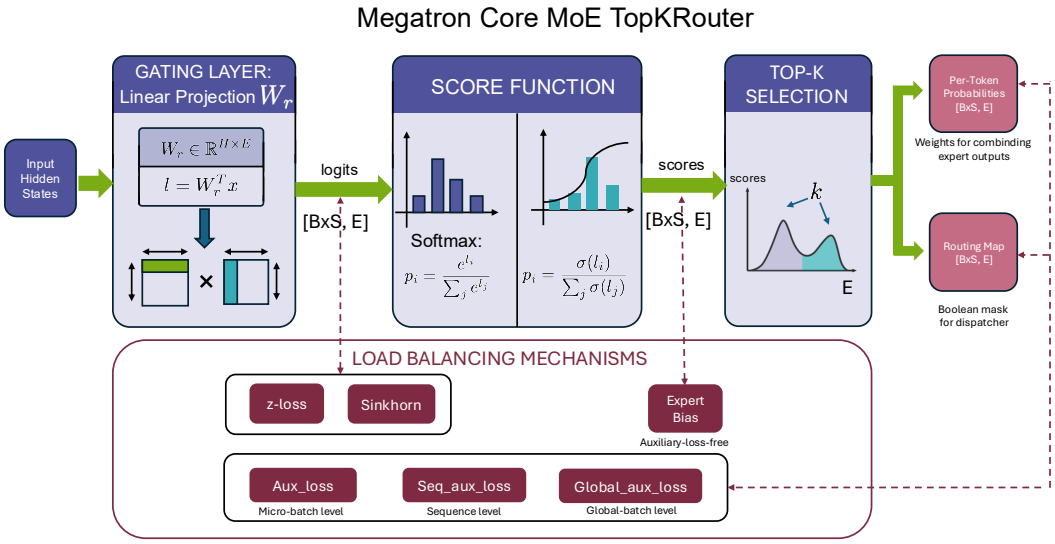


Figure 2: Router architecture: linear projection, score function, top-k selection, and load balancing.

`combine_postprocess` (backward).

Communication Backends. Three dispatcher types are available:

- **AllGather** (`allgather`): Each GPU gathers all tokens and filters for local experts. Simple but memory-intensive; suitable for small EP sizes.
- **All-To-All** (`all-to-all`): Standard NCCL-based point-to-point communication [12]. Each GPU sends only the tokens needed by each destination. Scales well but incurs synchronization overhead.
- **Flex** (`flex`): Unified design supporting DeepEP (only high-throughput kernels are integrated, hiding NVLink communication latency through overlap) [23] and HybridEP (high-bandwidth MoE communication kernels that deliver better performance on NVLink-rich topologies like NVL72).

2.1.4. Experts: Computation Module

Each expert is a two-layer MLP with optional gating (for SwiGLU/GeGLU activations [30]). Grouped GEMM enables efficient batching of multiple expert computations [24, 14]. Two implementations are available:

- **TEGroupedMLP**: Transformer Engine [31] optimized implementation supporting FP8 and FP4 quantization.
- **SequentialMLP**: Executes experts one at a time in a loop. Useful for debugging but significantly slower.

Shared Experts. Some architectures (DeepSeek-V2/V3 [17, 6], Qwen [32]) include a shared expert that processes all tokens regardless of routing. Shared expert computation can run in parallel with the dispatch-compute-combine pipeline, hiding its latency. See Section 7 for details.

2.2. System Integration: Parallelism and Optimizer

Having described how an MoE layer works internally, we now examine how it integrates into the distributed training system: parallel process groups and optimizer handling.

2.2.1. Parallel Group Management

MoE layers require distinct process groups from dense layers because different components have different communication patterns. Megatron-Core organizes these groups through `ProcessGroupCollection`:

```
ProcessGroupCollection
|-- Attention Layer Groups: tp, cp, dp, pp
|-- Expert Layer Groups:    ep, expt_tp, expt_dp, pp
```

Each MoE component uses specific groups based on its communication requirements:

Component	Groups Used	Reason
Router	`tp`, `cp`, `tp_cp`	Weights duplicated across EP ranks
Token Dispatcher	`ep`, `tp_ep`	all-to-all across expert ranks
Experts	`ep`, `expt_tp`, `expt_dp`	Sharded across EP; gradients reduced in EDP
Shared Experts	`tp`	Same as dense MLP

Table 1: MoE component to process group mapping.

This separation enables **Parallel Folding** (Section 3.3): attention and MoE layers can use different TP/DP configurations. For example, attention layers might use TP=4 while MoE layers use ETP=1 with higher EP, optimizing each layer type independently.

2.2.2. Optimizer and Gradient Handling

Expert parameters require distinct handling in distributed optimization. Megatron-Core uses a `ChainedOptimizer` that wraps separate optimizers for dense and expert parameters. Three key design decisions support correct MoE optimization:

1. **Parameter Identification.** Expert parameters are marked with `allreduce=False`, distinguishing them from dense parameters that use standard data-parallel gradient reduction.
2. **Separate Reduction Groups.** Dense layers reduce gradients across `dp_cp_group` (full data parallelism), while experts reduce across `expt_dp_group` (expert data parallelism). This ensures gradients are averaged over the correct number of replicas.
3. **Gradient Scaling.** Expert gradients are scaled by `edp_size / dp_size` to account for the different effective batch sizes seen by experts (which process routing-dependent token subsets) versus dense layers.

This design allows ZeRO-style optimizer state sharding [33] to work seamlessly with MoE: optimizer states for expert parameters are sharded across the EP group, while states for dense parameters follow standard DP sharding.

With the architecture established, Section 3 addresses how to distribute it across devices, and Section 4 tackles the memory, communication, and compute efficiency challenges that arise at scale.

3. Scaling MoE: Parallel Folding and Multi-Dimensional Parallelism

Section 1.2 established that MoE's sparsity decouples model size from per-token computation, creating a parameter-compute mismatch that makes Expert Parallelism (EP) necessary but introduces all-to-all communication. This section examines how that mismatch concretely affects parallelism design. We first review the parallelism baseline for dense models and the trade-offs it entails, then show how MoE's sparsity breaks the assumptions underlying these strategies. Finally, we present **MoE Parallel Folding**, Megatron-Core's solution to the resulting *dense-sparse mismatch*: attention and MoE layers have conflicting optimal parallelism configurations, and Parallel Folding decouples their mappings so each can use its optimal topology.

3.1. Why Parallelism, and Why MoE is Different

Before examining MoE-specific challenges, we establish the fundamental reasons parallelism is necessary for large model training and the trade-offs it introduces.

3.1.1. Why Large Model Training Needs Parallelism

Memory is the fundamental constraint. A single GPU has limited memory, but training a large model requires storing model parameters, optimizer states, gradients, and activations simultaneously [34]. Consider Llama-405B [35, 36, 37] trained with BF16 precision and Adam optimizer:

Component	Memory (Llama-405B, BF16)
Model parameters	∼810 GB
Optimizer states (Adam)	∼4860 GB
Gradients	∼1620 GB
Activations (8K sequence)	∼5575 GB
Total	**∼12865 GB**

This far exceeds any single GPU's capacity. Multiple GPUs are not optional; they are *required* to hold the model.

Compute throughput is the efficiency reason. Beyond memory, a single GPU has limited compute capacity. Large model training requires astronomical FLOPs; aggregating multiple GPUs increases throughput and reduces wall-clock training time from years to weeks.

3.1.2. The Trade-off of Parallelism

Parallelism is not free. Every parallelism strategy introduces overhead:

- **Communication overhead**: Data exchange between GPUs consumes time and bandwidth.
- **Synchronization**: Fast GPUs must wait for slow GPUs at synchronization points.
- **Pipeline bubbles**: Pipeline parallelism introduces idle time at pipeline boundaries.
- **Reduced compute intensity**: Sharding reduces per-GPU matrix sizes, lowering GEMM efficiency.

The result: Model FLOP Utilization (MFU) is heavily influenced by parallelism strategy, and effective parallelism design can minimize the gap between ideal and actual MFU.

Key insight for dense models: The "cost" and "benefit" of parallelism scale proportionally. More parameters require more GPUs, but more parameters also mean more computation per forward-backward pass. Because computation grows with model size, communication takes a smaller share of each step, keeping MFU relatively stable as models scale.

MoE's sparsity disrupts this balance. Because only K of E experts activate per token, total parameters scale with E while per-token compute scales only with K. More GPUs are needed for memory, but per-token computation does not grow to match, leaving communication overhead exposed. The following section examines how this asymmetry breaks the assumptions underlying traditional parallelism strategies and why a new parallelism dimension is needed.

3.2. The Challenge of MoE Parallelism

Traditional parallelism strategies were designed for dense Transformers. MoE models have fundamentally different computation patterns that require a new parallelism dimension, Expert Parallelism (EP), and create unique challenges when combining EP with existing strategies.

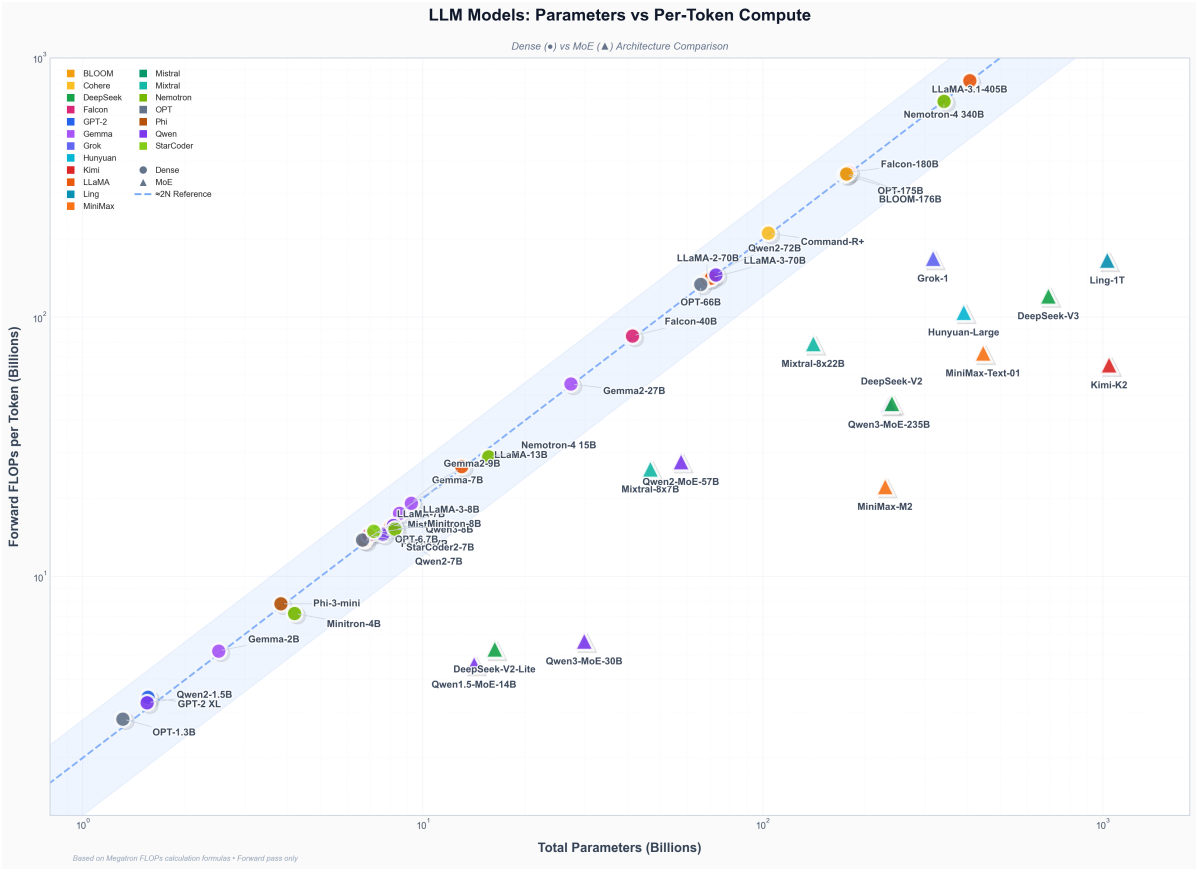


Figure 3: Dense Model vs MoE Model parameter/compute scaling.

3.2.1. The Parallelism Paradox of MoE

Figure 3 illustrates this paradox by plotting total parameters against *forward* FLOPs per token for major LLMs. Dense models (circles) follow the ≈2N reference line, exhibiting a *virtuous cycle* as they scale: increasing parameters requires more GPUs for memory, but more parameters also means more computation per token. Because computation grows with model size, communication takes a smaller share of each step, keeping MFU stable.

MoE models (triangles) break this cycle. They fall significantly below the 2N line, achieving equivalent capability with far fewer FLOPs per token. Consider the fundamental asymmetry:

Model	Total Params	Active Params	Ratio
Llama-70B (Dense)	70B	70B	1:1
DeepSeek-V3 (MoE)	685B	37B	**18:1**

Note: DeepSeek-V3 comprises 671B Main Model weights and 14B Multi-Token Prediction (MTP) Module weights (685B total). The table shows the total model parameters including MTP.

DeepSeek-V3 has 18× more parameters than its active computation suggests, visible in Figure 3 as the gap between DeepSeek-V3's position and where a dense model of equivalent parameters would lie. This creates a *compounding effect*:

1. **Memory grows fast**, forcing distribution across many GPUs: All E experts' parameters, gradients, and optimizer states must reside in memory, and larger top-k further increases activation memory through token replication.

2. **More communication**: Distributing experts via EP introduces all-to-all traffic whose volume scales with K (each token is dispatched to K experts across EP ranks).
3. **But compute stays low**: Per-token FLOPs scale only with N_{active} ($\propto K$), not N_{total} ($\propto E$), leaving insufficient computation to overlap with the growing communication.

The consequence: **MoE training is fundamentally communication-bound** unless parallelism is designed specifically for this asymmetry. This is not a matter of degree; it is a *qualitative difference* from dense models.

3.2.2. Traditional Parallelism Strategies

Dense Transformer training typically combines four parallelism strategies:

Tensor Parallelism (TP) shards weight matrices across GPUs along the hidden dimension [8]. Each GPU computes a partial result, then AllGather or ReduceScatter collectives combine results. TP works well when matrices are large enough to offset communication overhead.

Pipeline Parallelism (PP) splits the model by layers across GPUs [38, 39, 8, 40, 41, 42, 43]. Micro-batches flow through the pipeline, with point-to-point communication between stages. PP introduces pipeline bubbles (idle time at boundaries) and P2P communication overhead, but scales better across nodes than TP.

Data Parallelism (DP) replicates the model across GPUs, with each GPU processing different data batches [44, 45]. Gradients are synchronized via AllReduce. DP is simple but requires each GPU to hold the full model.

Context Parallelism (CP) [46, 47, 48] partitions the input sequence across GPUs along the sequence dimension. Each GPU processes a contiguous chunk of the sequence, with communication required only for attention computation where tokens must attend across chunk boundaries. CP is essential for long-context training where activation memory scales quadratically with sequence length.

Why traditional parallelism fails for MoE:

- **TP on MoE experts**: Expert hidden dimensions are small; applying TP creates even smaller shards, reducing compute efficiency while increasing communication's share of total time. High TP can distribute attention efficiently but *hurts* MoE.
- **PP with many experts**: MoE models have massive parameter counts, requiring many pipeline stages if using PP alone. This creates excessive pipeline bubbles and reduces throughput.
- **DP alone**: DP replicates the full model on each GPU. For a trillion parameter model, this is impossible because DP cannot partition parameters, only data.

3.2.3. Expert Parallelism: The Fifth Dimension

Traditional parallelism partitions *layers* (PP), *weight matrices* (TP), *sequence* (CP), or *data* (DP). But MoE has a unique structure: **experts are independent sub-networks**. This enables a fifth parallelism dimension that partitions *experts themselves* across GPUs.

Expert Parallelism (EP) distributes experts across GPUs [12]. With EP degree equal to E, each GPU holds E/EP experts. The forward pass proceeds as:

1. **Route**: The router selects top-k experts for each token.
2. **Dispatch**: all-to-all communication sends tokens to the GPUs holding their assigned experts.
3. **Compute**: Each GPU processes tokens using only its local experts.
4. **Combine**: all-to-all communication returns results to original GPUs.

EP's unique trade-off:

- **Communication**: all-to-all collectives. Volume scales with token count, not expert count.
- **Compute**: Each GPU runs fewer experts, but each expert processes its *full* hidden dimension.

- **Memory**: Higher EP = fewer experts per GPU = lower memory pressure.

EP provides two key benefits (figure 4). First, grouping tokens from different GPUs to a single expert increases computation intensity, improving GEMM efficiency. Second, all-to-all communication volume remains constant as expert count increases; only the number of GPUs changes.

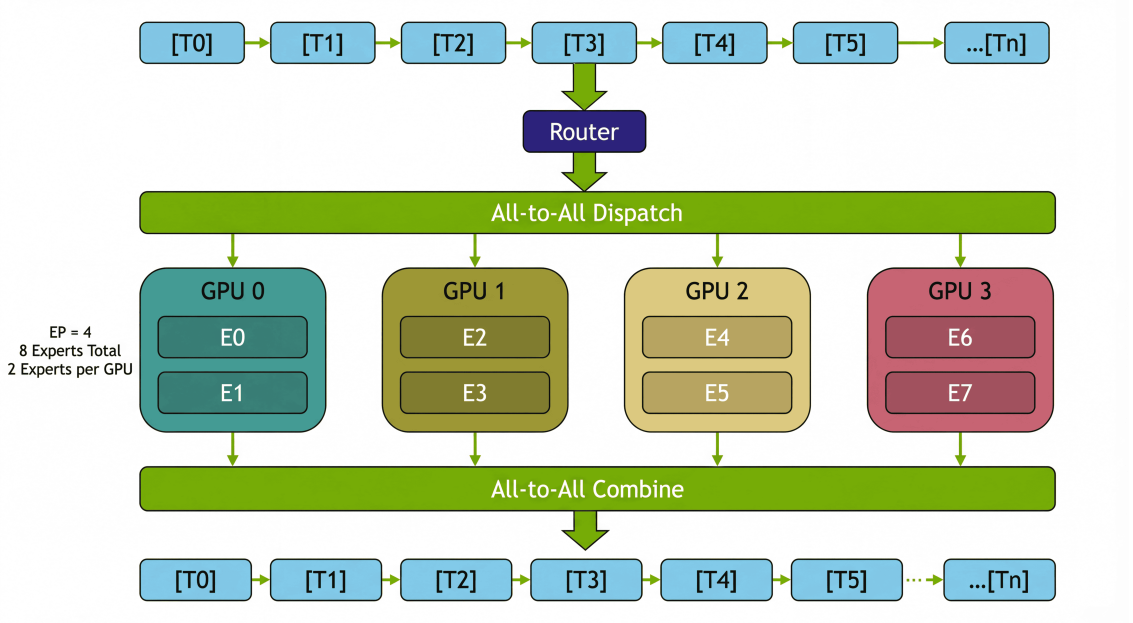


Figure 4: Expert Parallelism (EP) distributes experts across GPUs. The all-to-all communication dispatches tokens to their assigned experts and combines results.

But EP alone is not enough. EP applies only to MoE layers; attention layers have no experts and still require TP, CP, or other strategies. Moreover, both layer types need Pipeline Parallelism to further partition model parameters for large-scale MoE training. This heterogeneity creates the dense-sparse mismatch discussed next.

3.2.4. The Challenges of Combining EP with Traditional Parallelism

A single Transformer block contains **two fundamentally different computation patterns** [49], summarized in Table 2:

Table 2: Contrasting parallelism requirements of attention and MoE layers within a single Transformer block.

Aspect	Attention (Dense)	MoE (Sparse)
Computation	Every token attends to all others	Each token routes to K of E experts
TP	Large QKV matrices benefit from high TP	Small per-expert dimensions make high TP counterproductive
CP	Long sequences benefit from high CP	No sequence dependency; CP is irrelevant
EP	Not applicable (no experts)	Essential for distributing many experts

The Dense-Sparse Mismatch. Traditional frameworks force *one* parallelism configuration for *both* layer types, but their optimal configurations conflict directly: high TP benefits attention but fragments expert shards; high CP helps long-context attention but provides no benefit to MoE layers; and high EP is essential for MoE but irrelevant to attention. Like the parameter-compute mismatch (Section 1.2), this stems from MoE's sparsity, but manifests at the parallelism configuration level: it is a **structural mismatch** between dense and sparse layers' needs, not a tuning problem.

Prior MoE frameworks treat EP as a sub-dimension of DP [12, 4]:

$$\text{World Size} = \text{TP} \times \text{CP} \times \text{PP} \times \text{DP}, \quad \text{where EP} \subseteq \text{DP}$$

This constraint exists because frameworks assumed uniform parallelism: attention layers use (TP, CP, PP, DP), and MoE layers simply carve EP out of the DP group. This design creates three critical challenges:

Challenge 1: Multiplicative GPU Requirements. Traditional frameworks require TP $\times$ CP $\times$ PP $\times$ DP GPUs at minimum. Since EP $\subseteq$ DP, requesting EP=8 forces DP $\geq$ 8. Combined with CP=8 for long sequences, the minimum becomes $1 \times 8 \times 1 \times 8 = 64$ GPUs—even if attention and MoE could theoretically share the same 8 GPUs. This inflates the entry barrier for MoE training.

Challenge 2: Forced Suboptimal Parallelism. Since attention and MoE share the same TP configuration, practitioners must choose between two suboptimal configurations: use high TP (e.g., TP=8) to efficiently shard large attention matrices, which fragments small experts into inefficient shards; or use low TP (e.g., TP=1) to preserve expert computation efficiency, which leaves attention layers underparallelized. Neither option achieves optimal performance for both layer types.

Challenge 3: Cross-Node Communication. With EP constrained within DP, high EP often forces all-to-all communication to cross node boundaries, where bandwidth is 5–10$\times$ lower than NVLink. Meanwhile, CP communication for attention may also span nodes. Without the ability to independently map EP and CP to high-bandwidth domains, communication overhead dominates training time.

These challenges are not independent tuning problems. They stem from the fundamental assumption that all layers must share one parallelism configuration. **The question becomes**: How do we break these constraints while still allowing each layer type to use its optimal parallelism?

3.3. Megatron-Core's Solution: Parallel Folding and Multi-Dimensional Framework

Parallel Folding is Megatron-Core's answer to the dense-sparse mismatch. Rather than forcing attention and MoE layers to share the same parallelism configuration, Parallel Folding **decouples their parallelism mappings**, allowing each layer type to use its optimal topology [26]. This section covers the key concepts and benefits; implementation details such as parallelism transition handling and token dispatch design are described in the companion paper [26].

3.3.1. Decoupled Parallelism Mappings

The core idea is simple: *do not force attention and MoE to share parallelism*. Let each use its optimal configuration independently.

Parallel Folding introduces separate parallelism groups for attention and MoE layers:

- **Attention layers** form groups over TP $\times$ CP $\times$ DP $\times$ PP, optimized for sequence-level dense computation.
- **MoE layers** form groups over ETP $\times$ EP $\times$ EDP $\times$ PP, where ETP (Expert Tensor Parallelism) and EDP (Expert Data Parallelism) are MoE-specific dimensions.

The sole constraint: **Pipeline Parallelism (PP) must remain consistent** across both layouts to ensure correct gradient flow through the model.

3.3.2. Complete Multi-Dimensional Parallelism Stack

With Parallel Folding as the foundation, Megatron-Core orchestrates five parallelism dimensions:

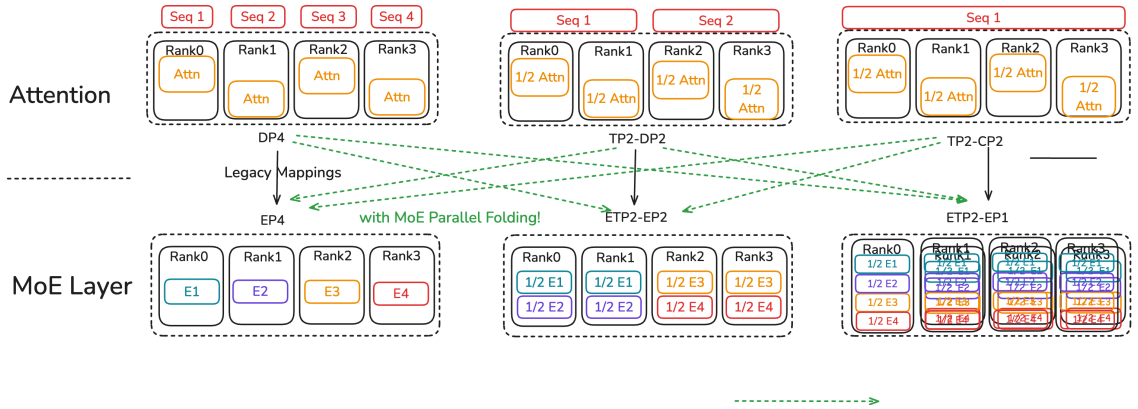


Figure 5: Parallelism mappings: traditional constraints vs. MoE Parallel Folding decoupling.

Dimension	Applies To	Purpose
TP (Tensor)	Attention	Shard large QKV/projection matrices
CP (Context)	Attention	Distribute long sequences
DP (Data)	Attention	Process different batches
PP (Pipeline)	Both	Split model by layers (must be consistent)
EP (Expert)	MoE	Distribute experts across GPUs
ETP (Expert Tensor)	MoE	Shard within experts (rarely used)
EDP (Expert Data)	MoE	Replicate experts for throughput

Key configuration principles:

- **Attention layers**: Optimize for large matrices (high TP) and long sequences (high CP).
- **MoE layers**: Optimize for many small experts (high EP, typically ETP=1).
- **PP**: Must remain consistent across both to ensure correct data flow.

Beyond the parallelism strategies described above (figure 6), Megatron-Core's Parallel Folding framework also integrates Distributed Optimizer and FSDP with EP support to further reduce memory footprint:

Distributed Optimizer + EP. Only weights and gradients for local experts reside on each rank; optimizer states are sharded among replicas of the same expert (via EDP). This removes redundant optimizer memory for non-local experts and confines gradient synchronization to minimal groups.

FSDP + EP. For even greater memory efficiency, Megatron-Core's custom FSDP (Megatron-FSDP) fully shards parameters, gradients, and optimizer states across data/expert groups via a dual DeviceMesh architecture, reducing memory footprint while overlapping AllGather and ReduceScatter with computation. It is compatible with TP/EP/CP and mixed precision (BF16, FP8, FP4). See section 4.1.7 for the full design.

3.3.3. Benefits of Parallel Folding

As illustrated in figure 5, MoE Parallel Folding eliminates the EP $\leq$ DP limitation by allowing EP to "fold" across arbitrary sub-groups of the attention parallelism configuration. This provides four key advantages:

1. **Breaks the EP $\leq$ DP constraint**: EP can now exceed DP by folding across TP$\times$CP groups. Consider attention configured with TP=4, CP=2, DP=8, PP=4 (total 256 GPUs):
   - Traditional: EP $\leq$ DP = 8, so maximum EP is 8.
   - With Parallel Folding: MoE uses ETP=1, EP=64, EDP=1 (same PP=4). EP "folds" across the TP$\times$CP$\times$DP groups, enabling 8$\times$ higher expert parallelism while attention layers maintain their

optimal TP=4, CP=2 configuration.

2. **Reduces minimum GPU requirements**: Traditional configurations with CP=8, EP=8 require at least 64 GPUs. With Folding, CP and EP share the same GPU group, requiring only 8 GPUs.
3. **Enables independent optimization**: Attention can use high TP for large matrices while MoE uses ETP=1 for full expert width and better GEMM efficiency.
4. **Keeps high-bandwidth communication in NVLink domain**: Both CP (for attention) and EP (for MoE) all-to-all communication can remain within the NVLink-connected GPU group, avoiding slower cross-node transfers.

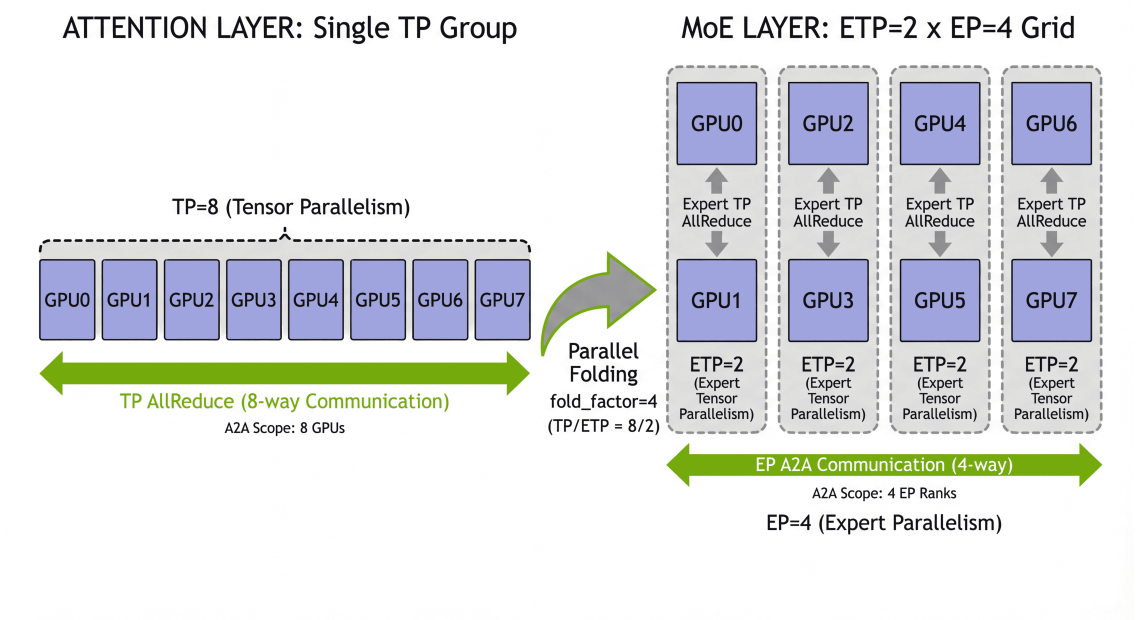


Figure 6: Parallel Folding: decoupled attention and MoE parallelism mappings.

3.3.4. Summary

This section addressed the *distribution challenge* for MoE training. The core problem is the *dense-sparse mismatch*, rooted in MoE's sparsity: attention and MoE layers have conflicting optimal parallelism configurations, yet prior frameworks forced them to share one configuration. **MoE Parallel Folding** solves this by decoupling parallelism mappings, allowing attention to use high TP/CP while MoE uses high EP independently. Together with Distributed Optimizer and FSDP, Megatron-Core orchestrates all parallelism dimensions into a unified framework that scales MoE training to thousands of GPUs.

However, scalable distribution alone does not guarantee training efficiency. The three fundamental barriers identified in Section 1.2, the Memory, Communication, and Compute Efficiency Walls, must still be addressed. The following section presents Megatron-Core's solutions to break through these walls.

4. Scaling MoE: Breaking the Memory, Communication, and Compute Efficiency Walls

Section 3 established *how* to distribute MoE training across thousands of GPUs. This section addresses *how to make it efficient*. The challenges are significant: Section 1.2 identified three fundamental barriers (the Memory Wall, Communication Wall, and Compute Efficiency Wall) that emerge from MoE's sparsity-driven

parameter-compute mismatch. These walls are not merely inconveniences; without addressing them, training large-scale MoE models is either infeasible (memory) or prohibitively slow (communication, compute).

Why These Three Walls? These barriers emerge from the fundamental resources of GPU-accelerated training: memory stores parameters and activations, interconnects transfer data between devices, and compute units execute operations. Every training system must acquire sufficient memory, move data where it is needed, and perform computations efficiently. Other potential bottlenecks, such as storage I/O for checkpointing or host-side data loading, exist but operate at timescales that allow them to be overlapped with training iterations. The three walls represent the hard constraints on every forward-backward pass.

The Walls Interact. These challenges are not independent; they form a tightly coupled system where fixing one wall can expose or worsen another. Consider a concrete optimization trajectory: A team training a 1000B MoE model first encounters the memory wall: activations alone exceed GPU capacity. They enable activation recomputation (Section 4.1), trading memory for computation by regenerating activations during the backward pass. Memory constraints resolved, profiling reveals that all-to-all communication now dominates; the compute wall was hiding the communication wall. They implement communication-computation overlap (Section 4.2), pipelining all-to-all transfers with expert execution. But fine-grained experts complete faster than communication, limiting overlap effectiveness. They enable reduced-precision training (Section 5) to reduce memory footprint, allowing larger batch sizes that provide more computation to hide communication. This works, but introduces quantization kernels that fragment the execution stream and increase kernel launch overhead. The system becomes host-bound: GPUs idle between kernel launches. They deploy CUDA Graphs (Section 4.3) to reduce launch overhead, but graphs require static tensor shapes, conflicting with dropless routing's dynamic expert assignments. Each solution ripples through the system, demanding awareness of all three walls simultaneously. Addressing walls in isolation leads to suboptimal solutions; effective optimization requires treating memory, communication, and compute as a unified system.

Megatron-Core addresses these challenges through integrated system design. The remainder of this section presents solutions organized by the wall they primarily address:

- **Section 4.1: Breaking the Memory Wall.** How do we fit training within GPU memory constraints? This section covers activation management, recomputation strategies, offloading, and distributed parameter storage.
- **Section 4.2: Breaking the Communication Wall.** How do we minimize time lost to inter-device communication? This section covers optimized dispatchers, communication-computation overlap, and pipeline integration.
- **Section 4.3: Breaking the Compute Efficiency Wall.** How do we keep GPUs saturated with work? This section covers kernel fusion, CUDA Graphs, and sync-free execution for dropless MoE.

Reduced-precision training in FP8/FP4, which provides benefits across all three walls simultaneously, is covered separately in Section 5. Long-context MoE training, which changes the optimization balance when attention dominates computation, is addressed in Section 6.

4.1. Breaking the Memory Wall

Memory is the first hard constraint in MoE training: if the combined footprint of parameters, optimizer states, and activations exceeds GPU capacity, training cannot proceed. As model and expert counts grow, memory requirements grow quickly, making memory optimization essential for practical training. Understanding *where* memory is consumed is essential for effective optimization.

4.1.1. Memory Anatomy: Sources of Memory Consumption

MoE models typically consume substantially more memory than equivalent dense models, owing to several distinct sources of overhead. Consider DeepSeek-V3 trained with BF16 precision using a PP4 $\times$ VPP4 $\times$ EP64 configuration across 256 GPUs. Table 3 presents the per-GPU memory breakdown, revealing a total requirement of 199.5 GB, well beyond the 80 GB capacity of an H100 GPU without optimization.

Table 3: Memory breakdown per GPU for DeepSeek-V3 with BF16 training (PP4 $\times$ VPP4 $\times$ EP64, 256 GPUs).

Component	Memory per GPU	Optimization Techniques
Weights & Gradients	36.4 GB	PP, EP, or TP sharding
Main Weights & Optimizer States	32.1 GB	Distributed optimizer, BF16 moments
Activations	131.0 GB	Low Precision, Recomputation, Offloading
Total	**199.5 GB**	

Three components contribute to this footprint:

Weights and Gradients (36.4 GB). All E expert parameters must stay in memory even though only K activate per token. DeepSeek-V3's 256 experts with top-8 routing illustrate this: 685B parameters but only 37B activated per token, an 18$\times$ gap.

Optimizer States (32.1 GB). Adam stores momentum and variance per parameter, tripling parameter memory in FP32. Mixed-precision training with BF16 moments reduces this overhead but does not eliminate it.

Activations (131.0 GB). The largest memory consumer, exceeding weights and optimizer states combined. MoE activations scale with layer depth, hidden dimension, and top-k, as well as batch size and sequence length. This makes activations the primary optimization target.

> **Note**
>
> Activations dominate memory consumption in large-scale MoE training, often exceeding the combined memory of weights, gradients, and optimizer states. This makes activation memory optimization the highest priority for enabling larger batch sizes or more flexible parallelism configurations.

Additional MoE-specific factors add to memory pressure: dynamic routing can create load imbalance where certain experts temporarily receive too many tokens, and tokens must often be padded to fit efficient computation kernels, consuming memory beyond the actual data. These are activation memory costs that can be reduced through the recomputation techniques discussed below. Load imbalance is further addressed by ECHO (Section 4.3.7), which dynamically clones popular experts to balance token distribution across ranks.

Model architecture cannot be changed to reduce memory, so optimization must target how data is stored and managed. Four complementary strategies address memory constraints:

1. **Reduce storage precision.** Lower-precision formats (FP8/FP4 instead of BF16) reduce activation memory with minimal impact on accuracy. Memory-Efficient Permutation eliminates redundant intermediate tensors entirely.
2. **Trade computation for storage.** Activation recomputation [50, 27, 51] discards intermediate results during the forward pass and regenerates them during backward, exchanging compute cycles for memory capacity.
3. **Offload to host memory.** When GPU memory is exhausted, activations can be transferred to CPU memory during forward pass and retrieved during backward, trading PCIe bandwidth for GPU memory [52].

4. **Distribute across devices.** Fully Sharded Data Parallel (FSDP) [53, 33, 54, 55] partitions parameters, gradients, and optimizer states across data-parallel ranks, enabling training of models that exceed single-device capacity.

The following subsections present these techniques in two groups. First, activation memory optimizations ordered by overhead: zero-overhead (Memory-Efficient Permutation), precision trade-offs (FP8/FP4 vs. BF16), computational trade-offs (recomputation), and bandwidth trade-offs (offloading). Then, optimizations targeting weights and optimizer states: precision-aware optimizer and distribution strategies (FSDP).

4.1.2. Memory-Efficient Permutation: Zero-Overhead Activation Reduction

The most desirable memory optimizations are those with no computational overhead. Memory-Efficient Permutation achieves exactly this by eliminating redundant intermediate tensors through a simple algebraic rearrangement. The technique is zero-overhead because it merely changes *when* router probabilities are applied, not *whether* they are applied; the pre-activation buffers required for the backward pass would be stored anyway in conventional implementations.

As described in Section 2.1, the router assigns each token to its top-k experts with a learned routing weight, and the weighted expert outputs are combined to produce the final result.

Consider a token $\mathbf{x}$ routed to its top-k experts. Let $\mathcal{T}(\mathbf{x}) \subset \{1, \ldots, E\}$ denote the selected expert set with routing weights $\{p_i\}_{i \in \mathcal{T}(\mathbf{x})}$. Each expert E_i is a two-layer MLP with weight matrices $\mathbf{W}_1^{(i)}, \mathbf{W}_2^{(i)}$ and nonlinear activation ϕ (e.g., SwiGLU):

$$E_i(\mathbf{x}) = \mathbf{W}_2^{(i)}\, \phi\big(\mathbf{W}_1^{(i)}\mathbf{x}\big).$$

In the **standard formulation**, routing weights are applied *after* expert computation:

$$\mathbf{y} = \sum_{i \in \mathcal{T}(\mathbf{x})} p_i \cdot \mathbf{W}_2^{(i)}\, \phi\big(\mathbf{W}_1^{(i)}\mathbf{x}\big). \tag{1}$$

Memory-Efficient Permutation absorbs p_i into the activation, applying it *before* the second linear layer:

$$\mathbf{y} = \sum_{i \in \mathcal{T}(\mathbf{x})} \mathbf{W}_2^{(i)}\, \big(p_i \cdot \phi\big(\mathbf{W}_1^{(i)}\mathbf{x}\big)\big). \tag{2}$$

When the experts have no bias terms, $\mathbf{W}_2^{(i)}$ is a pure linear map and scalar multiplication commutes: $p_i \cdot \mathbf{W}_2^{(i)}\mathbf{h} = \mathbf{W}_2^{(i)}(p_i \cdot \mathbf{h})$ for any vector $\mathbf{h}$, so equation 1 and equation 2 are mathematically equivalent.

This rearrangement reduces peak memory by eliminating saved tensors needed for the router's backward pass. Let $\mathbf{z}_i = \mathbf{W}_1^{(i)}\mathbf{x}$ denote the pre-activation input. In the standard formulation, computing $\partial \mathcal{L}/\partial p_i$ requires retaining each expert output $E_i(\mathbf{x})$ throughout the backward pass. In the memory-efficient formulation, p_i multiplies $\phi(\mathbf{z}_i)$ directly, so $\partial \mathcal{L}/\partial p_i$ only depends on $\phi(\mathbf{z}_i)$, which a fused backward kernel recomputes from $\mathbf{z}_i$ on the fly. Since $\mathbf{z}_i$ must already be saved for the SwiGLU activation's own backward pass regardless of whether Memory-Efficient Permutation is used, no additional buffers are introduced, yielding a net reduction in peak memory with zero computational overhead. Figure 7 illustrates this transformation.

For DeepSeek-V3 (Table 3), Memory-Efficient Permutation saves approximately 26.3 GB of activation memory per GPU, a significant reduction with zero computational cost.

4.1.3. Reduced-Precision Training: FP8/FP4 for Activation Memory Reduction

Storing activations in FP8/FP4 instead of BF16 reduces their memory footprint with minimal impact on model quality.

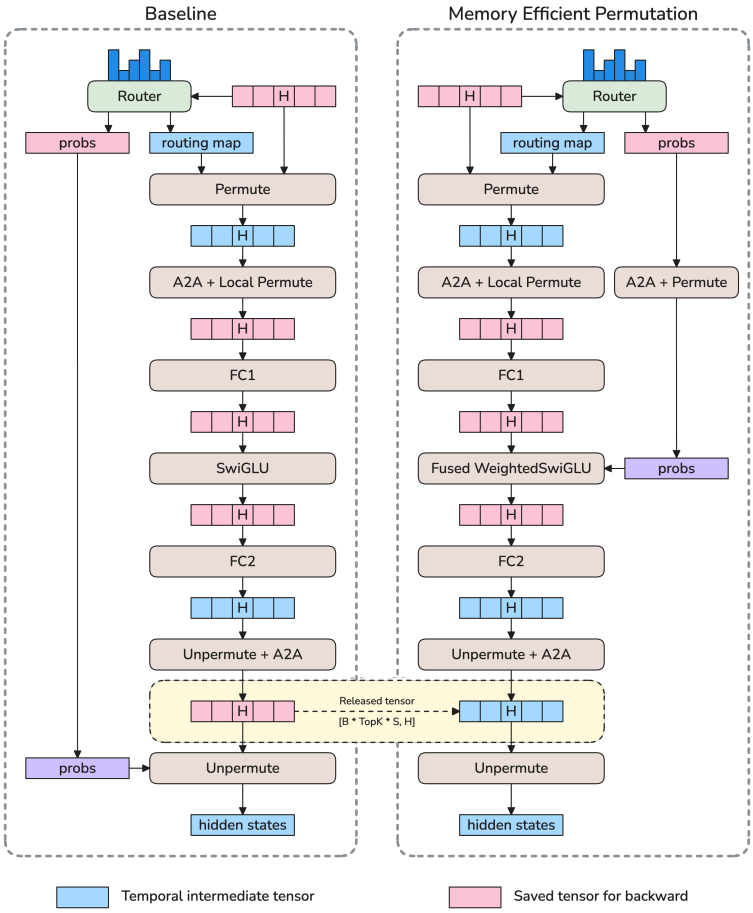


Figure 7: Memory-Efficient Permutation.

During the forward pass, the input to each linear layer must be retained to compute weight gradients in the backward pass. In Transformer models, these linear layer inputs make up most of the activation memory: attention projections (Q, K, V, output) and MLP layers (including expert MLPs in MoE) each save their inputs for gradient computation. By storing these input tensors in FP8/FP4 instead of BF16, each tensor's memory footprint is reduced by 50%/75%.

For the DeepSeek-V3 configuration in Table 3, enabling FP8 training reduces approximately 16 GB of activation memory, representing roughly 12% of the 131 GB activation budget. This corresponds to halving approximately 32 GB of linear layer inputs that are eligible for FP8 storage. The remaining activations (attention scores, normalization intermediates, routing tensors) either require higher precision for numerical stability or are not stored across the forward-backward boundary. This reduction is orthogonal to other activation optimizations such as recomputation and offloading, allowing them to be combined for cumulative savings.

For details on reduced-precision training mechanisms, including recipes, quantization strategies, and MoE-specific optimizations, see Section 5.

4.1.4. Recomputation: Trading Compute for Memory

Activation recomputation (or activation checkpointing) is a well-established technique that discards intermediate activations during forward pass and recomputes them during backward pass [50]. However, trivial full-layer recomputation can add ∼33% computational overhead, and for MoE layers it is even more costly because recomputing expert computation also re-triggers EP all-to-all communication. Megatron-Core introduces

fine-grained recomputation that targets only the most memory-intensive yet computationally cheap operations, achieving significant memory savings with minimal overhead [27].

Two techniques compose to form this strategy (figure 8):

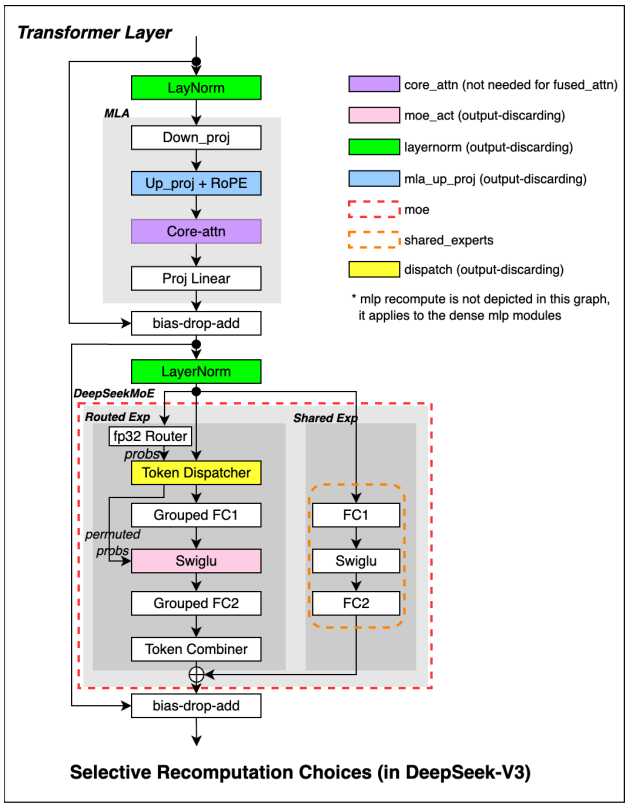


Figure 8: Selective Recomputation.

Granular recomputation. Rather than applying activation checkpointing to large, monolithic sections, users specify exactly which computations to recompute in the backward pass. For instance, one may recompute only the activation functions within expert MLPs, the LayerNorm modules, or the up-projection within Multi-Latent Attention (MLA). By recomputing individual operations or submodules, significant memory savings are achievable with only a modest increase in computational workload, as only the selected portions require recalculation (less than 5% additional compute overhead).

Output-discarding recomputation. Conventional activation checkpointing workflows pass the outputs of checkpointed modules to downstream layers, storing these outputs for potential use in backpropagation. However, since these outputs will be recomputed during the backward pass, this storage is redundant. To avoid this, Megatron-Core MoE promptly releases the outputs of checkpointed modules after they are consumed by subsequent layers. During the backward pass, the outputs are restored from recomputed results. This strategy ensures that memory is not unnecessarily reserved for activations that can be cheaply restored, reducing memory footprint without compromising gradient correctness or training dynamics.

Table 4 summarizes the memory reduction achieved by different recomputation targets for the DeepSeek-V3 configuration in Table 3.

4.1.5. Fine-grained Activation Offloading

When GPU memory remains insufficient even after precision and recomputation optimizations, *offloading* activations to CPU memory provides additional capacity. Unlike recomputation which trades compute cycles for

Table 4: Memory reduction per GPU from fine-grained recomputation for DeepSeek-V3 (PP4 × VPP4 × EP64, 256 GPUs).

Recomputation Target	Memory Saved per GPU
MLA Up-Projection	30.4 GB
Activation Function (SwiGLU)	3.8 GB
LayerNorm	8.2 GB
Total	**42.4 GB**

memory, offloading trades PCIe bandwidth instead. The challenge is hiding transfer latency behind computation so that offloading appears "free".

Motivation Fine-grained MoE models have extreme parameter inflation: DeepSeek-V3 activates only 37B per token (18× ratio, Table 3); Kimi-K2 reaches 1T total with 32B active (31×). This parameter-compute mismatch (Section 1.2) is especially acute for offloading because activation memory does not decrease with Expert Parallelism (EP) or Pipeline Parallelism (PP); these strategies reduce parameter memory, not activation memory.

Transformer Engine provides layer-level offloading, but this coarse granularity limits effectiveness. Different modules within a layer have significantly different memory-to-compute ratios: LayerNorm activations are small and cheap to recompute, while `expert_fc1` inputs are large but computationally expensive. Layer-level offloading cannot distinguish these cases. It either offloads everything (wasting bandwidth) or nothing (wasting memory).

High-Level Idea: Overlap and Prefetch. The GPU's Copy Engine and Compute Engine operate independently. When a module's computation time exceeds its activation transfer time, the D2H copy can run concurrently with subsequent computation at zero cost. Figure 9 illustrates the stream overlap mechanism for both forward and backward passes.

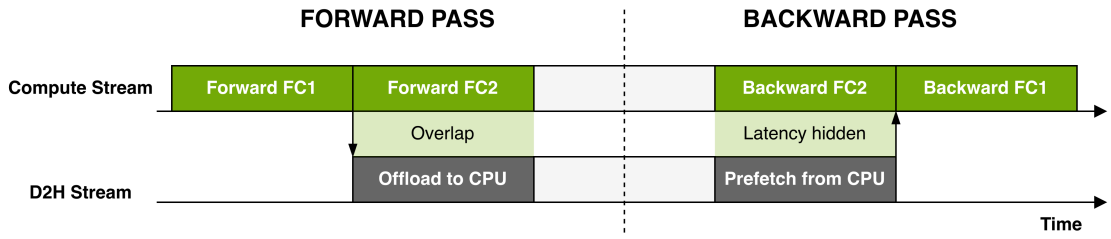


Figure 9: Fine-grained activation offloading: stream overlap for forward and backward passes.

Forward Pass. During forward propagation, input activations are offloaded to CPU immediately after module computation, running in parallel with the next module's computation via a dedicated D2H stream. One exception: the last layer's activations are not offloaded because they are needed immediately during backward, so no computation is available to hide the transfer latency.

Backward Pass. During backward propagation, activation reloading follows a *Layer-Staggered Reload* pattern: the system reloads the activation of the same module (e.g., `expert_fc1`) from the *next* layer while computing gradients for the current layer. The reload occurs *after* backward completes for each module, so only one activation per module type resides in GPU memory at any time, avoiding the need to double activation storage. This is essential when a single module has very large activations where 2× memory footprint would cause unexpected memory peaks.

PP and VPP Handling. For PP/VPP scenarios, a `ChunkOffloadHandler` manages the offloading/reloading logic for each (microbatch, VPP stage) combination, identical to the PP=1 case. The main challenge is managing data dependencies and execution order between virtual pipeline chunks. Handlers are enqueued into a deque with VPP stages in reverse order (FILO) and microbatches in normal order (FIFO). During backward, the popped handler automatically matches the VPP chunk execution order, correctly pairing offloaded activations with their corresponding backward passes.

Key Technical Features.

- **Module-level granularity:** Users specify which modules to offload via `--offload-modules`, enabling mixed strategies. Lightweight modules (LayerNorm, activations) use recomputation while expensive ones (attention, experts) use offloading.
- **Asynchronous transfers:** Dedicated D2H/H2D CUDA streams run transfers in parallel with computation; CUDA events coordinate synchronization only when necessary.
- **Recomputation integration:** Combines with fine-grained recomputation (section 4.1.4). For `moe_act`, both strategies apply: recompute the activation while offloading its input, releasing the entire `fc1`→`act` chain.
- **CUDA Graphs compatibility:** Uses external events rather than stream synchronization, allowing offloading modules to be hidden by computations outside the CUDA graph.
- **Full-scenario compatibility:** Supports PP=1/PP>1/VPP>1, all precisions (BF16/FP8/MXFP8/NVFP4), 1F1B with all-to-all overlap, and MoE/MLA architectures.

Peak Memory Advantage over Full Recomputation. Full recomputation stores each layer's *input* on GPU while releasing intermediate activations; backward recomputes intermediates from these stored inputs. For an L-layer model, GPU peak memory is $L \times$ layer_input $+ 1 \times$ layer_intermediate. In contrast, offloading moves layer inputs to CPU; during backward, each input is reloaded just before use and released immediately after. This reduces GPU peak memory up to $1 \times$ layer_input $+ 1 \times$ layer_intermediate, independent of model depth. For deep models (e.g., 60+ layers), this represents a fundamental memory advantage that full recomputation cannot achieve.

Performance. Fine-grained offloading and recomputation work together as complementary strategies (figure 10): lightweight operations like LayerNorm use recomputation, while expensive modules like attention and experts use offloading. Asynchronous transfers overlap with computation to hide PCIe latency. Table 5 shows results across multiple configurations. Fine-grained offloading reduces memory by 10–18% with only 1.6–2% throughput overhead. In the case of training Qwen3-235B, offloading enables reducing Tensor Parallelism degree, which *improves* throughput by 15.0% with nearly the same memory cost.

Table 5: Memory and throughput impact of fine-grained activation offloading.

Model & Config	**Baseline**	**+Offload**	**Mem Δ**	**Throughput Δ**
DeepSeek-V3 full TP1PP8EP32VPP4, MXFP8	169 GB 945 TF/s	151 GB 930 TF/s	−10.7%	−1.6%
Qwen3-235B TP2→TP1 + EP16→EP64	172 GB 800 TF/s	175 GB 920 TF/s	+1.7%	+15.0%

4.1.6. Weight and Optimizer Optimization: Low-Precision Storage and Offloading

The preceding optimizations target activation memory, which dominates the memory breakdown. However, Table 3 shows that main weights and optimizer states consume 32.1 GB per GPU, representing 16% of the total memory footprint. For models with hundreds of billions of parameters, this component becomes a significant

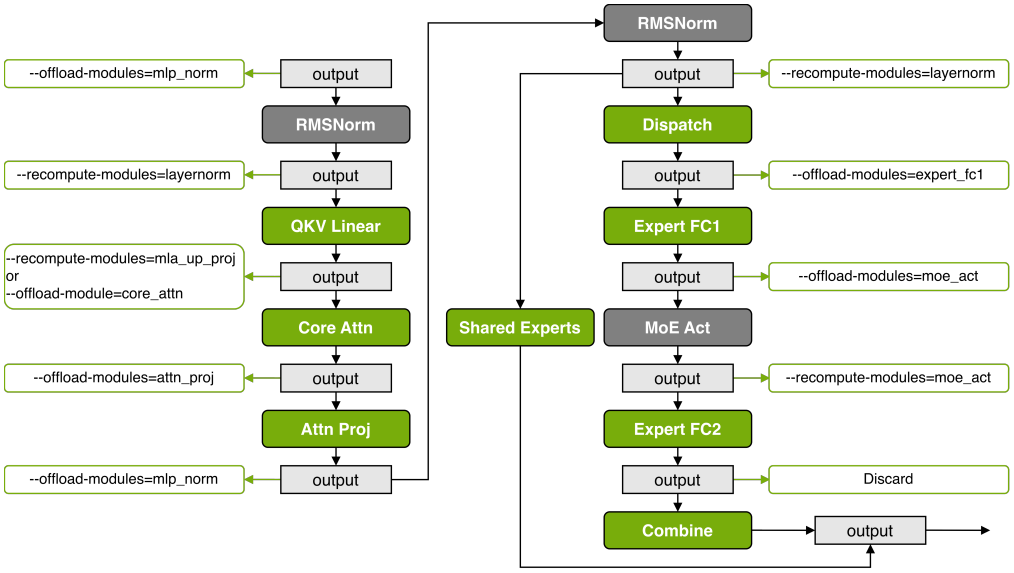


Figure 10: Fine-grained offloading and recomputation: complementary memory optimization strategies.

optimization target. Megatron-Core provides two techniques: precision-aware optimization that reduces storage requirements, and CPU offloading that moves inactive state off-GPU.

Precision-Aware Optimizer. The Adam optimizer [56] maintains two state tensors per parameter: first moment (`exp_avg`) and second moment (`exp_avg_sq`). Traditional implementations store these in FP32, consuming 8 bytes per parameter, which poses a significant memory bottleneck for large-scale training. The key insight is that optimizer states can tolerate lower storage precision without affecting convergence, provided that the actual update computation remains in higher precision.

The precision-aware optimizer decouples storage precision from computation precision [57]. The first and second moments can be stored in BF16 (2 bytes each) or even FP8 (1 byte each), reducing per-parameter storage from 8 bytes to 4 bytes (BF16) or 2 bytes (FP8). During each optimizer step, these lower-precision states are dynamically cast to FP32 within TransformerEngine's FusedAdam kernel; the update is computed with full-precision arithmetic to maintain numerical stability.

The implementation provides four configurable precision levels: main gradients, main parameters, first moment, and second moment precision. In typical configurations, main parameters and gradients remain in FP32 to ensure high-quality gradient updates, while moment estimates are stored in BF16 [58]. This achieves approximately 50% reduction in optimizer state memory (roughly 10–12 GB savings from the 32.1 GB budget in Table 3) with negligible impact on training dynamics.

When combined with the distributed optimizer, which shards optimizer states across data-parallel ranks of size d, the theoretical memory requirement per rank is further reduced. In DeepSeek-V3 training with BF16 moments, the memory consumption per parameter per DP rank decreases from $6 + 12/d$ bytes to $6 + 8/d$ bytes.

State Offloading. During forward and backward passes, optimizer states occupy GPU memory but remain inactive; offloading reclaims this memory for other operations. Optimizer state offloading keeps the optimizer step on GPU but transfers optimizer states (`exp_avg`, `exp_avg_sq`) and master weights to CPU after `optimizer.step()` and reloads them before the next step. This approach uses GPU compute while reclaiming memory during forward and backward passes.

State offloading is particularly effective on systems with high-bandwidth interconnects. On GB200 with NVLink-C2C, asynchronous transfers overlap with computation, and pinned memory enables maximum bandwidth

utilization. For DeepSeek-V3, state offloading saves 15–20 GB of GPU memory (47–62% of the 32.1 GB optimizer and weight budget) with only 0.1–0.2 seconds per iteration overhead.

These two techniques offer trade-offs that work well together. The precision-aware optimizer incurs no performance overhead since the FP32 cast occurs within the fused Adam kernel; it reduces optimizer state memory by up to 50%. CPU offloading saves more memory (all the optimizer state and master weights) but introduces modest transfer overhead. Importantly, the techniques compose well: using precision-aware storage with BF16 moments reduces the state size that must be offloaded, thereby decreasing transfer time and making offloading more practical even on systems without the highest-bandwidth interconnects.

4.1.7. FSDP for MoE

Fully Sharded Data Parallelism (FSDP) [53, 33] shards model parameters, gradients, and optimizer states across data-parallel ranks so that each GPU holds only a local shard. Expert parameters often dominate memory in MoE models, making FSDP a natural complement to Expert Parallelism. However, the two must compose correctly.

Why FSDP for MoE? Megatron-FSDP composes seamlessly with multiple parallel strategies, including Expert Parallelism (EP), Tensor Parallelism (TP), and Context Parallelism (CP). For large-scale MoE models, combining FSDP with EP turns the general advantages of FSDP into concrete benefits for expert-heavy workloads.

- **Reduced Memory and Communication**: EP assigns each GPU a subset of experts, and FSDP shards those local experts across the expert data-parallel (EDP) group rather than the full DP group. Per-GPU memory and collective volume both scale with EDP size instead of total DP size, so MoE models can support more experts or larger batches under the same hardware budget.
- **PP-free Flexibility**: FSDP+EP avoids several engineering pain points of pipeline parallelism, including uneven PP/VPP stage balancing, placement of output and MTP layers in DeepSeek-style models, and partitioning vision encoders in multimodal settings. Configuration reduces to choosing EP size and FSDP sharding degree, without complex pipeline-stage design.
- **Broad Model Compatibility**: Megatron-FSDP supports two model implementation paths: models built with Megatron-Core's own modules and PyTorch-native models composed from standard `torch.nn` modules. Both paths receive the same FSDP+EP sharding, communication optimizations, and checkpointing support. For interoperability with the HuggingFace ecosystem, Megatron-Bridge handles online weight conversion between HuggingFace models and Megatron FSDP, so users can load a pretrained HuggingFace checkpoint, train with FSDP+EP, and export weights back without manual format conversion.

FSDP+EP: Dual DeviceMesh Design The core design challenge is that dense and expert layers need different sharding scopes. Dense modules (attention, normalization) benefit from FSDP sharding over the full DP group. EP partitions expert modules first, so each GPU holds only its assigned experts. FSDP should therefore shard within each expert's data-parallel (EDP) group, not globally.

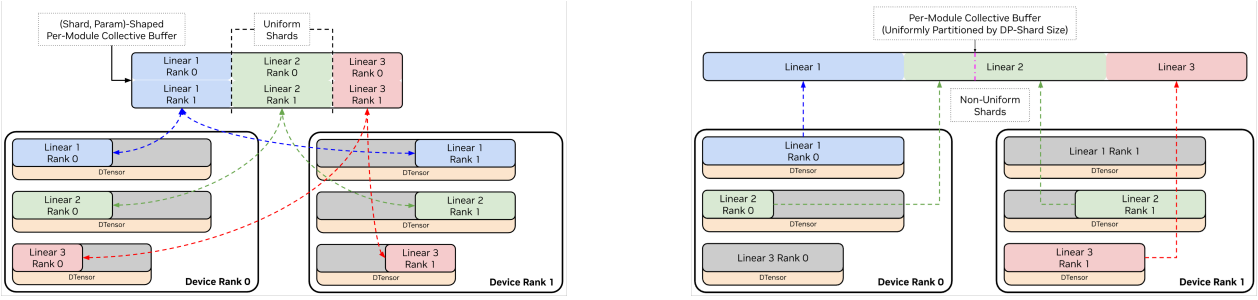
Megatron-FSDP resolves this through a *dual DeviceMesh* architecture. A primary DeviceMesh governs DP-Shard, DP-Outer, TP, and CP for dense modules. An auxiliary Expert DeviceMesh manages EP modules with FSDP scoped to the EDP dimension. Each transformer layer routes its sub-modules to the appropriate mesh automatically. Attention and normalization use the primary mesh, while MoE expert FFN layers use the Expert DeviceMesh. As a result, AllGather and ReduceScatter for expert parameters stay within small EDP groups instead of spanning all DP ranks.

For large-scale deployments, this design extends to *Hybrid Sharded Data Parallelism* (HSDP), which adds an outer DP replication dimension. HSDP fully shards parameters within a subset of ranks and replicates across subsets, bounding AllGather to the intra-group size. The dual DeviceMesh keeps separate outer-DP groups

for expert and non-expert parameters, exploiting the bandwidth gap between NVLink (intra-node) and the scale-out interconnect (inter-node).

Zero-Copy Communication The dual DeviceMesh determines which ranks participate in each FSDP collective, but collectives themselves still carry overhead from buffer management and data copying. Megatron-FSDP eliminates this overhead through two optimizations.

1. Non-uniform sharding: Standard FSDP2 shards each parameter independently along its primary dimension, producing uniform per-parameter shards (Figure 11a). Megatron-FSDP instead flattens and concatenates all parameters within a module, then applies non-uniform sharding across devices (Figure 11b). Shard boundaries then align with communication buffer layouts, and collectives read directly from flattened storage without redundant copying. In Llama3 405B training, this reduces communication overhead by roughly 10%.



(a) FSDP2 per-parameter uniform sharding.

(b) Megatron-FSDP per-module non-uniform sharding.

Figure 11: Comparison of sharding strategies: (a) FSDP2 shards each parameter uniformly; (b) Megatron-FSDP flattens per-module and shards non-uniformly, aligning with communication buffers.

2. Persistent double buffers with NCCL User Buffer Registration: Baseline FSDP frequently allocates and frees communication buffers, and NCCL copies data between user buffers and its internal staging area on every collective. Megatron-FSDP pre-allocates two persistent buffers at training start and cycles between them (Figure 12), eliminating allocation churn. Megatron-FSDP then registers these buffers with NCCL via User Buffer Registration (UBR), so NCCL reads and writes the pre-registered memory directly without intermediate copies. The combined effect is true zero-copy communication. On NVLink systems, the SM footprint of communication kernels drops from 8–32 SMs to 1–4 SMs. On SHARP-enabled InfiniBand, network switches handle reductions and free GPU SMs entirely.

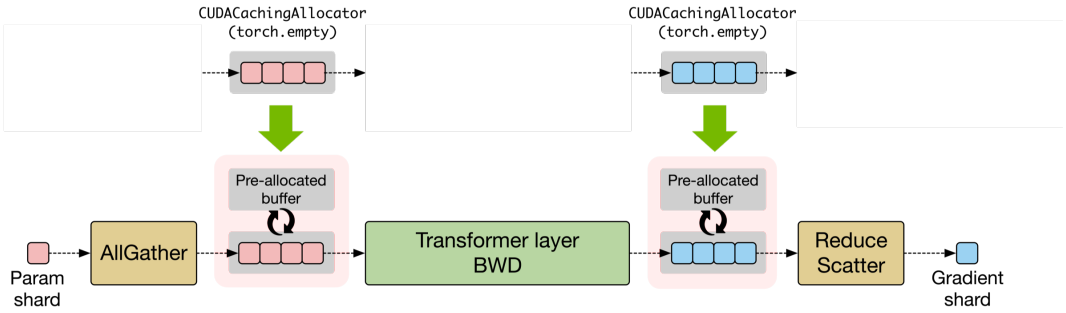


Figure 12: Persistent double-buffer design: two pre-allocated buffers are cycled across FSDP collectives, eliminating allocation overhead and enabling NCCL User Buffer Registration.

Computation–Communication Overlap Even with zero-copy collectives, AllGather and ReduceScatter still occupy the network while GPUs wait for parameters or gradient synchronization. Megatron-FSDP pipelines these collectives with computation using dedicated CUDA streams: AllGather for the next FSDP unit is issued while

the current unit's forward or backward pass is still executing, and ReduceScatter for gradients runs concurrently with the backward pass of subsequent layers. Increasing the micro-batch size extends the computation window available for hiding communication, though at the cost of higher memory usage. Users can tune this trade-off through the `overlap_param_gather` and `overlap_grad_reduce` flags.

4.1.8. Summary

Table 6 summarizes the memory optimization techniques described in this section and their primary targets.

Table 6: Summary of memory optimization techniques.

Technique	Memory Target	Trade-off
Reduced-Precision Training	Activations	Numerical Precision and CPU Overhead
Memory-Efficient Permutation	Activations	-
Fine-grained Recomputation	Activations	Compute Overhead
Fine-grained Offloading	Activations	CPU Overhead and Non-Overlapped Copy
Precision-aware Optimizer	Optimizer States	Numerical Precision
FSDP (with EP)	Params + Optimizer	Communication Overhead

These memory optimizations complement each other: Memory-Efficient Permutation eliminates redundant storage with zero overhead; FP8/FP4 activations reduce precision with minimal quality impact; fine-grained recomputation offers favorable compute-memory trade-offs for specific modules; activation offloading provides additional headroom when other techniques are insufficient; low-precision storage and state offloading reduce weight and optimizer memory; and FSDP enables scaling beyond single-device capacity. Together, they reduce memory from a blocking barrier to a manageable constraint.

Memory optimization is not a one-time gate that, once passed, can be forgotten. For large-scale MoE models, memory is a persistently scarce resource throughout the entire optimization lifecycle. Beyond enabling training to proceed at all, memory headroom unlocks other optimizations: larger batch sizes provide more computation to hide communication latency (Section 4.2.3), CUDA Graphs require additional static buffers (Section 4.3.6), and EP communication overlap must hold activations from multiple microbatches simultaneously. Many optimizations described in the following sections consume memory, and the techniques above are what make that consumption feasible.

4.2. Breaking the Communication Wall

Communication overhead directly reduces GPU utilization: every microsecond spent in collective operations represents lost compute capacity. Before optimization, EP all-to-all communication typically consumes 20–60% of training time, depending on model configuration, EP size, and hardware topology. When EP stays within the NVLink domain (e.g., DeepSeek-V3 with EP64 on GB200 NVL72), the overhead is around 20%; when EP crosses node boundaries (e.g., DeepSeek-V3 with EP64 on H100 across nodes), it rises to 40–60%. The techniques in this section target all points on this spectrum.

Expert Parallelism (EP) distributes experts across devices to scale MoE models beyond single-device capacity, but this distribution introduces a unique communication pattern. Unlike AllReduce, whose volume is independent of parallelism degree, all-to-all volume is proportional to token count and hidden dimension, and larger EP pushes this communication cross-node where bandwidth is limited. For fine-grained MoE architectures like DeepSeek-V3 and Kimi-K2, three factors compound this challenge:

- **High frequency**: More experts mean more dispatch/combine operations per layer (DeepSeek-V3 has 58 MoE layers, each requiring two all-to-all operations).

- **Cross-node bottleneck**: With large EP sizes spanning multiple nodes, inter-node all-to-all latency dominates due to lower bandwidth.
- **Low arithmetic intensity**: Small experts complete computation quickly, leaving less time to overlap with communication.

4.2.1. Communication Anatomy: The Expert Parallel Pattern

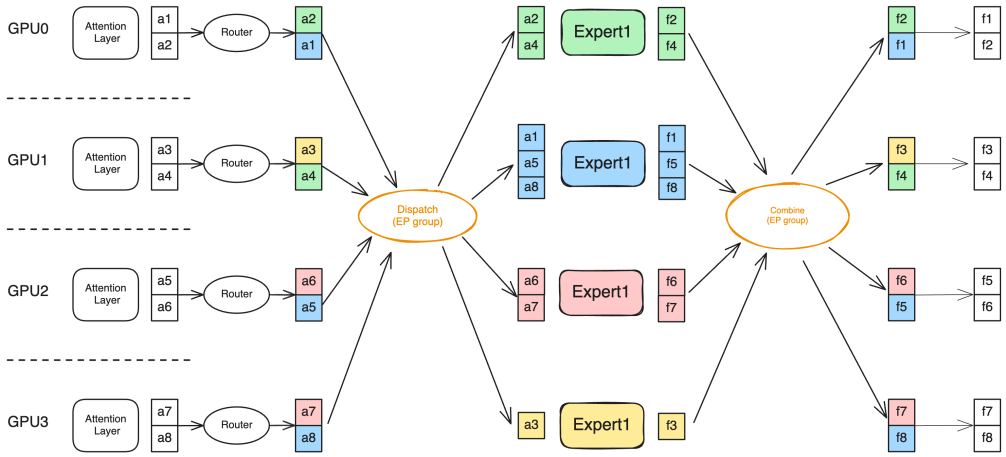


Figure 13: Expert parallelism across 4 GPUs with 4 experts.

Figure 13 illustrates the expert parallelism communication pattern. In standard EP implementations, each MoE layer requires two collective communication operations: *dispatch* sends tokens to their assigned experts, and *combine* returns processed tokens to their original ranks. For a model with L MoE layers, B tokens per batch, hidden dimension h, and EP degree EP, each forward pass involves $2L$ dispatch/combine operations, each transferring $O(Bh)$ data across EP ranks.

At DeepSeek-V3 scale, this translates to 58 MoE layers $\times$ 2 operations/layer = 116 dispatch/combine operations per forward pass. The backward pass doubles this count. At 50 GB/s inter-node bandwidth (e.g., InfiniBand NDR), a single dispatch with 200 MB payload takes several milliseconds, accumulating to hundreds or thousands of milliseconds per iteration.

Communication volume is determined by the EP configuration and cannot be reduced without changing the parallelism strategy. Optimization must therefore target how communication is executed and scheduled. Two strategies work together to address communication overhead:

1. **Maximize bandwidth utilization.** Standard NCCL all-to-all implementations do not fully exploit available bandwidth, particularly for fine-grained MoE workloads. Optimized dispatchers (DeepEP, HybridEP) use specialized kernels that fuse operations and exploit hardware primitives to approach peak bandwidth.
2. **Hide latency behind computation.** Even with optimal bandwidth, all-to-all operations take time. By overlapping communication with computation from adjacent microbatches, this latency can be hidden rather than exposed on the critical path.

The following subsections present techniques organized by these strategies: bandwidth optimization (DeepEP, HybridEP), and latency hiding (EP communication overlapping, pipeline integration).

4.2.2. DeepEP and HybridEP: Maximizing EP Bandwidth

In conventional EP implementations, token exchange relies on all-to-all communication. Even with optimized NCCL collectives, this path has inherent limitations: a permutation stage is needed before dispatch, which

replicates each token top-k times and creates redundant traffic. In some settings, this preprocessing also surfaces as host overhead.

To address these issues, Megatron-Core provides two token-based dispatch backends, HybridEP and DeepEP [23]. Token-based dispatch eliminates the permutation step and avoids sending redundant tokens, reducing overall communication volume and improving effective bandwidth.

HybridEP was developed by NVIDIA. It follows the same token-based principle as DeepEP, exploits hardware primitives such as TMA and IBGDA, and targets comparable or higher bandwidth with lower SM usage, including Multi-Node NVLink (MNNVL) deployments.

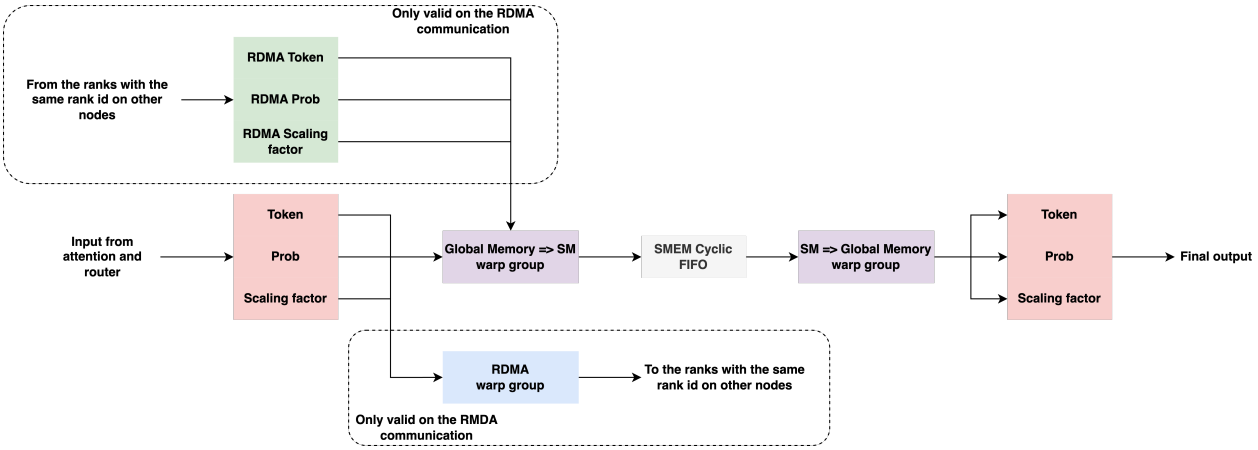


Figure 14: The dispatch kernel design of HybridEP.

For dispatch, as illustrated in figure 14, HybridEP reads data from global memory into shared memory based on routing information, then writes tokens to destinations through a FIFO queue. In the inter-node case, instead of sending duplicated payloads directly through the network interface, HybridEP first uses an RDMA warp group to exchange data between GPUs with the same local index across nodes, then forwards within each node. This reduces cross-node traffic and allows inter-node and intra-node transfers to overlap.

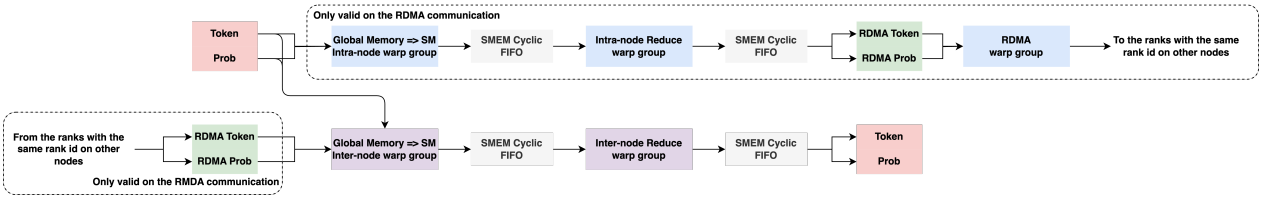


Figure 15: The combine kernel design of HybridEP.

For combine, standard all-to-all dispatch performs communication only, so a separate unpermutation stage is required afterward. HybridEP instead fuses reduction into the communication kernel. As shown in figure 15, HybridEP reads data through a FIFO queue, performs reduction, and writes results directly to target locations. In inter-node settings, HybridEP first performs reduction across nodes for data that must be communicated cross-node, then completes a second reduction within each node.

We evaluate all-to-all dispatch and HybridEP on GB200 and H100 with hidden size 7168, sequence length 4096, and 256 experts under multiple EP sizes. Results are summarized in Table 7. Across all tested settings, HybridEP consistently outperforms all-to-all, with larger gains in inter-node scenarios. The table reports communication latency only; in end-to-end training, the gap is typically larger once permutation and host overhead are included.

Table 7: EP Scaling Performance for HybridEP and all-to-all (in µs).

		GB200 (µs)		H100 (µs)	
	EP size	HybridEP	all-to-all	HybridEP	all-to-all
dispatch	8	391	735	661	1265
	16	578	743	1485	5774
	32	612	769	3064	8059
	64	675	930	4626	9164
combine	8	353	741	624	1277
	16	527	765	1688	5628
	32	646	758	3088	7815
	64	744	827	4398	8727

4.2.3. EP Communication Overlapping: Hiding EP Communication Latency

Optimized dispatchers improve all-to-all *bandwidth utilization*, but the fundamental issue remains: all-to-all still lies on the end-to-end critical path. For DeepSeek-V3 with EP64, EP all-to-all communication can still account for 30–40% of iteration time, directly limiting throughput.

The key observation is that all-to-all latency can be *hidden* behind computation when enough independent work exists in the overlap window [59, 60]. For fine-grained MoE models such as DeepSeek-V3 and Kimi-K2, this is challenging because cross-node EP communication can consume about half of layer time, while adjacent operators (for example, GEMMs) are often too short to hide it.

To address this imbalance, Megatron-Core uses a dedicated 1F1B all-to-all overlap scheme [61] that merges forward and backward passes from neighboring micro-batches and interleaves compute and all-to-all kernels across CUDA streams [39, 38]. Concretely, backward all-to-all for one micro-batch overlaps with forward attention/MLP for another. Conceptually, this is a DualPipe-like [62] bidirectional schedule built on top of standard 1F1B while preserving Megatron-Core compatibility.

To enable all-to-all overlap in 1F1B, we merge adjacent micro-batch forward and backward passes and evaluate two patterns:

1. **Merged FWD-FWD / BWD-BWD**: This strategy merges passes of the same type from two micro-batches (figure 16). In *FWD-FWD*, forward passes for micro-batches 0 and 1 run in parallel and overlap all-to-all with computation. *BWD-BWD* applies the same principle to backward passes. This approach has clear trade-offs:
   - **Memory Overhead**: 2x peak activation memory.
   - **all-to-all Overlap**: Less opportunity to hide all-to-all because forward computation is roughly half of backward computation.

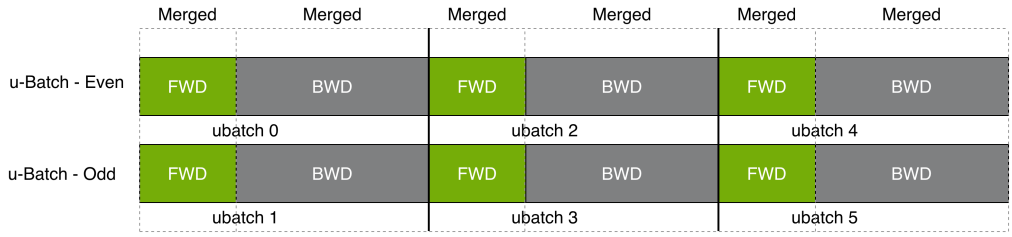


Figure 16: Merged FWD-FWD Timeline with all-to-all Overlapping.

2. **Merged FWD-BWD (DualPipe Equivalent)**: This is the preferred strategy (figure 17). It merges the forward pass of one micro-batch with the backward pass of another (for example, FWD of micro-batch 1

with BWD of micro-batch 0). Compared with *FWD-FWD*, it offers:

- **Memory Overhead**: No additional memory overhead (activations from the forward pass are reused for the backward pass).
- **Compatibility**: Matches DualPipe's design but avoids complex scheduling.
- **Limitation**: The first FWD and last BWD remain on the end-to-end critical path, so their all-to-all cannot be hidden.

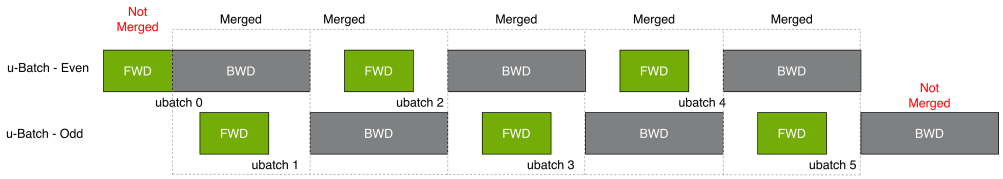


Figure 17: Merged FWD-BWD Timeline with all-to-all Overlapping.

To maximize all-to-all hiding in merged FWD-BWD, we use two key optimizations:

1. **Stream Separation**: We split workloads into two CUDA streams:
   - *Compute Stream*: Runs forward/backward computations (e.g., attention, expert MLP).
   - *Comm Stream*: Runs all-to-all communication (e.g., token dispatch/combine for EP).

   By alternating tasks across streams, all-to-all runs in parallel with computation—minimizing idle cycles.
2. **W/D Split (Weight-Gradient / Data-Gradient Split)** [6]: A key dependency blocks overlap: backward dispatch (B/dispatch) requires outputs from backward MLP (B/mlp). To break this dependency, we split backward MLP into:
   - *W/mlp*: Weight gradient calculation (independent of B/dispatch).
   - *D/mlp*: Data gradient calculation (feeds into B/dispatch).

   This split reduces compute-stream idle time: W/mlp can overlap with F/mlp to hide B/dispatch when F/mlp alone is too short.

Figure 18 summarizes these optimizations. The baseline (top) executes sequentially, where all-to-all blocks computation. The 1F1B FWD-BWD scheme (bottom) interleaves adjacent micro-batches so communication can be hidden behind compute. The W/D split further increases overlap opportunities. Together, these methods reduce EP communication overhead from 30–40% (after DeepEP) to under 5% of iteration time in DeepSeek-V3 training on H100.

To scale overlap to large models, merged FWD-BWD can be combined with *Interleaved Pipeline Parallelism* (Interleaved PP), which partitions the model into virtual pipeline stages (VPP) and expands overlap opportunities across pipeline ranks. Figure 19 shows an interleaved 1F1B schedule with three phases: warmup, 1F1B, and flush. Adjacent FWD-BWD pairs in the 1F1B phase can apply the EP overlap pattern above. However, adjacent pairs may still have data dependencies when they belong to the same micro-batch. To avoid this, we run *one extra micro-batch* at the end of warmup before entering 1F1B, ensuring adjacent FWD/BWD pairs are dependency-free and enabling full all-to-all overlap across virtual stages.

A latency comparison between merged FWD-BWD (1F1B) and DualPipeV (a refined DualPipe variant) shows:

- *1F1B is faster* with large VPP sizes (more virtual stages enable more overlapping opportunities).
- The latency gap shrinks at large PP sizes and micro-batch counts (both strategies reach near-optimal overlap).
- For hybrid models (e.g., DeepSeek-V3 with mixed dense/MoE layers), workload balance across PP stages is critical. Megatron-Core's *Flexible Asymmetric VPP* supports custom per-stage layer placement to maximize overlap.

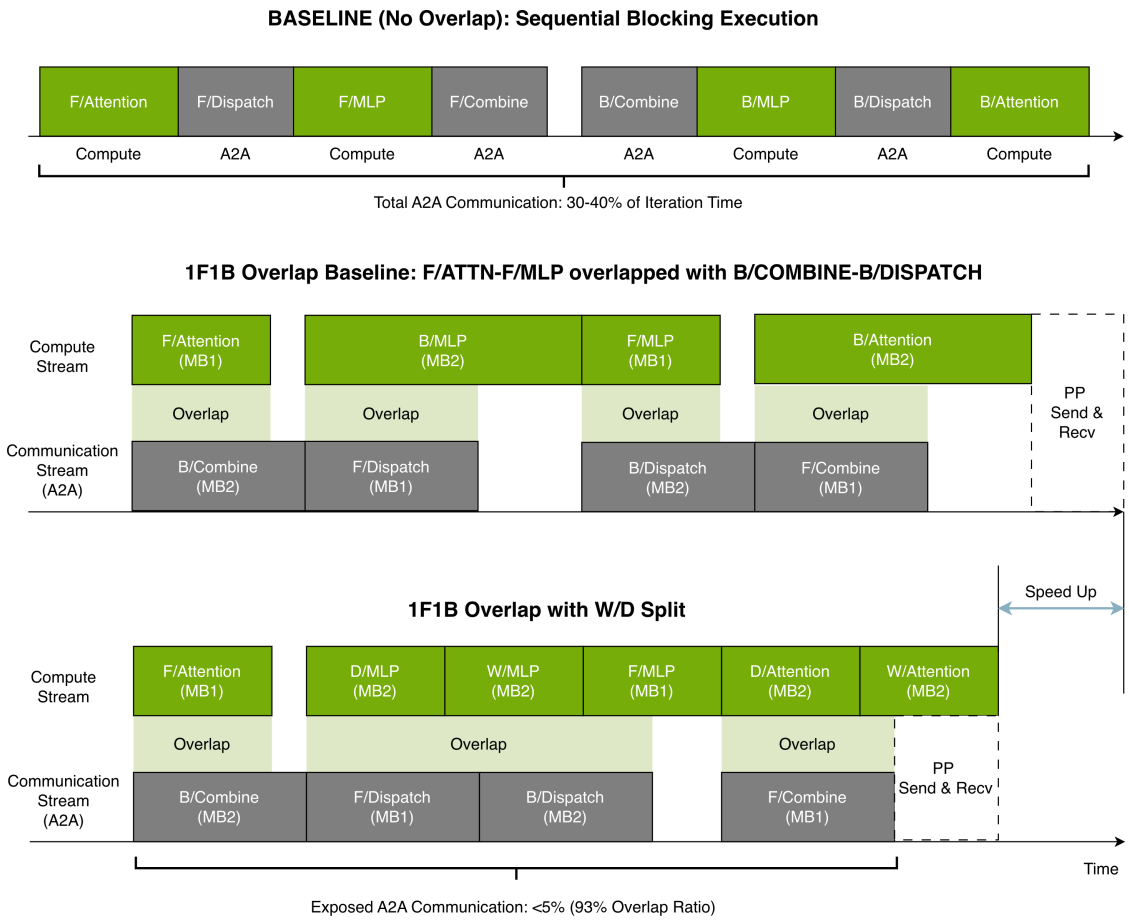


Figure 18: EP all-to-all communication overlap strategies: baseline vs. 1F1B with W/D split.

Several factors limit the achievable speedup from all-to-all overlap:

- **Proportion of overlapped batches**: More micro-batches increase the proportion of overlapped batches.
- **all-to-all Proportion**: EP overlap gains are larger when all-to-all dominates, especially for cross-node EP and fine-grained MoE models with low compute-to-communication ratios.
- **SM Carve-out Overhead**: Reserving SMs for all-to-all can reduce GEMM efficiency. In our DeepSeek-V3 benchmark, DeepEP uses 20 SMs per GPU and introduces roughly 20% GEMM-efficiency overhead.

On fine-grained MoE models such as DeepSeek-V3, combining all-to-all overlap with optimized dispatchers (Section 4.2.2) achieves a 93% overlap ratio for expert communication latency, reducing expert communication share from 30–40% of iteration time to under 5%.

4.2.4. Summary

The communication wall optimizations work together: DeepEP/HybridEP maximize bandwidth utilization; FWD-BWD overlap hides latency behind computation; Interleaved PP extends overlap opportunities across pipeline stages; Flexible VPP enables optimal load balance. Combined, they reduce all-to-all's contribution to training time under 10%.

With communication latency hidden behind computation, high GPU utilization might be expected. However, profiling reveals the Compute Efficiency Wall, which has two distinct aspects: *kernel efficiency* (fine-grained experts produce small GEMMs that underutilize GPU resources) and *host overhead* (numerous small operations create gaps between kernel executions where the GPU awaits work from the host).

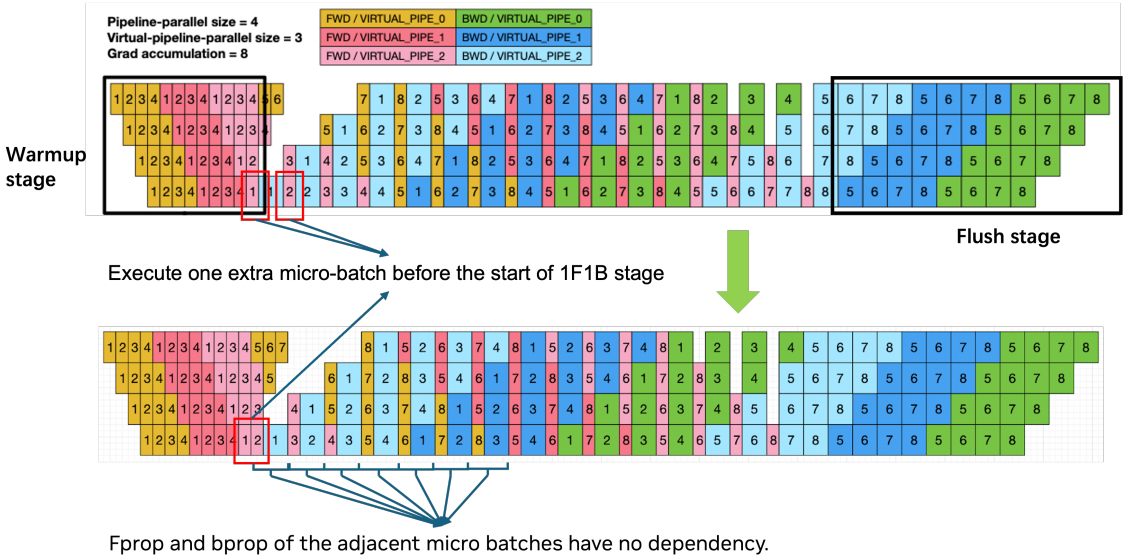


Figure 19: Interleaved PP Timeline with all-to-all Overlapping.

4.3. Breaking the Compute Efficiency Wall

Compute efficiency is the final constraint: even with sufficient memory and hidden communication, GPU resources can remain underutilized if kernels are inefficient or the host cannot dispatch work fast enough. As model and expert counts grow, compute inefficiencies compound, making optimization essential for achieving peak hardware utilization.

4.3.1. Compute Anatomy: Sources of Inefficiency

Compute inefficiency in MoE training stems from two distinct sources:

Kernel Efficiency. Fine-grained MoE architectures produce workloads that underutilize GPU resources. DeepSeek-V3's 256 small experts yield GEMMs with M dimensions of ∼128 tokens per expert, far from the thousands needed for peak Tensor Core utilization. Beyond expert computation, MoE introduces complex routing and dispatching operations composed of many small kernels that individually cannot saturate GPU compute capacity.

Host Overhead. Numerous small operations create kernel launch overhead, and the CPU cannot dispatch work fast enough to keep the GPU saturated. Three factors make this worse:

- **Fine-grained experts**: Many individual GEMMs rather than few large ones, each requiring a separate kernel launch.
- **Reduced-precision training**: Additional quantization kernels add to the launch overhead.
- **Dropless routing**: Dynamic token counts require host-device synchronization to determine tensor shapes, placing the CPU on the critical path.

The result is *host-boundedness*: GPU kernels separated by microseconds of host-side latency, visible as gaps in profiling traces where compute resources sit idle.

Kernel shapes are determined by model architecture and cannot be changed without modifying the model. Optimization must therefore target how computation is organized and scheduled. Five strategies work together to address compute inefficiency:

1. **Fuse related operations.** Kernel fusion consolidates routing, permutation, and auxiliary loss computa-

tions into fewer, larger kernels.
2. **Accelerate with low precisions.** Reduced-precision training in FP8/FP4 uses faster Tensor Core operations, increasing throughput for the same kernel shapes.
3. **Eliminate per-iteration CPU logic.** CUDA Graphs capture kernel sequences and replay them with minimal host involvement.
4. **Remove host-device synchronization.** Sync-free execution enables GPU kernels to proceed without waiting for shape information from the host.
5. **Balance expert load.** ECHO dynamically clones hot experts to underutilized ranks, reducing load imbalance that causes some ranks to wait while others compute.

The following subsections present techniques organized by these strategies: batching and fusion for kernel efficiency (Section 4.3.2), low-precision acceleration (Section 4.3.5), CUDA Graphs for host overhead elimination (Section 4.3.6), sync-free execution for dropless MoE (Section 4.3.7), and load balancing via ECHO (Section 4.3.7).

4.3.2. Grouped GEMM and Kernel Fusion: Improving Kernel Efficiency

The most direct approach to improving kernel efficiency is combining small operations into larger ones. Grouped GEMM batches expert computations for better hardware utilization [24], while kernel fusion consolidates routing and permutation operations into fewer kernels [63]. Megatron-Core implements these optimizations at three levels: expert computation (Grouped GEMM), token routing (Permutation Fusion), and router logic (Router and Aux-Loss Fusion).

Grouped GEMM The computation of experts is essentially a series of independent GEMMs. Grouped GEMM improves performance over separate sequential GEMMs by overlapping the wave tail effect of kernels.

Megatron-Core provides two Grouped GEMM implementations:

i. Multi-stream launch of cuBLASLt GEMMs. By launching individual cuBLASLt GEMMs into multiple CUDA streams, they can overlap with each other. This method supports various precision and scaling modes, including BF16, per-tensor FP8, blockwise FP8, MXFP8, and NVFP4, with performance comparable to highly optimized Grouped GEMM implementations.
ii. CUTLASS Grouped GEMM. By fusing individual GEMMs into a single kernel, CUTLASS Grouped GEMM achieves better performance when the number of GEMMs is large. However, different precisions, scaling modes, and hardware platforms require individual development effort, and additional tuning is needed for different problem sizes. The current implementation in TE only supports BF16 on the Hopper platform.

Two additional implementations are under development:

iii. cuBLASLt Grouped GEMM via `CUBLASLT_BATCH_MODE_GROUPED`. The cuBLASLt Grouped GEMM assumes the shape information on device and conducts the computation in one single kernel, which unblocks applying CUDA Graphs to the expert part. It covers all precisions and scaling modes with built-in heuristics, and is intended to supersede (i) and (ii) where supported.
iv. cuteDSL Grouped GEMM with fusions. Grouped GEMM fused with activation functions, quantization (for the next layer), and scaling factor swizzling via cuteDSL. These kernels are optimized for MXFP8 and NVFP4 [64] on the Blackwell platform, targeting the `fprop` GEMM of FC1 and `dgrad` GEMM of FC2. By consolidating expert computation, activation, and quantization into a single kernel, this approach significantly reduces kernel count and is compatible with CUDA Graphs.

4.3.3. Permutation Fusion

Currently, Grouped GEMM requires tokens assigned to the same expert to be stored contiguously (i.e., permuted tokens). If all-to-all collective communication is enabled, further permutation is necessary to ensure tokens

assigned to each rank are grouped together. Implementing efficient permutation using native PyTorch is challenging, as it tends to launch many small kernels and incurs additional CPU overhead.

The permute fusion pipeline in the training workload (figure 20) consists of three stages:

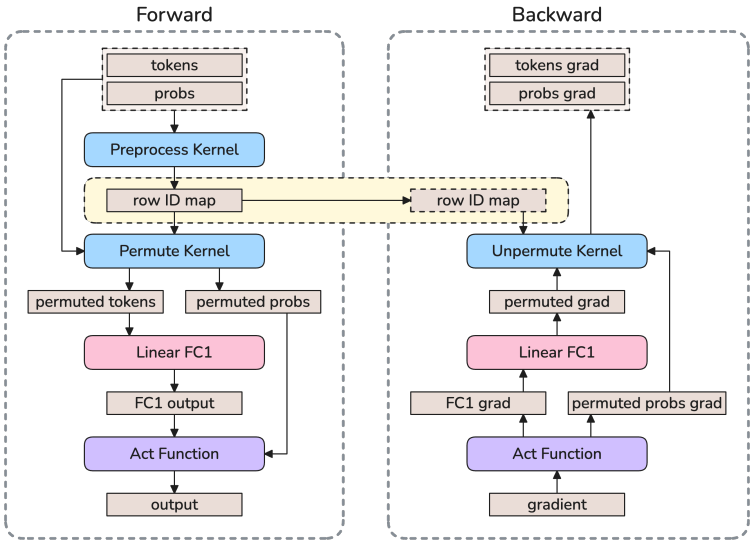


Figure 20: The pipeline for permute fusion in the training process.

- Preprocessing: Permutation is fundamentally a data transfer process that requires tokens to be stored consecutively in the buffer corresponding to each expert. The purpose of the preprocessing step is to generate an offset map (Row ID map in figure 20), which indicates the offset of each token in the input and output buffers. This ensures that permute and unpermute operations can be executed efficiently. The preprocessing kernel is called only once before permute during the forward pass, while other components reuse the results.
- Permute: The functionality of the permute kernel is straightforward: it moves tokens from the input buffer to the output buffer according to the offset map, and then passes them to the subsequent expert MLP. In memory-efficient permute, probabilities also need to be permuted, with the results directly entering the activation part of the expert.
- Unpermute: The unpermute step is the inverse of the permute operation. Since a token will be copied multiple times and sent to different destinations during permutation, these tokens must be summed together during the combine phase. In memory-efficient permute, the combine step simply adds them up; otherwise, the probabilities are used as weights. All accumulation is performed in FP32 precision.

4.3.4. Router and Aux-Loss Fusion

The router and auxiliary loss [4, 65, 14, 15] computations are other areas that generate many small kernels and introduce considerable CPU overhead. As device computing power grows, fusion becomes increasingly important.

The challenge is that the router contains components difficult to fuse, such as GEMM and communication. We decompose the remaining operations into three sections, each fused into a single kernel (figure 21).

- Score computation, including top-k and softmax/sigmoid: This is the most complex function, with various branches for different models. It includes different top-k algorithms (naive top-k, group top-k [17, 6]), score functions (sigmoid, softmax), their combinations, and possible scaling operations.
- Score computation for auxiliary loss: This is a subset of the first section. After the score function and naive top-k computation, the input for auxiliary loss calculation is generated.

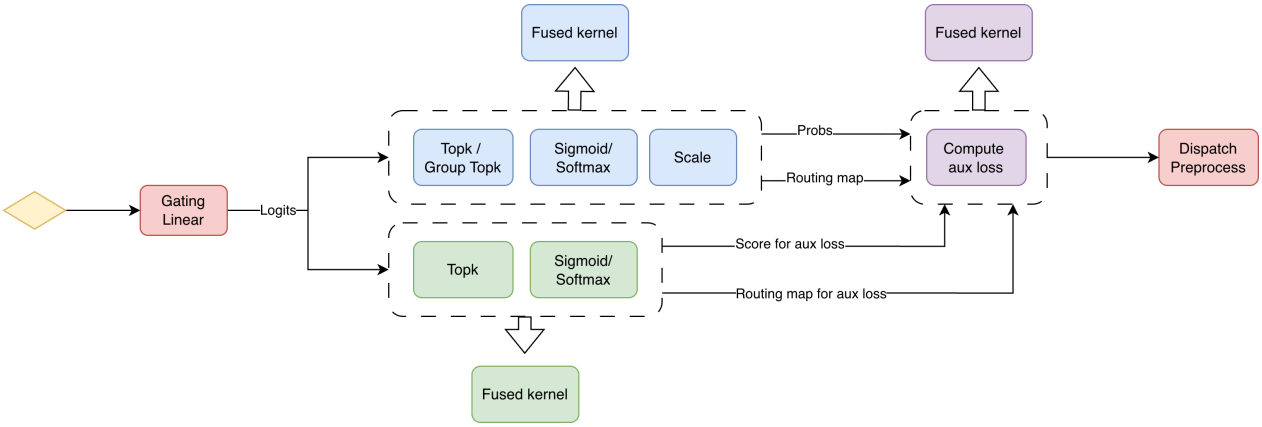


Figure 21: The workflow of the router fusion.

- Computation of MoE auxiliary loss: Building on step 2, the auxiliary loss computation is fused into a single kernel.

After enabling router fusion, the workflow within the router module is shown in figure 21. Future work may incorporate communication into the fusion scope.

4.3.5. Reduced-Precision Training: Low-Precision Acceleration

Fine-grained MoE workloads with small expert GEMMs are often memory-bandwidth bound, limiting Tensor Core utilization. Reduced-precision training addresses this issue by reducing data movement and using faster FP8/FP4 Tensor Core operations on Hopper and Blackwell GPUs.

From a computation efficiency perspective, reduced-precision training provides a key benefit: **FP8/FP4 GEMMs** maximize Tensor Core utilization by performing Linear Layer computation in FP8/FP4. Together with the memory benefits in Sections 4.1.3, FP8 training achieves approximately **10% to 25% end-to-end performance improvement** for large-scale MoE training on different hardware platforms, while FP4 gives it even more speedup.

For complete coverage of reduced-precision training recipes, MoE-specific challenges, and quantization strategies, see Section 5.

4.3.6. CUDA Graphs: Eliminating Host Overhead

Kernel fusion reduces the *number* of kernel launches; CUDA Graphs eliminate the *per-iteration cost* of those launches. This subsection presents how CUDA Graphs address CPU overhead, the different strategies for drop-and-pad versus dropless MoE, and the memory optimizations required to make CUDA Graphs practical.

In MoE training, CPU overhead often becomes the dominant performance bottleneck. This overhead stems from three sources:

1. **Python execution**: Interpreter overhead, C foreign function interface (CFFI), and garbage collection introduce latency.
2. **Framework overhead**: Even simple operations such as `torch.empty` add microseconds of host-side cost; library layers (TransformerEngine, cuBLAS, cuDNN) add more.
3. **Kernel launch overhead**: Each kernel launch incurs several microseconds of CPU-side cost.

Three trends amplify this in modern training:

- **Faster GPUs**: As kernel execution speeds increase, less time remains to overlap CPU work, making CPU overhead increasingly visible.
- **MoE complexity**: MoE models add substantial complexity to FFN layers—routers, dispatch, Grouped GEMMs, and combine operations—yielding many small kernels beyond the GEMMs themselves and thus substantial CPU overhead.
- **Reduced-precision training**: Extra quantization kernels, especially in fine-grained MoE where operation count scales with the number of experts.

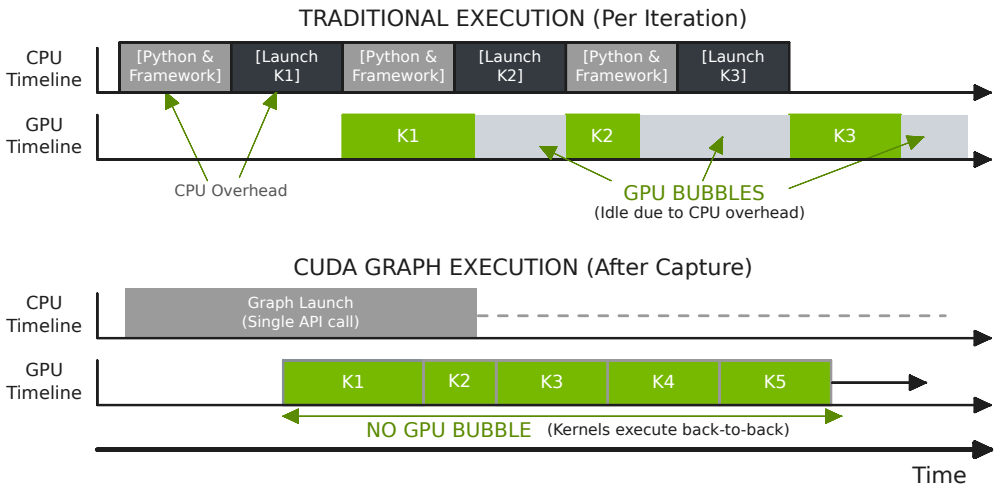


Figure 22: Traditional execution (top) versus CUDA Graph execution (bottom).

Figure 22 shows the large CPU overhead and the resulting GPU bubbles in a traditional execution pipeline, revealing large gaps between GPU kernels. This clearly indicates that the CPU cannot launch kernels fast enough to keep the GPU fully utilized, a phenomenon known as CPU overhead or host-boundedness. CUDA Graphs address this by capturing a sequence of GPU kernels into a replayable graph during an initial iteration [66, 67]. Subsequent iterations issue a single *Graph Launch* call, largely bypassing per-op Python/framework overhead and per-kernel launch overhead, thereby reducing CPU-side latency.

However, CUDA Graphs require all captured operations to have **static, predetermined shapes**. Dynamic shapes or control flow cause graph capture to fail. This creates a fundamental tension with MoE, where token counts per expert vary dynamically. The next paragraph describes Megatron-Core's two CUDA Graph modes—full CUDA Graphs and layer-wise CUDA Graphs—and how they address this dynamic-shape constraint in MoE.

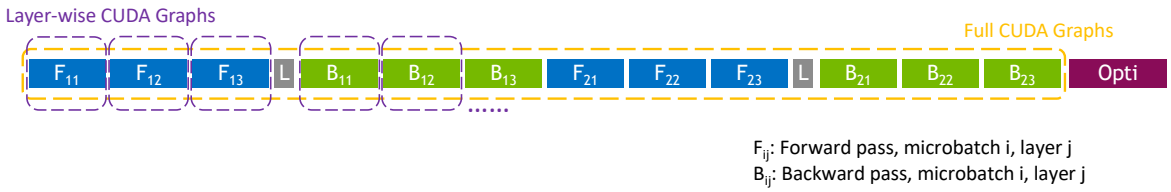


Figure 23: Full versus layer-wise CUDA Graphs in one training iteration (three layers, two microbatches).

Full CUDA Graphs vs Layer-wise CUDA Graphs As shown in figure 23, two types of CUDA Graphs are supported in Megatron-Core: full CUDA Graphs and layer-wise CUDA Graphs. *Full* CUDA Graphs, also noted as full-iteration CUDA Graphs, capture the entire forward-backward pass into a single CUDA Graph. This is only supported for dense or MoE models using **token dropping with padding**, which means each expert has a fixed receiving capacity; tokens exceeding capacity are dropped, and inputs are padded to ensure constant tensor

shapes. With all shapes static, the whole iteration can be graphed, maximizing CPU overhead reduction. As for **dropless** MoE models such as DeepSeek-V3, where no routed tokens can be dropped, full CUDA Graphs are not feasible because the number of tokens assigned to each expert varies dynamically based on routing decisions. Expert GEMM shapes change from iteration to iteration, violating the static-shape requirement. Additionally, *device-host synchronization* is required to retrieve per-expert token counts for dispatching. Section 4.3.7 describes our effort to overcome the dynamic dispatching and synchronization issues so that the full CUDA Graphs can also be applied to dropless MoE. Alternatively, here we introduce a simpler method based on layer-wise CUDA Graphs, called *partial* CUDA Graphs.

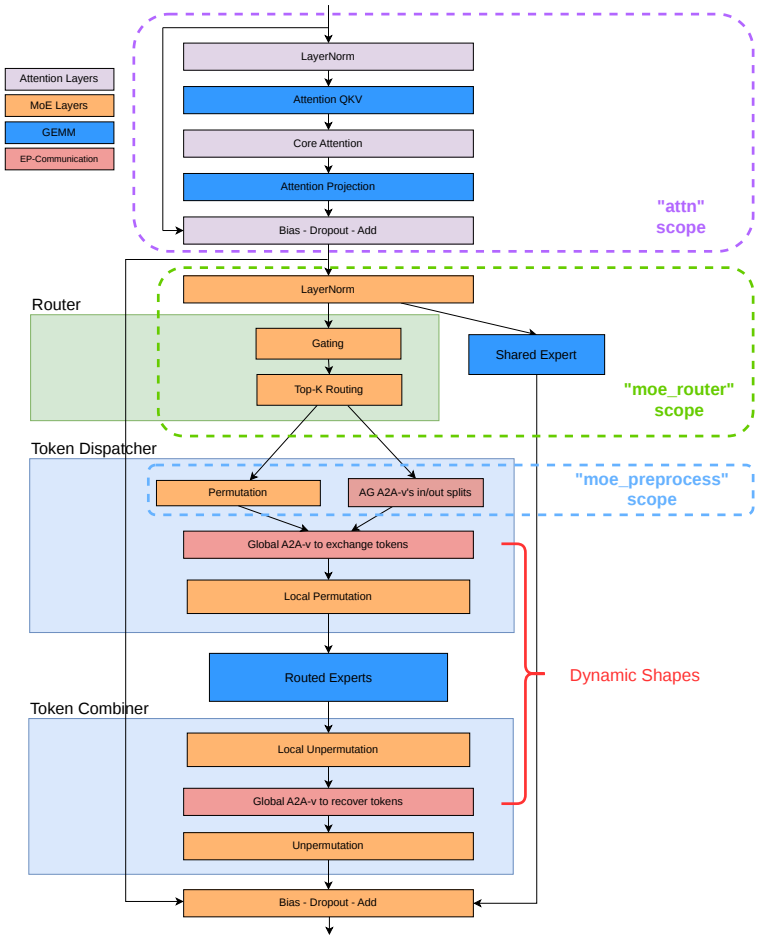


Figure 24: Partial CUDA Graphs capture static components (attention, shared experts, router, preprocessing) while leaving dynamic expert computation outside the graph.

Layer-wise CUDA Graphs capture each transformer layer separately. As with full CUDA Graphs, capturing an entire transformer layer still runs into dynamic shapes in the MoE part. Partial CUDA Graphs avoid this by capturing only the **static components** of each layer while leaving dynamic parts outside the graph, yielding large performance gains for dropless MoE even without capturing the full model. In a dropless MoE layer:

- **Static components** (can be graphed):
  - Attention layers (process a fixed number of tokens)
  - Router computation (fixed input/output shapes)
  - Expert Parallelism (EP) preprocessing (static permutation metadata)
  - Shared experts (if present, process all tokens with fixed shapes)
  - MLP layers in dense transformer blocks

- **Dynamic components** (cannot be graphed):
  - Token dispatch (from fixed shape to dynamic shape)
  - Expert GEMM operations (dynamic M dimension based on token assignment)
  - Token combine (from dynamic shape to fixed shape)

Partial CUDA Graphs capture the static portions of each layer independently, leaving routed expert computation to execute normally. As shown in figure 24, the forward path of each transformer layer is split into several "scopes", and the user can choose which scopes to graph. With the "*attn+moe_router+moe_preprocess*" scopes enabled, one graph captures all static components before the dispatching operation for that layer: attention, router (projection, top-k selection, and auxiliary loss computation), and EP preprocessing (token metadata calculation and permutation). A shared expert, if present, is also captured in this graph. This approach eliminates CPU overhead while preserving correctness by allowing dynamic expert computation to execute with varying shapes. Despite not capturing the entire model, this achieves significant performance gains.

Figure 25: Transformer layer forward pass: without (upper) and with (lower) partial CUDA Graphs. CPU overhead is largely eliminated for static components.

Megatron-Core has been extensively adapted so that CUDA Graphs work correctly with key features like Multi-Token Prediction (MTP), EP communication overlapping, fine-grained offloading and checkpointing, and flexible PP layouts. Nsight Systems profiling with partial CUDA Graphs enabled shows greatly reduced CPU overhead (figure 25). The remaining CPU overhead is primarily confined to the region between token dispatch and combine operations, which cannot be graphed due to their dynamic nature. See Section 4.3.7 for more details on how to address this remaining bottleneck by making the whole layer graphable.

Memory Optimizations for CUDA Graphs CUDA Graphs add memory overhead from three sources:

1. **Graph structure**: Memory for storing the graph topology (typically negligible).
2. **Separate memory pools**: Graphed and non-graphed operations need separate PyTorch memory pools and cannot share memory. With partial CUDA Graphs, a single transformer layer needs two pools: one for graphed work and one for non-graphed work.
3. **Static buffers**: Each graph needs dedicated input/output static buffers that, once allocated, are taken out of PyTorch's memory allocator.

If not carefully optimized, partial CUDA Graphs would incur much higher memory overhead from the above three sources. Megatron-Core and Transformer Engine implement several optimizations to minimize this overhead:

Reducing the number of graphs. With pipeline parallelism (PP), **each microbatch** requires a dedicated CUDA Graph. The reason is that if microbatches share a graph, microbatch $i+1$'s forward pass will overwrite the

saved context before microbatch i's backward pass executes, causing memory corruption (figure 26). This results in $L \times M \times 2$ graphs in total (where L = layers per GPU, M = microbatches, $\times 2$ for forward/backward).

On the contrary, if PP is not used, microbatches can share the same graph, reducing the count to $L \times 2$. To enable graph sharing, an `is_first_microbatch` GPU flag is introduced to control microbatch-specific behaviors within the shared graph, e.g., quantization that runs only on the first microbatch. Setting this flag to 0 or 1 before replay causes the captured quantization kernel to be skipped or executed as needed.

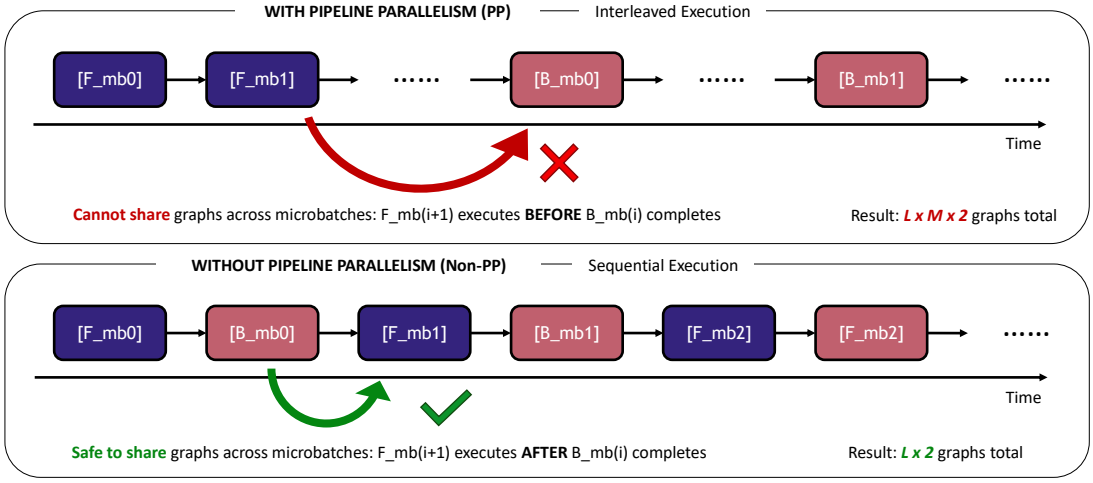


Figure 26: Why Pipeline Parallelism prevents CUDA Graphs from being shared across microbatches. **With PP** (top): Execution is interleaved—multiple forward passes run before any backward pass. If microbatches share a graph, F_mb1 overwrites saved context of F_mb0 before B_mb0 uses it, causing memory corruption. Each microbatch needs its own graph ($L \times M \times 2$ graphs total). **Without PP** (bottom): Execution is sequential—each microbatch completes (forward+backward) before the next starts. Context is consumed before being overwritten, so microbatches can safely share graphs (only $L \times 2$ graphs total).

Pool sharing. Although graphed and non-graphed operations require separate memory pools, all graphs can share one pool when captured in execution order. Transformer Engine's `make_graphed_callables()` API accepts an `_order` argument that specifies the inter-microbatch scheduling, allowing sequential capture with correct pool sharing.

Buffer reuse. Static input/output buffers can be reused across graphs according to the PP execution order. Once a microbatch finishes its backward pass of a layer, its forward buffers and backward input buffer become available for the next forward and backward graphs to reuse. Only the backward output buffer must be kept until the data is sent to the next PP stage.

With these memory optimizations, we achieve 10% end-to-end speedup with about 7 GB extra memory on DeepSeek-V3 GB200 training.

The Remaining Problem: Dynamic Expert Computation Full CUDA Graphs do not support dropless MoE. Partial CUDA Graphs eliminate CPU overhead for static components, but **the dynamic expert computation region remains outside the graph**. Is there any way we could make CUDA Graphs *cover* the dynamic part? There are at least two challenges we must solve to achieve this:

1. **Dynamic shapes require host–device synchronization**: The host must query per-expert token counts from the GPU to launch token dispatching and Grouped GEMMs.
2. **Memory allocation requires known buffer sizes**: Worst-case buffer allocation wastes memory; actual-size allocation requires synchronization.

The next subsection presents how Megatron-Core achieves full CUDA Graphs coverage for dropless MoE through three complementary techniques: sync-free kernels, ECHO, and Paged Stashing.

4.3.7. Full CUDA Graphs Coverage for Dropless MoE via Sync-Free Kernels, ECHO, and Paged Stashing

While kernel fusion and CUDA Graphs help reduce host overhead, dropless MoE introduces a new challenge: dynamic tensor shapes require host-device synchronization, preventing CUDA Graphs from covering expert computation.

In dropless MoE, token counts received by each EP rank vary dynamically based on routing decisions. The router generates this shape information on the GPU, but the host needs it to launch subsequent operations and allocate memory. Obtaining this information requires a device-to-host copy and synchronization, placing the CPU on the critical path and preventing CUDA Graphs capturing of the dynamic portions. To enable sync-free MoE for dropless training with dynamic routing, there are two fundamental challenges:

Challenge 1: Kernel launch without knowing the actual problem size. Conventionally, many GPU operators assume that (dynamic) shape information is available on the host. The host-side code determines launch configurations, such as grid size and tile size, as well as the amount of work the kernel must perform based on the shape information obtained at run time. In dropless MoE, this creates a host-device synchronization point: the host must query per-expert token counts from the device before launching Grouped GEMM or communication kernels. To tackle this issue, the GPU kernels handling dynamic shapes need to be redesigned. We developed **device-initiated GPU kernels**, including device-initiated Grouped GEMM and sync-free dispatch with HybridEP.

Challenge 2: Memory allocation without knowing the actual size. Sync-free execution means the size of buffers needs to be pre-determined on the host. To avoid token overflow, buffers often need to be over-sized, e.g., based on the worst-case size to accommodate the case where all tokens are routed to the same expert. This over-provisioning, however, leads to severe memory fragmentation: if the actual token distribution is balanced, the vast majority of the preallocated memory remains unused, yet it cannot be reclaimed by other operations within the graph. The worst-case buffer potentially requires $O(\text{EP\_size})$ times more memory than the actual working set.

This memory fragmentation can be mitigated through two complementary strategies: *reducing load imbalance* so that worst-case provisioning is closer to actual usage, and *dynamic memory management* within the CUDA Graph to reclaim unused buffer space. **ECHO** employs the first strategy by dynamically cloning popular experts on underutilized ranks, reducing the variance in per-rank token counts and thus the gap between worst-case and average memory requirements. **Paged Stashing** adopts the second strategy by enabling fine-grained memory management within CUDA Graph, allowing unused portions of preallocated buffers to be repurposed for other operations. ECHO and Paged Stashing are detailed below.

Device-Initiated GPU Kernels To eliminate the mandatory host-device synchronization required by conventional host-initiated GPU operators, kernels must be *device-initiated*. This requires three things:

i. The GPU kernel needs to read shape information from GPU memory and use it to guide the computation.
ii. The GPU kernel needs to decouple the actual amount of work it performs, which is only known at runtime, from its static launch configuration.
iii. The GPU kernel needs to skip unnecessary computation, such as operations on padded data.

To satisfy these requirements, existing kernels must be redesigned. The Grouped GEMM kernel and HybridEP kernel serve as two concrete examples below.

Device-Initiated Grouped GEMM. As discussed in Section 4.3.2, TE offers two Grouped GEMM implementa-

tions: the multi-stream cuBLASLt GEMMs and the CUTLASS Grouped GEMM. Both are host-initiated. They require a CPU-side list of per-expert token counts to determine each GEMM's shape and launch configuration, creating a host-device synchronization barrier. To eliminate this, TE introduces device-initiated Grouped GEMM, which reads shape information directly from device memory. Two implementations are provided:

i. **cuBLASLt Grouped GEMM.** Since CUDA 13.1, cuBLASLt Grouped GEMM supports passing matrix shapes as device arrays. This implementation includes built-in heuristics for selecting optimal kernel configurations, now covering various precisions and scaling modes across recent CUDA releases.
ii. **cuteDSL Grouped GEMM with Fused Activation and Quantization.** In the cuteDSL-based implementation, the SwiGLU activation can be fused into the epilogue stage, and for FP8 training, quantization can also be fused. While current support is limited to specific precisions and fusion patterns, ongoing development continues to expand coverage across new configurations.

Sync-Free Dispatch with HybridEP. HybridEP, introduced in Section 4.2.2, provides an efficient implementation of MoE communication and also plays an important role in sync-free MoE. After each dispatch and permutation, the number of tokens received by a given rank is dynamic, normally requiring synchronization to obtain buffer sizes. By estimating an upper bound and passing it to HybridEP, the dispatcher can pre-allocate output buffers according to this bound, eliminating all synchronizations at the cost of additional GPU memory.

Even with device-initiated kernels, worst-case buffer allocation wastes memory. Two complementary strategies address this: **ECHO** reduces load imbalance so worst-case provisioning approaches actual usage, while **Paged Stashing** enables fine-grained memory management to reclaim unused buffer space.

ECHO: Elastic Cloning for Hot Experts. [2]

Load imbalance is inherent in MoE: popular experts receive far more tokens than others, creating two problems. First, EP ranks hosting hot experts become compute bottlenecks, causing other ranks to wait at synchronization points. Second, high load variance means worst-case buffer provisioning wastes significant memory compared to actual usage. ECHO addresses both by dynamically cloning hot experts to spare slots on underutilized ranks.

figure 27 illustrates the ECHO workflow. In the forward pass, the ECHO planner identifies hot experts and generates two outputs: a *hot expert map* indicating which experts to clone to which spare slots, and an updated *routing map* redirecting overflow tokens to clones. Expert Dispatch copies hot expert weights to spare slots via HybridEP-based sync-free communication; Token Dispatch routes tokens to both home and cloned experts; expert computation proceeds on all experts. In the backward pass, Expert Gradient Dispatch collects gradients from clones and reduces them to home experts, ensuring consistency. Cloned experts are discarded after computation to save memory.

Cloning experts for training is much more expensive than for inference: each cloned expert requires weight communication during forward and gradient reduction during backward to ensure consistency. Cloning all hot experts to all spare slots would maximize load balance but introduce excessive communication overhead. The goal of the ECHO planner is to minimize the number of expert clones while achieving sufficient load balance. It computes the *spillover* for each expert (tokens exceeding the average load per EP rank) and the *spare capacity* on each rank (available capacity below average). Using a bin-packing algorithm, the planner efficiently matches spillover tokens to spare capacity, determining which specific experts to clone and to which ranks, producing the hot expert map and routing map with minimal cloning overhead.

ECHO provides two key benefits: (1) *reduced memory fragmentation for CUDA Graphs*, since reducing load variance across EP ranks makes worst-case buffer sizing closer to actual usage, enabling smaller static buffers; and (2) *improved compute efficiency*, since balanced workloads reduce the straggler problem, improving overall GPU utilization.

[2] Implementation details: https://github.com/NVIDIA/Megatron-LM/pull/2368

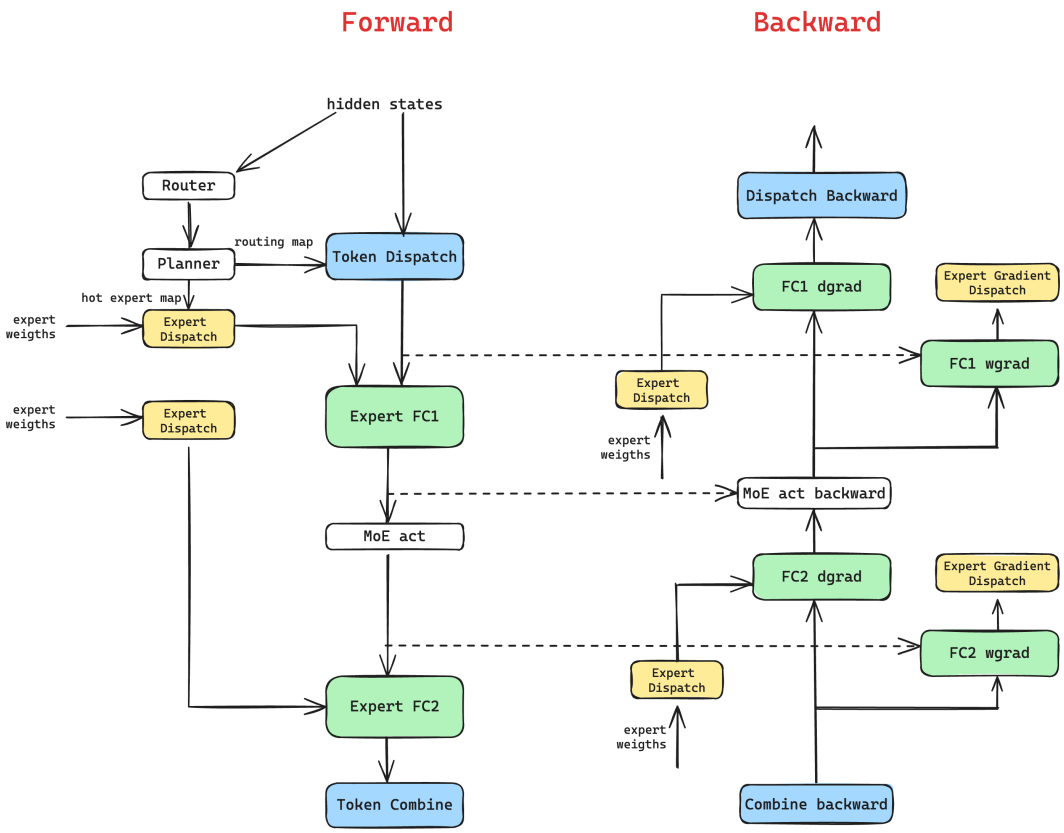


Figure 27: ECHO workflow for forward and backward passes. The planner generates routing and hot expert maps. Expert Dispatch clones hot expert weights to spare slots; Expert Gradient Dispatch reduces gradients back to home experts.

Paged Stashing. [3]

While ECHO reduces load imbalance to narrow the gap between worst-case and typical memory requirements, Paged Stashing addresses the complementary problem: enabling fine-grained memory management within CUDA Graph to reduce internal memory fragmentation.

The fragmentation problem arises because sizes of buffers involved in CUDA Graphs need to be pre-determined. To avoid undesired overflow, the buffers usually need to be over-sized. In a straightforward approach, buffers in all layers are allocated according to the worst-case size (figure 28, middle). For dropless MoE, this means each layer reserves a buffer based on maximum possible tokens each rank may receive, resulting in $O(\text{layers} \times \text{worst\_case})$ memory overhead even when actual token counts are much lower.

Paged Stashing is based on a key observation: there is a vast gap (often more than an order of magnitude) between the actual memory needed to store activations for backward computation and the worst-case memory allocated to avoid overflow at each layer. Paged Stashing addresses this by decoupling these two memory buffers. This is illustrated in figure 28 (right). Instead of allocating worst-case-sized buffers for every layer, the paged-stashing manager maintains: (1) a single *tmp buffer* sized for the worst-case token counts, which is shared across all layers for computation, and (2) a *stashing buffer* organized in the form of pages that stores only the actual tokens used by each layer. When a layer completes its forward computation, its activations are copied from the tmp buffer to the stashing buffer, storing only the actual token count rather than the worst-case allocation. The tmp buffer can then be readily reused for the computation in following layers. This significantly reduces memory consumption from $O(\text{layers} \times \text{worst\_case})$ to $O(\text{worst\_case} + \text{actual\_total})$.

[3] Implementation details: https://github.com/NVIDIA/Megatron-LM/pull/2690

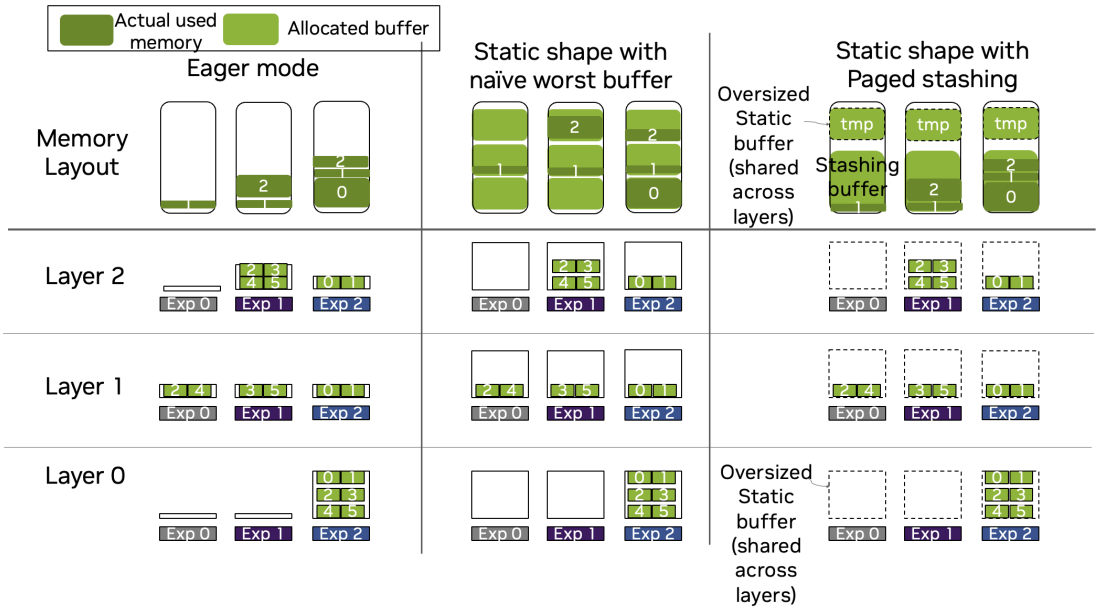


Figure 28: Memory layout comparison across three execution modes. **Left**: Eager mode allocates memory dynamically based on actual usage. **Middle**: Baseline static shape requires worst-case sized buffers for each layer independently, causing severe fragmentation when actual usage is lower. **Right**: Paged Stashing uses a single worst-case tmp buffer shared across layers for computation, while a paged stashing buffer stores only the actual tokens, significantly reducing total memory footprint.

The stashing buffer uses the classic idea of paging to manage memory in a flexible yet efficient way. The `PagedStashBuffer` is organized as pages of fixed size (default 64 tokens per page). The page stashing manager tracks available pages through a free list implemented by a circular buffer. Paging operations are implemented as device-initiated *stash kernels*. The stash kernel copies activations to free pages allocated from the head of the free list and records the target pages. The *reload kernel* retrieves activations from the recorded pages and returns the unused pages to the tail of the free list.

To minimize overhead, Paged Stashing overlaps memory transfers with computation using dedicated CUDA streams (figure 29). The system pre-fetches activations for the next backward layer while current computation executes, hiding reload latency. This overlap requires slight memory overhead due to double buffering.

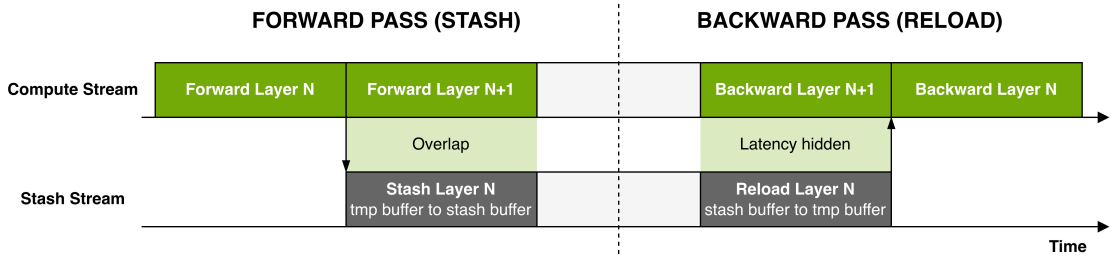


Figure 29: Paged Stashing stream overlap. **Forward pass**: After Layer N computes, its activations are stashed (copied from tmp buffer to paged stashing buffer) on a dedicated Pack stream while Layer N+1 computes on the main Compute stream—the stash is completely overlapped. **Backward pass**: Activations for Layer N are pre-fetched (reloaded from stashing buffer to tmp buffer) on the Unpack stream before Layer N backward starts, hiding reload latency.

Putting It Together: Full CUDA Graphs Coverage Together, these three techniques enable full CUDA Graphs coverage for dropless MoE, eliminating CPU overhead while preserving the flexibility of dynamic routing:

- **Device-initiated kernels** remove host-device synchronization entirely by reading dynamic shapes directly from GPU memory, making all MoE operations capturable by CUDA Graphs.
- **ECHO** balances expert workloads across EP ranks, narrowing the gap between worst-case and actual buffer requirements. This enables smaller static allocations and mitigates the straggler problem that otherwise limits GPU utilization.
- **Paged Stashing** reduces memory from $O(\text{layers} \times \text{worst\_case})$ to $O(\text{worst\_case} + \text{actual\_total})$ by sharing a single worst-case tmp buffer across layers and storing only actual activations in a paged stashing buffer.

Overhead. These benefits come with measured trade-offs. ECHO introduces extra communication for cloning hot experts in the forward pass and reducing their gradients in the backward pass; in practice, only a small fraction of experts are cloned (determined by the spillover above average load), so the additional communication remains modest relative to the standard all-to-all dispatch. Paged Stashing adds memory copy operations (stash in the forward pass, reload in the backward pass) and a persistent worst-case-sized tmp buffer with double-buffering overhead; these copies run on dedicated CUDA streams overlapped with layer computation, so the latency is largely hidden and the primary cost is additional memory bandwidth. Both overheads are relatively small compared to the gains from eliminating per-iteration host-device synchronization and enabling static memory planning with CUDA Graphs.

4.3.8. Summary

The compute efficiency optimizations address both sources of inefficiency. For kernel efficiency: Grouped GEMM batches expert computations, kernel fusion consolidates routing and permutation operations, and reduced-precision training accelerates Tensor Core throughput. For host overhead: CUDA Graphs eliminate per-iteration CPU logic, and Sync-Free MoE removes host-device synchronization barriers. ECHO addresses load imbalance that compounds both issues. Together, these optimizations transform compute inefficiency from a dominant bottleneck into a manageable constraint.

4.4. Summary: Breaking the Three Walls

This section addressed the three fundamental barriers to efficient MoE training, each a hard constraint that, unaddressed, prevents practical training at scale.

- **Memory Wall**: Memory is the first constraint. If parameters, optimizer states, and activations exceed the GPU capacity, training cannot proceed. Memory-Efficient Permutation, FP8/FP4 activations, fine-grained recomputation, offloading, precision-aware optimizers, and FSDP together transform the 199.5 GB per-GPU requirement of DeepSeek-V3 into a feasible training configuration.
- **Communication Wall**: Communication overhead directly reduces GPU utilization. Optimized dispatchers (DeepEP, HybridEP) maximize bandwidth and FWD-BWD overlap hides latency behind computation. Together, these reduce all-to-all overhead from up to 60% of the training time to less than 10% in DeepSeek-V3 training.
- **Compute Efficiency Wall**: Compute inefficiency stems from two sources: small kernels that underutilize GPU resources, and host overhead that leaves GPUs idle. Grouped GEMM and kernel fusion improve kernel efficiency; CUDA Graphs and Sync-Free MoE eliminate host overhead. These transform computation inefficiency from the dominant bottleneck into a manageable constraint.

These optimizations are not independent. They form a coherent system where solutions to one wall enable or enhance solutions to others. Reduced-precision training shows this clearly: it reduces memory (activations) and improves compute efficiency (faster Tensor Core operations), while requiring careful integration to avoid introducing new bottlenecks (quantization kernel overhead).

The following sections address two cross-cutting concerns: Section 5 covers reduced-precision training in FP8/FP4, which simultaneously impacts all three walls, and Section 6 addresses long-context MoE training, which changes the optimization balance when attention dominates computation.

5. Reduced-Precision Training in FP8/FP4 for MoE

The preceding sections addressed each of the three walls with targeted optimizations. Reduced-precision training is unique: it simultaneously impacts memory, communication, and computation efficiency, making it a cross-cutting optimization that deserves dedicated treatment.

Mixed-precision training has been a cornerstone of efficient deep learning, with BF16 as standard practice [68, 69, 70]. Reduced-precision training in FP8/FP4 represents a more aggressive step, reducing the precision from 16 bits to 8 or even 4, with correspondingly larger benefits and risks [71, 72]. For DeepSeek-V3, FP8 training reduces activation memory by approximately 16 GB per GPU, improves expert GEMMs through faster Tensor Core operations, and accelerates the parameter AllGather communication. These gains compound across the three walls, making FP8 essential for efficient large-scale MoE training.

However, low-precision training introduces convergence risks that must be systematically addressed. This section presents Megatron-Core's approach to reduced-precision training: understanding where precision matters, selecting appropriate quantization recipes, and implementing MoE-specific optimizations that capture benefits of reduced-precision training while maintaining training stability.

5.1. Why Reduced-Precision Training Matters for MoE

MoE architectures amplify both the benefits and risks of low-precision training compared to dense models:

Amplified Benefits. With hundreds of experts, activation memory scales proportionally. FP8/FP4 activations provide larger absolute memory savings than in dense models. The communication volume for parameter AllGather is halved[4], and expert GEMMs (which dominate MoE computation) benefit from faster Tensor Core throughput.

Amplified Risks. Router decisions depend on precise scores to assign tokens to experts. Quantization noise could destabilize expert selection, leading to training instability, degraded model quality, or expert collapse [73]. The numerically sensitive components must be protected from aggressive quantization.

The Strategy: Selective Precision. The solution is precision where it matters, efficiency everywhere else. Three principles guide the deployment of reduced-precision training in MoE:

1. **Protect routing decisions.** The router remains in FP32 to ensure stable expert selection.
2. **Preserve precision for key components.** Embeddings, output layers, main gradients, master weights, and optimizer states remain in their original precision to maintain model quality.
3. **Quantize bulk computation.** Expert GEMMs, which constitute the majority of computation, use reduced-precision training with carefully designed quantization schemes (called *recipe*).

> **Note**
>
> The key to successful reduced-precision MoE training is selective precision: keep numerically sensitive components (router, embeddings, output layers, gradients, and optimizer states) in higher precision while aggressively quantizing the bulk of computation (expert GEMMs). This strategy captures most

[4]For the NVFP4 recipe on Blackwell, since we need to gather both the row-wise and column-wise weight tensors, the saved communication volume is also 50%.

benefits of the reduced-precision training while avoiding convergence issues.

5.2. The Impact from Reduced-Precision Training on All Three Walls

Reduced-precision training provides benefits across all three performance walls, making it a unifying optimization. Table 8 summarizes these impacts.

Table 8: Summary of the impact from reduced-precision training on the Three Walls.

Wall	Reduced-Precision Training Benefit	Details
Memory	50% (FP8)/75% (FP4) activation reduction	Section 4.1.3
	Eliminate BF16 weight copies	Section 5.3.5
	BF16 optimizer states	Section 4.1.6
Communication	50% parameter AllGather [1]	Section 5.3.5
Compute	Faster Tensor Core GEMMs	Section 4.3.5
	Quantization kernel overhead	Section 4.3.2

[1] For NVFP4, we need to gather both row-wise and column-wise FP4 weights, so the reduced traffic of parameter AllGather is also 50%.

5.2.1. Breaking the Memory Wall with Reduced-Precision Training

Reduced-precision training reduces memory consumption in three ways:

Activation Memory (detailed in Section 4.1.3). The primary source of activation memory savings comes from input tensors of linear layers saved for backward computation. By storing these tensors in FP8/FP4 instead of BF16, the memory footprint is reduced by 50%/75%. For example, FP8 saves approximately 16 GB of activation memory per GPU for DeepSeek-V3.

Parameter Memory. Conventional reduced-precision training maintains three parameter copies: FP32 master weights, BF16 model weights, and FP8/FP4 computation weights. Our *native FP8/FP4* approach (detailed in Section 5.3.5 below) eliminates the BF16 copy by casting weights directly from FP32 to FP8/FP4 and doing parameter AllGather in reduced precisions.

Optimizer State Memory (detailed in Section 4.1.6). Adam optimizer states (first and second moments) can be stored in BF16 instead of FP32, reducing optimizer memory by 50%. This is orthogonal to reduced-precision training and is also applicable to BF16.

5.2.2. Breaking the Communication Wall with Reduced-Precision Training

Parameter AllGather in FP8/FP4. When using FP8/FP4 primary weights with distributed optimizer, parameter AllGather communication is reduced by 50% (1 byte vs 2 bytes per parameter). Note that

- For NVFP4, we need to gather both row-wise and column-wise FP4 weights, so the reduced traffic of parameter AllGather is also 50%.
- For MXFP8, even with FP8 primary weights, we first copy the FP32 master weights into a temporary BF16 buffer and communicate in BF16 precision. This is because MXFP8 requires different quantization directions for forward and backward passes. Communicating MXFP8 weights would require both row-wise and column-wise quantized versions, effectively communicating two bytes per parameter and eliminating any advantage over communicating BF16. Hence, for MXFP8, we communicate parameters in BF16 precision.

5.2.3. Breaking the Compute Efficiency Wall with Reduced-Precision Training

Faster GEMMs. FP8 (Ada/Hopper and later) and FP4 (Blackwell and later) Tensor Cores provide higher throughput than BF16, accelerating both forward and backward passes.

The Trade-off: Quantization Overhead. Reduced-precision training introduces additional quantization kernels that increase CPU overhead, which is particularly problematic for fine-grained MoE where many small operations already stress the CPU. This overhead is managed through:

- **Kernel fusion** that fuses quantization with other operations, such as normalization and activation functions.
- **Fuse padding/unpadding** with the permutation kernel or routing map to avoid explicit padding/unpadding.
- **Grouped quantization** that fuses the quantization of multiple input tensors into one single kernel.
- **CUDA Graphs** that captures quantization kernels to eliminate launch overhead. See Section 4.3.6 for more details.

5.3. Reduced-Precision Training Recipes: Per-Tensor FP8, Blockwise FP8, MXFP8, and NVFP4

Having established the impact of reduced-precision training on all three walls, we now examine the technical components that define a reduced-precision training configuration.

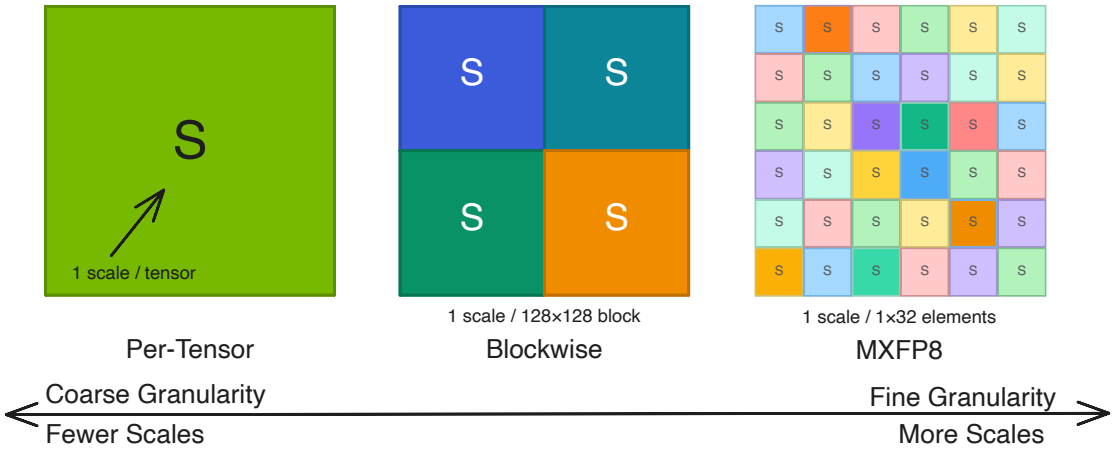


Figure 30: FP8 training recipes: Per-Tensor Scaling, Blockwise FP8, and MXFP8.

A reduced-precision training recipe consists of:

- **Data format.** There are two types of FP8 format: E4M3 and E5M2 [71, 74]. Usually there are two combinations used in training:
  - **E4M3**: Inputs, weights, and gradients are all quantized in the E4M3 format.
  - **Hybrid**: Inputs and weights are quantized in E4M3, while gradients are quantized in E5M2.

  While for FP4, E2M1 is the only supported format.
- **Scaling granularity.** Choices include per-tensor and per-block (also called group or tile). For per-block scaling, the block size also varies between recipes.
- **Which parts are quantized.** Usually all linear layers are quantized, while keeping embedding, output layer (LM head), and optimizers in their original high precision. Part of the communication can be in FP8/FP4 (e.g., TP AllGather, DP AllGather, EP all-to-all). Current reduced-precision training recipes usually keep SDPA in BF16.

- **Additional mechanisms**, such as stochastic rounding, Random Hadamard Transforms (RHT), etc. These mechanisms are recipe-specific and require careful verification.

Megatron-Core provides three FP8 recipes, each validated at scale (figure 30). The evolution from per-tensor to blockwise to MXFP8 reflects both hardware advances and lessons learned from large-scale deployments. **Per-Tensor Scaling** uses a single scale factor per tensor, simple but limited precision, suitable for experimentation. **Blockwise FP8** (128×128 blocks) provides finer granularity with proven stability at scale, recommended for Hopper. **MXFP8** (1×32 granularity) offers native Blackwell Tensor Core support with hardware-accelerated scaling, recommended for Blackwell. Besides, Megatron-Core also provides an NVFP4 recipe [75] on Blackwell.

5.3.1. Per-Tensor FP8 Recipe

The per-tensor FP8 recipe, supported on Hopper and Blackwell, usually adopts the hybrid format (E4M3 for inputs/weights, E5M2 for gradients). It calculates the absolute max value (amax) of all values in the input tensor and uses that to quantize the tensor into FP8.

There are two variants of per-tensor scaling:

- **Delayed scaling**: Use the amax from a history window. This breaks the data dependency between the calculation of amax and scaling, offering the best performance at the risk of losing precision due to the estimated amax.
- **Current (live) scaling**: Calculate the amax in a just-in-time manner, providing better precision.

We do not recommend delayed scaling due to precision issues; current scaling provides better precision and model convergence. Figures 31a and 31b illustrate the computation of a linear layer with per-tensor current scaling on the Hopper and Blackwell platforms, respectively. Since only TN layout FP8 GEMM is supported on Hopper, we must store the transposed FP8 activations for backward calculation. In Transformer Engine, the quantization kernels fuse the cast and transpose into a single kernel to reduce global memory access. On the Blackwell platform, FP8 GEMMs in all layouts are supported, so we do not need the transposed version, further reducing the memory footprint.

Per-tensor scaling is a good starting point for migrating to FP8 training due to its simplicity and relatively good precision and performance.

5.3.2. Blockwise FP8 on Hopper

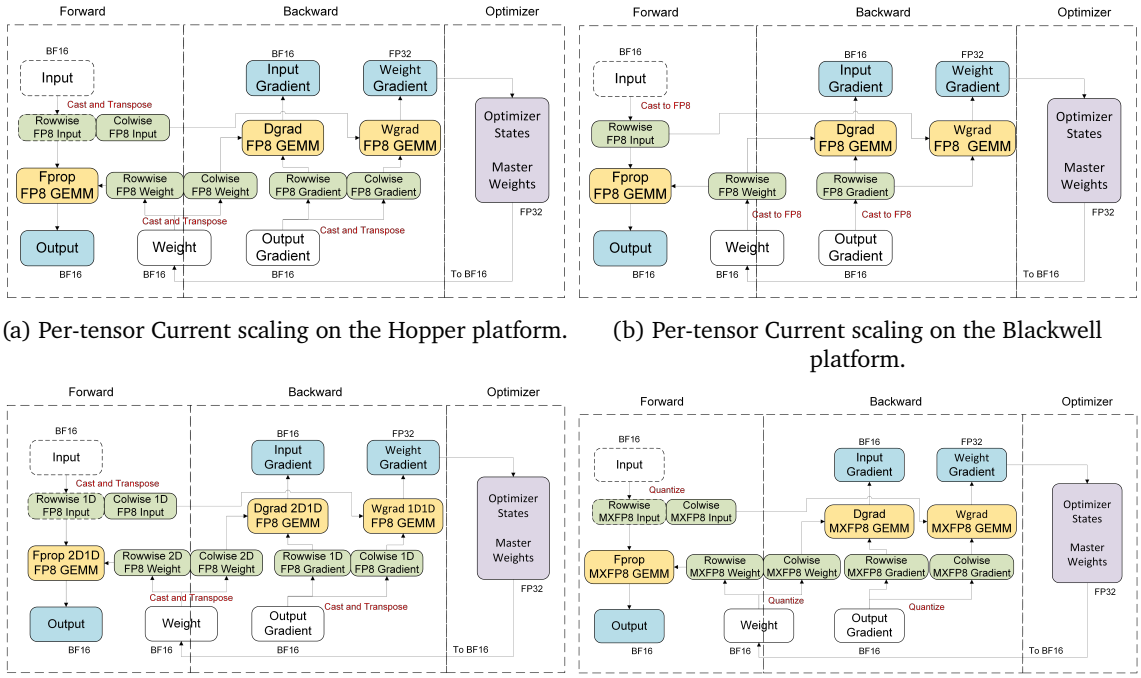
The blockwise FP8 recipe adopts the E4M3 format for all tensors, quantizing activations and gradients in 1×128 tiles and weights in 128×128 blocks. Blockwise scaling uses fine-grained scaling granularity for better precision and has been **proven successful in production** at very large-scale MoE models, including DeepSeek-V3 [6], Minimax-M2 [76], Ant Ling-2.0 [77], etc. As a result, blockwise FP8 is the recommended FP8 recipe on the Hopper platform.

Transformer Engine provides highly optimized quantization kernels and GEMM kernels (via cuBLAS) for the blockwise recipe, as well as layer-level APIs and an autocast context to enable FP8 training with just a few lines of code.

Figure 31c illustrates the computation of a linear layer with the blockwise FP8 recipe on the Hopper platform, which is very similar to the per-tensor current scaling on Hopper, except for tile-based quantization.

5.3.3. MXFP8 on Blackwell

On the Blackwell platform, thanks to native fifth-generation Tensor Core support for the MXFP8 format [78], we adopt MXFP8, a more fine-grained quantization scheme for training. Both activations and weights are quantized at 1×32 granularity, and E8M0 is used for the scaling factor. Theoretically, MXFP8 should be more



(a) Per-tensor Current scaling on the Hopper platform.

(b) Per-tensor Current scaling on the Blackwell platform.

(c) Blockwise FP8 recipe on Hopper.

(d) MXFP8 recipe on the Blackwell platform.

Figure 31: The computation of a linear layer with various FP8 recipes. Note the differences in quantization granularity and tensor layout requirements across platforms.

precise due to the finer-grained scaling granularity, and has better performance due to the native support of MXFP8 in the Tensor Core. Therefore, MXFP8 is the default FP8 recipe on the Blackwell platform.

Figure 31d illustrates the computation of a linear layer with the MXFP8 recipe on the Blackwell platform. Although FP8 GEMMs with all layouts are supported on Blackwell, we need to store additional column-wise quantized FP8 data for activations and weights due to quantization in the other direction.

Ongoing optimizations for the MXFP8 recipe in MoE training include:

1. Grouped quantization to reduce the quantization overhead.
2. Fuse activation and quantization (for the following GEMM) with the GEMM kernel.
3. Ensure that the whole pipeline of MXFP8 quantization, scaling factor swizzling, and Grouped GEMM is CUDA-graphable.

5.3.4. NVFP4 on Blackwell

In [75], we presented NVFP4 as a 4-bit microscaling format for LLM training that improves numerical fidelity while preserving the efficiency benefits of FP4.

NVFP4 uses FP4 elements in E2M1 format, so values are quantized with scaling because raw FP4 has a very limited representable range. A central design choice is two-level microscaling. NVFP4 applies a per-tensor FP32 scale and a per-block 8-bit scale in E4M3. The tensor-level FP32 scale first remaps the tensor distribution into a range compatible with block scaling, and the block-level E4M3 scale then maps each block into FP4 range. NVFP4 uses blocks of 16 contiguous elements, and each block's amax is scaled to the FP4 maximum.

Building on this format design, we also found that stable NVFP4 training depends on several algorithmic choices around quantization. In particular, we add three practical techniques:

- **Random Hadamard Transforms (RHT)**: Applied to weight gradient computation to reduce the impact of outliers.
- **2D scaling**: Specifically, 16x16 weight block scaling (with the FP32 tensor scale retained) is used to keep weight quantization more consistent between forward and backward passes and reduce forward/backward quantization mismatch.
- **Stochastic rounding**: Used on gradients to reduce rounding bias during FP4 conversion, as deterministic rounding introduces bias that hurts convergence.

These additions are a key part of the training recipe used in the paper and are important for convergence at larger scales.

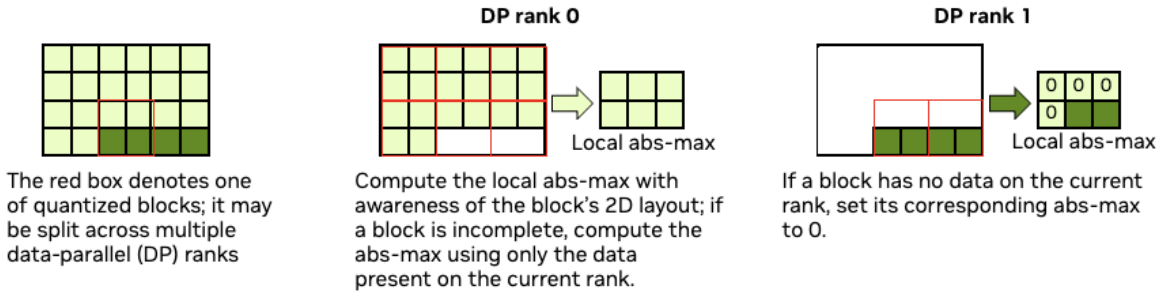


Figure 32: FP8 primary weight quantization scheme for blockwise scaling.

5.3.5. FP8/FP4 Primary Weights: Eliminating Redundant Storage

Conventional reduced-precision training maintains a three-tier parameter hierarchy: FP32 master weights for optimizer updates, BF16 model weights as an intermediate representation, and FP8/FP4 weights for forward and backward GEMM computation. This introduces redundant memory overhead by maintaining both BF16 and FP8/FP4 copies of parameters.

Native FP8/FP4 eliminates this redundancy by establishing a direct casting path from FP32 master weights to FP8/FP4 computation weights, bypassing the BF16 intermediate layer entirely, reducing memory footprint and accelerating parameter AllGather.

The core challenge in implementing native FP8/FP4 lies in managing the quantization metadata required for FP8/FP4 tensors. We provide a unified interface supporting different FP8/FP4 recipes (delayed scaling, current scaling, blockwise scaling, MXFP8 and NVFP4) that handles version-specific differences in TransformerEngine's implementations, ensuring compatibility across versions.

During each optimizer step in the distributed optimizer, the FP32 parameter shards are updated based on reduced gradients. Subsequently, these updated FP32 shards are directly quantized to FP8/FP4 format using the appropriate scaling factors. The quantization process computes the amax from the FP32 values, updates the scaling factors across data-parallel ranks to maintain consistency, and writes the quantized FP8/FP4 values to the model parameter buffers. This direct path eliminates the memory overhead of maintaining BF16 parameters, a reduction particularly impactful for large language models where parameter memory can exceed hundreds of gigabytes.

More specifically, the quantization of FP32 shards for delayed scaling and current scaling proceeds in three steps, see figure 33:

- Step 1: Get local abs-max from master weights. If a weight has no sharded part on the current rank, set its corresponding abs-max to 0.
- Step 2: Get global abs-max through AllReduce.

- Step 3: Use global abs-max and master weights to do partial cast.

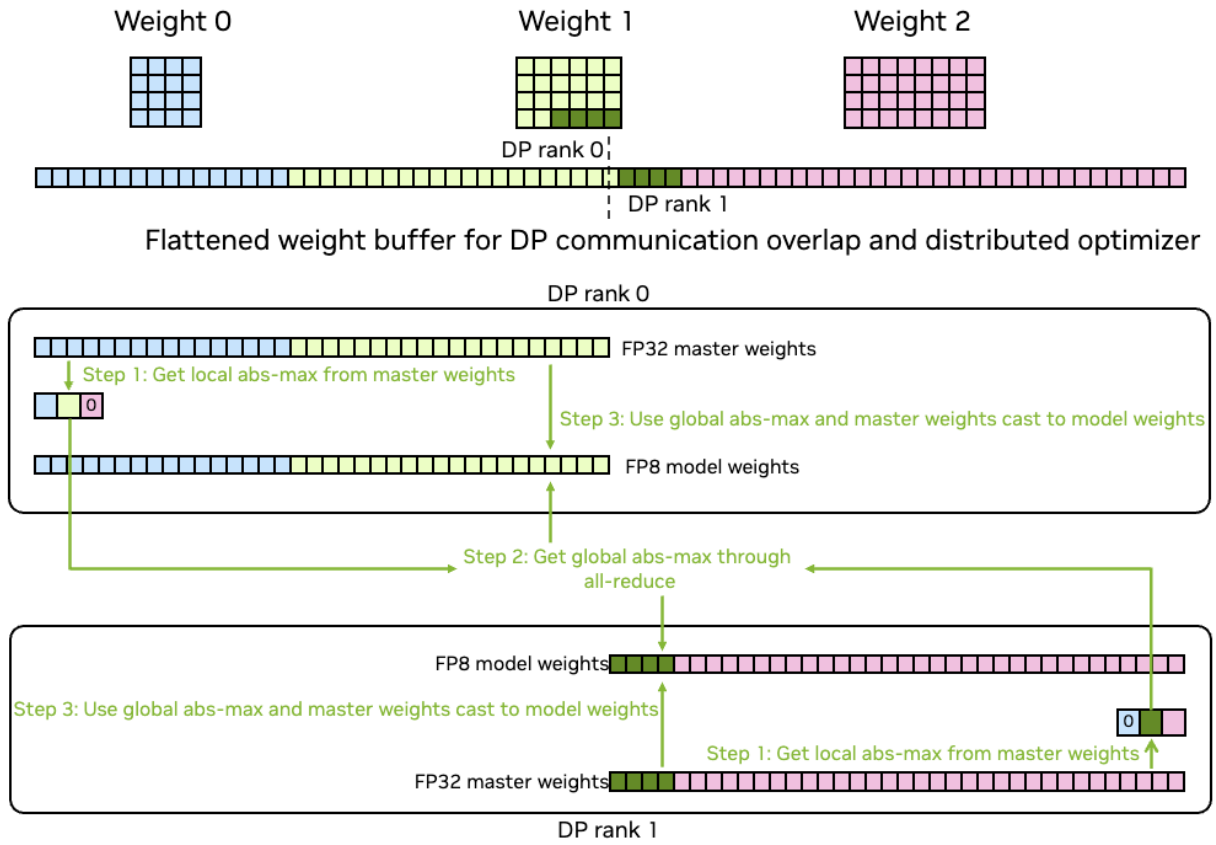


Figure 33: FP8 primary weight quantization scheme for delayed scaling and per-tensor current scaling.

In the Blockwise recipe, the abs-max of a weight is computed over a 2D block, so we cannot directly use a reduction kernel to obtain the local abs-max. We implemented specialized kernels that are aware of the weight's 2D layout and the correspondence between the master weight and the original weight to compute the abs-max, see figure 32.

Another advantage of FP8/FP4 primary weights is that the parameter AllGather communication volume is greatly reduced (see Section 5.2.2).

5.4. MoE-Specific Challenges and Solutions of Reduced-Precision Training

The preceding subsections covered the fundamentals of reduced-precision training applicable to any deep learning model. MoE architectures introduce unique challenges for reduced-precision training that require specialized solutions beyond standard implementations.

5.4.1. Fusion of Padding and Unpadding: Dynamic Shape Alignment

FP8 and FP4 GEMMs require tensor dimensions to be aligned to specific multiples: 16 for per-tensor and blockwise recipes, and 32 for MXFP8 and NVFP4. These padding requirements arise from the block scaling granularity of each quantization recipe, as well as the requirement that Tensor Memory Accelerator (TMA) accesses maintain 16-byte alignment along the GEMM dot-product dimension. In the forward pass, the hidden dimension K is the dot-product dimension and is usually already aligned by model design. However, in the weight-gradient GEMM, the dot-product dimension corresponds to the token dimension M, which varies

dynamically and often does not satisfy the alignment requirement. As a result, zero-padding along the token dimension is necessary. To further enable grouped quantization kernels with lower CPU overhead and CUDA Graph compatibility, we increase the tokens-per-expert padding to 128. Section 5.4.3 explains why this larger padding is required for the NVFP4 grouped quantization kernel; the same principle also applies to MXFP8 grouped quantization. In summary, the final padding applied to the token dimension is determined jointly by the requirements of all kernels involved in the routed expert layer.

Our baseline solution is the fused padding and unpadding kernels for multiple input tensors for different experts. However, explicit padding and unpadding kernels introduce non-negligible overheads. We propose two solutions:

- **Routing map padding**, which pads the routing map instead of the received tokens. This ensures the number of tokens per expert is aligned to the requirement at the cost of sending only a small number of extra tokens, avoiding expensive per-tensor padding operations.
- **Fusing padding into permutation** avoids one-pass of global memory read and write and should provide the best performance. It is the default choice when available.

5.4.2. Grouped Quantization and Grouped GEMM

Grouped Quantization Due to the different padding requirements between the quantized data and scaling factors, and the complex data layouts introduced by both row-wise and column-wise quantization, a naive way to quantize the input tensors for different experts is to apply the quantization kernel to them one by one. Nevertheless, it introduces plenty of tiny kernels, which adds to the CPU overhead and is not efficient from the GPU perspective.

To tackle this problem, we implement grouped quantization kernels to fuse the quantization of multiple tensors into one single kernel, which greatly reduces the CPU overhead, improves the GPU utilization and is CUDA-Graphable.

Grouped GEMM Optimized and CUDA-Graphable grouped GEMM in Section 4.3.7.

5.4.3. NVFP4 Quantization Fusion

NVFP4 quantization is particularly more complicated because of the numerical techniques we use to preserve numerical stability, so aggressive kernel fusion is necessary to keep quantization overhead under control. In practice, our NVFP4 quantization kernel is not just a simple “scale + cast” kernel: it needs to carefully absorb the training-recipe logic, including Random Hadamard Transform, 2D scaling and stochastic rounding.

Among these, RHT fusion is the most latency-sensitive. If implemented as a separate kernel, the Hadamard transform would introduce an additional full-precision (BF16) read/write of the tensor in global memory, significantly increasing bandwidth cost. By fusing RHT with NVFP4 quantization, we perform the Hadamard transform and FP4 quantization in a single kernel, avoiding that extra BF16 traffic.

A second implementation challenge is that Blackwell NVFP4 Tensor Core GEMMs are TN-oriented, while Wgrad uses transposed activations and gradients. Therefore, when RHT is fused into NVFP4 quantization for the Wgrad path, the kernel must also absorb the transpose, rather than relying on a separate BF16 transpose kernel (which would again incur a full BF16 tensor read and write through global memory).

As a result, the fused quantization pipeline must support multiple outputs from the same high-precision input: (1) standard FP4 quantization for forward-pass GEMMs, and (2) transpose + RHT + FP4 quantization for the backward/Wgrad path. In training, we launch this fused kernel during the forward pass to generate two FP4 copies: one consumed immediately by the forward GEMM, and one saved for backward. The original high-precision input is then discarded, so we avoid storing BF16 activations while still preparing the backward path efficiently.

A further complication comes from the per-tensor FP32 scale in NVFP4. Current NVFP4 pretraining recipe, the tensor-wide abs-max is measured after the Hadamard transform, which means we need a dedicated Hadamard+amax kernel that computes only the amax (without materializing a transformed BF16 output buffer). As a result, the Hadamard transform is effectively computed twice—first in the Hadamard-amax kernel and then again in the fused quantization kernel. Although this duplicates transform compute, it is still faster end-to-end than writing out a full transformed BF16 tensor to global memory, because it avoids the extra high-bandwidth BF16 read/write traffic.

In contrast, 2D quantization is less visible in end-to-end throughput because it applies only to weights: we can quantize weights once (for example, on the first microbatch) and cache the quantized weights and scales for reuse across later microbatches, so its cost is largely amortized.

Stochastic rounding, however, adds a kernel-level requirement on the critical path: the quantization kernel must also support an optional stochastic rounding path with random number generated in kernel (enabled for gradient tensors such as dY, disabled otherwise) so that FP4 quantization remain fused in a single pass. To keep this efficient, we use NVIDIA cuRANDDx[79] for device-side random number generation inside the fused CUDA quantization kernel.

Grouped NVFP4 Quantization for MoE Supporting NVFP4 quantization for MoE layers is especially challenging due to the algorithmic complexity of the NVFP4 recipe itself. Under a full-iteration CUDA Graph requirement, the MoE quantization path must also be CUDA Graph safe: the host cannot depend on dynamic expert-token counts on the CPU, and can only rely on a device-side tokens-per-expert tensor. This means that input activation can no longer be splitted and quantized individually. We must implement grouped quantization kernels for the full NVFP4 pipeline. The memory allocation of NVFP4 output must be allocated as a whole flat buffer without knowing the shapes in between.

A key observation is that much of the dense NVFP4 activation quantization kernel can be reused for grouped quantization, because tokens for different experts are continuously packed in memory after routing/permutation. The main differences are implementation constraints needed to preserve performance and graph safety.

1. A quantization thread block should not span tokens from multiple experts, since that introduces significant control-flow and indexing overhead; therefore, we zero-pad each expert's token count to an integer multiple of the thread block shape in the token dimension (typically 128).
2. The NVFP4 transpose must be performed per expert (transpose each expert's packed activation independently, then concatenate), which is not equivalent to transposing the entire grouped buffer as one tensor.
3. NVFP4 GEMM requires scale-factor swizzling: scaling factors must be padded to a 128×4-aligned shape and swizzled into the 32×16 layout before GEMM as described in the cuBLAS document[80]. This scaling factor shape constraint is applied to per-expert GEMM, which means that determining the required padding size would implicitly require CPU visibility into tokens-per-expert, which is not CUDA Graph safe. To avoid that, we enforce 128-token alignment per expert by construction.

Both point 1) and point 3) imposed 128 multiple tokens-per-expert, which requires a specific zero padding kernel to satisfy. To eliminate such zero padding overhead, we fuse the per-expert zero-padding capability into the token-permute kernel, so the routed expert activations are produced in an already aligned and zero padded layout, and downstream grouped NVFP4 quantization/GEMM kernels can assume the required alignment without additional preprocessing.

For the per-tensor FP32 second-level scale in the NVFP4 recipe, the per-tensor second-level scale in MoE means per-expert second-level scale instead of making every expert share the amax to preserve numerical stability during training. These amax values cannot be known ahead of time: they must be computed online from the routed tokens and generated as distinct per-expert amax values on every iteration. To address this efficiently,

we leverage the 128-aligned tokens-per-expert guarantee and adapt the dense-input implementation into a CUDA-Graph-safe grouped Hadamard-amax kernel.

6. Long-Context MoE Training

The preceding sections addressed the three walls (memory, communication, and compute efficiency) under the assumption that MoE layers dominate computational cost. This assumption holds for typical training tasks with sequence lengths of 4K–8K tokens, where expert computation and all-to-all communication are the primary bottlenecks. However, the rise of reasoning models (OpenAI o1, DeepSeek-R1 [81], etc.) and RL-based training have created a new frontier: sequences of 16K, 64K, or even longer. At these lengths, a fundamental shift occurs in the performance characteristics of MoE training: attention dominates the computation, and the relative importance of the three walls changes accordingly. This section examines how long-context training reshapes the three walls for MoE models, and how Megatron-Core addresses the new challenges.

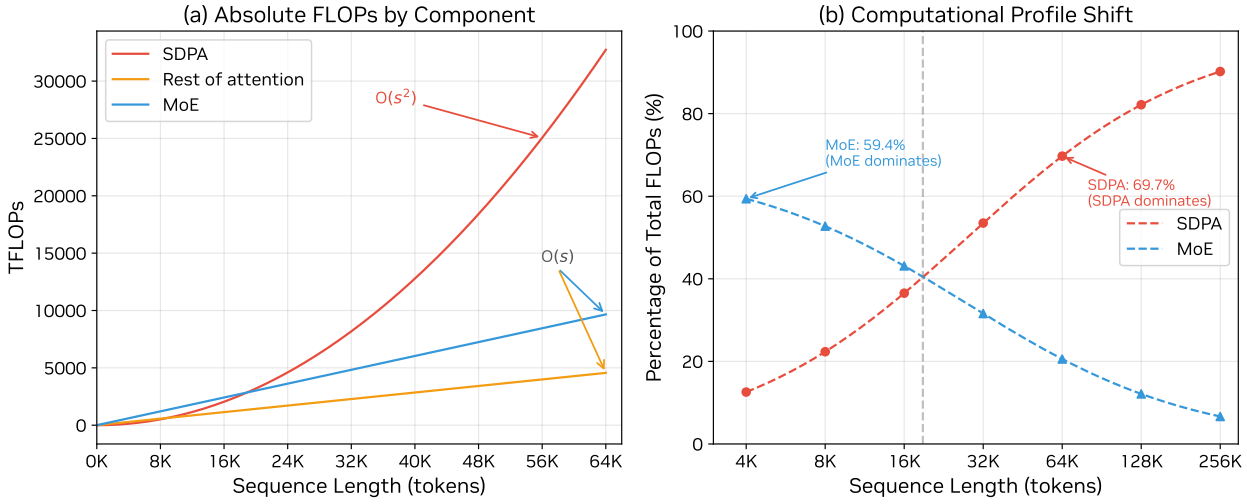


Figure 34: SDPA exhibits $O(s^2)$ complexity, while MoE and the remaining attention operations exhibit $O(s)$ complexity. Therefore, SDPA dominates the computation at longer sequence lengths.

6.1. When Attention Dominates: The Computational Shift

In Section 4, we established that MoE training faces three fundamental walls: memory, communication, and compute efficiency. The optimizations presented there (activation management, dispatcher optimization, Grouped GEMM, and CUDA Graphs) target scenarios where MoE layers dominate the computational profile. Long-context training fundamentally alters this assumption. The key insight is the differing computational complexity between MoE and attention layers, as shown in figure 34:

- **MLP Components**: Computational cost scales linearly with sequence length, exhibiting $O(s)$ complexity.
- **Attention Components**: Scaled dot-product attention (SDPA) dominates the computational cost, scaling quadratically with sequence length, exhibiting $O(s^2)$ complexity. This quadratic scaling has motivated extensive research into efficient attention mechanisms [82, 83].

For example, at 64K tokens, SDPA consumes 69% of FLOPs, compared to only 10–15% in short-sequence scenarios. **Attention becomes dominant in computation.** Fortunately, SDPA is highly optimized in most GPU-accelerated libraries such as FlashAttention [84, 85, 86] and cuDNN (Table 9), so SDPA does not become a performance bottleneck. The optimization focus therefore shifts to memory and communication: as long as we address these two challenges without introducing excessive overhead, training performance is preserved.

Table 9: SDPA performance in cuDNN for DeepSeek-V3.

Platform	Sequence Length	Forward TFLOPS	Backward TFLOPS
Hopper	4096	553	422
	16384	638	523
Blackwell	4096	1324	1083
	16384	1698	1298

6.2. Managing Activation Memory Growth

Activation memory growth with sequence length is the primary challenge in long-context training. To address this intensified memory wall, we apply a set of techniques that work together, informed by large-scale MoE workloads, including DeepSeek-V3 [6] and Qwen3 [87]. These techniques extend the memory-optimization principles introduced in Section 4.1.

Context Parallelism and Tensor Parallelism. Combining Context Parallelism (CP) and Tensor Parallelism (TP) distributes activation memory across devices, enabling sequence lengths that would otherwise exceed GPU capacity. The key principle is to keep sub-sequence length (sequence length per CP/TP shard) approximately constant, typically 4096 or 8192. Scaling CP $\times$ TP with sequence length keeps per-device memory near baseline levels. This makes the workload resemble short-context training, except for SDPA and TP/CP-specific communication, so most short-context optimizations remain applicable.

Optimizer CPU Offloading. Optimizer states often consume tens of gigabytes per GPU in large models. CPU offloading can reclaim this memory almost entirely, at the cost of transfer and host-side optimizer overhead. The choice is therefore a trade-off between memory headroom and throughput. For DeepSeek-V3 on 256 H100 GPUs, at around 50% MFU and sequence lengths of at least 16K, the worst-case overhead is about 2%. Although the exact trade-off depends on model size, sequence length, and cluster scale, optimizer CPU offloading is usually worthwhile for long-context training.

Selective Recomputation. Recomputation trades compute for memory by discarding intermediate activations in forward and regenerating them in backward. The key is module-level selectivity based on memory-to-compute trade-offs. In long-context settings, SDPA dominates total computation, so recomputing SDPA (core attention recomputation in Megatron-Core) is usually too expensive. In contrast, recomputing lower-cost components such as MLP-related modules often provides better net benefit. For DeepSeek-V3 at 64K sequence length, SDPA contributes up to 72% of total compute; recomputing it adds about 18% compute overhead, causes about 16% performance loss, and saves only 9 GB memory. Recomputing non-SDPA components instead saves 89.8 GB globally with comparable or lower performance impact. We therefore recommend disabling core attention recomputation and prioritizing recomputation of other modules.

In practice, CP and TP are the primary mechanisms for long-context memory efficiency. Optimizer CPU offloading and selective recomputation then provide additional headroom, especially on memory-constrained platforms. These techniques work well together and can be combined as needed.

6.3. Context Parallelism vs. Tensor Parallelism

Because CP and TP are the two most effective strategies in long-context training, the practical question is how to combine them when attention dominates computation.

Both CP and TP reduce activation memory in long-context training, but their communication patterns and memory effects differ (Figure 35). CP partitions activations along the sequence dimension and reduces activation memory by a factor of CP. For SDPA, CP has two communication modes: point-to-point (P2P) and all-to-

all. P2P CP exchanges the KV cache within the CP group in a ring-style pattern that overlaps with SDPA computation [47, 88, 48]. All-to-all CP transforms tensors from sequence-sharded to head-sharded form before SDPA, then restores sequence-sharded form afterward. TP additionally shards linear weights, reducing parameter memory but introducing extra collectives in linear layers. In both all-to-all CP and TP, SDPA runs on head-sharded tensors, which increases per-shard sub-sequence length and can improve SDPA kernel efficiency.

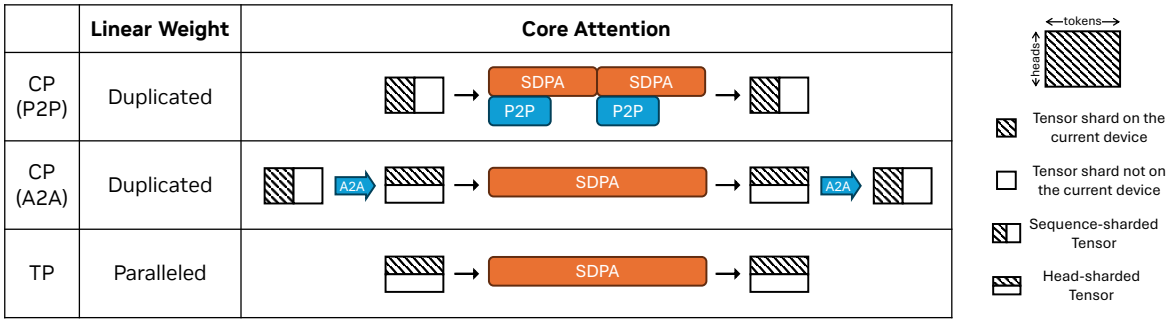


Figure 35: Communication and computation patterns of TP and two types of CP.

In practice, P2P CP is the more common CP mode. Relative to TP, its main advantage is that communication overlaps naturally with computation, while TP communication is often harder to hide. TP, however, reduces dense parameter memory and can improve SDPA efficiency. As a result, TP is usually preferred within a node, where communication is fast and memory benefits are strong. Across nodes, P2P CP often becomes preferable because TP communication overhead grows while P2P overlap remains effective. All-to-all CP sits between the two: it avoids multi-step ring exchange used by P2P CP and avoids linear-weight sharding overhead introduced by TP. In practice, all-to-all CP is often combined with TP for intra-node execution when ring-style exchange is undesirable and TP communication cost is high.

Megatron-Core supports combining both CP backends as **hierarchical CP**, enabling topology-aware pairing with TP. A practical starting point is to use all-to-all CP with TP inside nodes to improve SDPA kernel performance and reduce parameter memory, while using P2P CP across nodes to preserve communication-computation overlap. These are guidelines rather than hard rules, but they work well as an initial configuration for TP/CP tuning.

6.4. Packed Sequences for Variable-Length Training

The preceding subsections examined how the Three Walls (memory, communication, and compute efficiency) shift when sequence length increases. These analyses assumed fixed-length sequences within each batch. However, practical training scenarios, particularly reinforcement learning and supervised fine-tuning, often involve variable-length sequences that introduce additional efficiency challenges.

Traditional batching requires padding all sequences to the maximum length within each batch, leading to substantial waste when sequence lengths vary widely [89]. For instance, if a batch contains sequences of lengths 4K, 8K, and 32K tokens, all sequences must be padded to 32K, wasting ∼60% of computation and memory on padding tokens. This inefficiency compounds the memory and compute challenges discussed above.

To address this, Megatron-LM introduces **packed sequences** to process variable-length sequences within a single batch without padding waste. Furthermore, to address the DP imbalance and CP inefficiency caused by variable-length sequences, we introduce **Dynamic Context Parallelism (Dynamic-CP)**, which adaptively selects the effective CP size on a per-microbatch basis, jointly with the sequence packing plan.

6.4.1. Packed Sequence Support

Megatron-LM supports packed sequences, enabling multiple variable-length sequences to be concatenated and processed within a single batch without inter-sequence padding. This is achieved through the THD (Total tokens × Heads × Dimension) tensor format, as opposed to the conventional SBHD (Sequence × Batch × Heads × Dimension) format. THD represents attention tensors as `[total_tokens, num_heads, head_dim]`, where sequences from different samples are concatenated along the token dimension.

The core mechanism relies on cumulative sequence length tracking, which marks the start and end positions of each individual sequence within the packed tensor. These parameters are propagated through the attention mechanism to Transformer Engine's fused kernels, enabling efficient SDPA and RoPE operations that respect sequence boundaries. The attention computation uses cumulative sequence lengths to ensure queries and keys from different sequences do not attend to each other, maintaining correctness while eliminating padding overhead.

The implementation in Megatron-LM supports packed sequences across multiple training scenarios:

- **Reinforcement Learning**: Bin-packing algorithms group variable-length trajectories into fixed-size bins, maximizing GPU utilization.
- **Multimodal Training**: Vision-language models with variable image token counts use packed sequences to handle heterogeneous sequence lengths from different numbers of visual patches combined with text tokens.
- **Context Parallelism Integration**: When Context Parallelism is enabled with padding requirements, the system automatically switches to THD format and provides both padded and unpadded cumulative lengths to handle communication alignment while preserving computational efficiency.

The benefits of packed sequence support are particularly pronounced in long-context scenarios. For RL training with thinking models, where sequence lengths can vary from hundreds to tens of thousands of tokens, packed sequences can reduce memory usage by 40–60% and improve training throughput by 1.5–2× compared to traditional padding-based approaches. This efficiency gain becomes increasingly critical as context lengths extend beyond 32K tokens, where padding waste would otherwise dominate memory consumption and significantly reduce effective batch sizes.

6.4.2. Dynamic Context Parallelism for Packed Sequences

While packed sequences in THD format remove padding waste, they do not eliminate computational imbalance across data-parallel (DP) ranks. Even when packed samples have identical total token counts, their attention workload can still vary substantially due to the quadratic complexity of dot-product attention with respect to sub-sequence lengths. For example, Figure 36 shows sequences packed to equal total lengths, yet as Figure 37 reveals, their compute workloads differ significantly.

This DP imbalance leads to GPU idling at gradient synchronization points and can further amplify pipeline-parallel bubbles. Moreover, when Context Parallelism (CP) is enabled, a static CP size is typically chosen to satisfy the memory requirement of the longest packed sample in the batch. Consequently, shorter packed samples that would fit on fewer devices are still forced to use the same CP size, incurring unnecessary CP communication. In practice, CP communication is expected to be hidden by attention computation; however, under packed sequences that consist of many short sub-sequences, the compute per CP shard may become insufficient to hide CP collectives, especially when CP spans inter-node links. We refer to this effect as CP inefficiency.

To address both DP imbalance and CP inefficiency, we introduce Dynamic Context Parallelism (Dynamic-CP) [90], which adaptively selects the effective CP size on a per-microbatch basis, jointly with the packing plan. The key observation is that resizing CP is comparatively lightweight: it only changes how token slices

(a) Unpacked sequences. (b) Packed sequences.

Figure 36: Unpacked vs. Packed sequences.

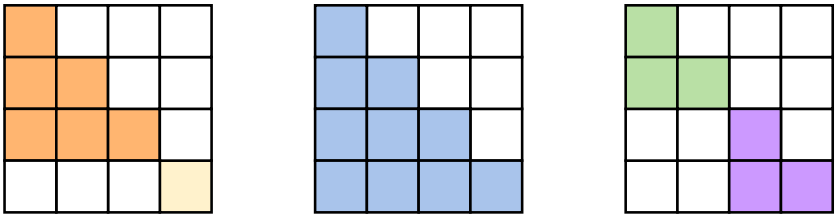
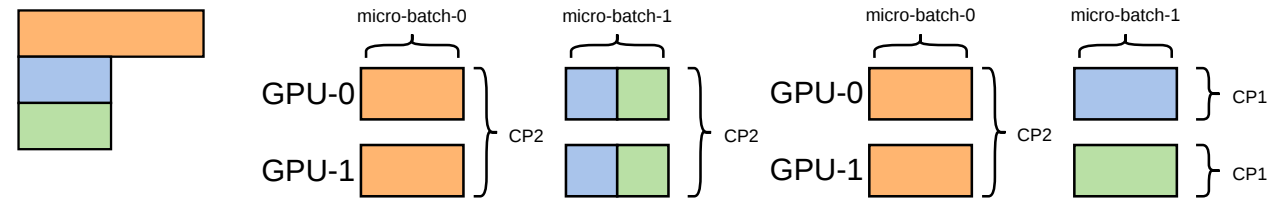


Figure 37: Compute imbalance in causal attention over packed sequences.

are partitioned and which CP communication group is used by attention operators, without requiring any parameter redistribution or optimizer-state migration. Therefore, Dynamic-CP provides a practical form of dynamic parallelism for variable-length training with minimal framework overhead. Related work, including ByteScale [91] and WLB-LLM [92], addresses similar load-balancing challenges.

For example, consider the case in Figure 38a with three sequences. We assume only two GPUs are available. If we adopt the standard CP2 configuration (i.e., DP=1, CP=2), the workload is executed in two micro-batches as illustrated in Figure 38b. In microbatch-0, the orange sequence is sufficiently long that it must be partitioned across the two GPUs. In microbatch-1, the packed blue and green sequences are also partitioned across the two GPUs. However, this introduces an unnecessary split for the blue and green sequences, since both sequences can fit on a single GPU without partitioning.

As shown in Figure 38c, with Dynamic-CP, the orange sequence in microbatch-0 still requires CP=2. In microbatch-1, we no longer need to split the blue and green sequences; instead, we let each of them run with CP=1 independently.



(a) Input sequences. (b) Standard CP2. (c) Dynamic-CP.

Figure 38: Dynamic Context Parallelism for Packed Sequences.

To make per-microbatch CP resizing practical in Megatron-Core, Dynamic-CP avoids any expensive redistribution of model states. Instead, it treats CP resizing as a lightweight runtime choice: changing only how token slices

are partitioned and which CP communication group is used by attention operators. Concretely, rather than binding each rank to a single statically constructed `cp_group`, the framework pre-constructs multiple CP groups per rank during initialization, with candidate `cp_size` ranging from 1 up to `dp`×`cp` (restricted to powers of two). At runtime, the scheduler selects the effective `cp_size` for each microbatch and the corresponding pre-built `cp_group`, enabling Dynamic-CP without incurring the overhead of dynamically creating communication groups.

Dynamic-CP is designed for the THD layout, where variable-length sequences are packed under a length constraint and the original batch/sequence dimensions collapse into a token dimension. A direct consequence is that the number of microbatches is no longer a fixed quantity derived from `global_batch_size` and `micro_batch_size`; it can vary across iterations because the number of original sequences packed into each microbatch is not constant. To minimize invasive changes to existing Megatron-Core pipeline schedulers, Dynamic-CP introduces a lightweight `data_iterator_wrapper` around the original `data_iterator`. For each global batch, the wrapper (i) reschedules and packs sequences to create balanced workloads across DP ranks, (ii) chooses an effective `cp_size` for each microbatch to avoid over-sharding short packed samples, and (iii) returns the effective `num_microbatches` for the current iteration. Because only a subset of ranks typically drives scheduling decisions under PP/VPP, the framework broadcasts the dynamic scheduling metadata (including `num_microbatches`, `max_seqlen`, and `cu_seqlens`) to ensure consistent execution across pipeline stages. Furthermore, `PackedSeqParams` is extended to carry both the selected `cp_size` and `cp_group`; all CP-dependent components (e.g., position embedding and attention) retrieve CP configuration from `PackedSeqParams` rather than relying on globally static CP variables.

Given variable-length sequences in THD, different samples contribute different numbers of valid tokens. Therefore, loss is computed on a per-token basis to avoid bias from padding:

$$\mathcal{L} = \frac{\sum_{t \in \mathcal{V}} \ell_t}{|\mathcal{V}|}, \tag{3}$$

where $\mathcal{V}$ denotes the set of valid (non-padding) tokens in the packed representation. On the planning side, the Dynamic-CP solver jointly determines packing and per-microbatch CP sizes under GPU memory constraints. Since attention compute scales roughly as $\mathcal{O}(S^2)$ with respect to sub-sequence lengths while activation memory scales closer to $\mathcal{O}(S)$, it is difficult to simultaneously balance compute and memory. Dynamic-CP therefore alternates between workload-oriented and memory-oriented decisions: microbatches whose estimated workload exceeds a target quota are assigned larger `cp_size` to reduce per-rank compute, after which memory becomes the dominant constraint and remaining capacity is filled by selecting less compute-heavy samples while preserving feasibility.

Finally, Dynamic-CP is engineered to keep runtime overhead negligible. Constructing a plan requires an additional pass over the global batch to probe lightweight shape and sequence-length metadata; this I/O pressure is mitigated by distributing the probing across the cluster and gathering only compact metadata. The solver itself runs asynchronously (e.g., within the `data_sampler`) to overlap with training iterations. To keep the search space manageable, exhaustive search is replaced by a one-dimensional grid search over the microbatch count, constrained so that all DP ranks use the same `num_microbatches`. This count is swept from `PP`×1 up to a small multiple of `PP`, capturing the trade-off between per-microbatch workload and pipeline bubbles; in practice, the search can be further narrowed by selecting the "knee" point on the workload-versus-microbatch curve and exploring only its neighborhood.

Overall, Dynamic-CP complements packed sequence training by (i) reducing synchronization stalls caused by DP imbalance and (ii) preventing unnecessary CP communication for microbatches that do not benefit from large CP sizes, thereby improving throughput in long-context variable-length training. According to the benchmark, Dynamic-CP yields a 35–60% end-to-end performance improvement in real-world scenarios with

highly imbalanced sequence length distributions, such as multi-modal training. [5]

6.5. Summary

Long-context MoE training represents a distinct optimization regime where the fundamental assumptions of Section 4 must be revisited. While the Three Walls (memory, communication, and compute efficiency) remain the fundamental barriers, their relative importance shifts substantially as sequence length increases:

- **Computational Shift**: Attention's $O(s^2)$ complexity displaces MoE layers as the dominant computational bottleneck, consuming $>$ 69% of FLOPs at $>$ 64K tokens. The MoE-focused optimizations of Section 4.3 remain valuable but no longer address the primary bottleneck.
- **Memory Wall Intensified**: Activation memory scales with sequence length, requiring the parallelism strategies from Section 3 (specifically CP and TP) combined with the memory techniques from Section 4.1 (recomputation, offloading) to maintain feasible memory footprints.
- **Communication Trade-offs**: The choice between CP and TP involves nuanced trade-offs between communication overhead, memory savings, and SDPA computational efficiency, a balance distinct from the MoE-centric all-to-all optimization focus of Section 4.2.

By combining context parallelism, tensor parallelism, selective recomputation, and optimizer offloading, Megatron-Core enables efficient training across diverse sequence lengths. Our experiments on 256 Hopper GPUs at 256K sequence length demonstrate this: DeepSeek-V3 achieves 88% of its short-context MFU using TP, optimizer CPU offloading, and selective recomputation, while Qwen3-235B-A22B reaches 129% of its short-context MFU with TP, CP, and selective recomputation (the latter exceeding 100% because SDPA kernels are highly efficient when dominating the computation at longer sequences). The packed sequence and Dynamic-CP features further extend these capabilities to variable-length workloads in RL and SFT training.

7. Production Features

The previous sections focused on performance optimizations that address the Three Walls. However, production MoE training also requires robustness, flexibility, and ease of use. This section describes features that address these operational requirements: load balancing to ensure stable training, shared and latent expert architectures, distributed checkpointing for flexible deployment, upcycling to use existing dense models, and integration with advanced training paradigms like multi-token prediction.

7.1. Load Balancing and Token Dropping

Dynamic routing in MoE models can lead to significant workload imbalance, where certain experts receive disproportionately more tokens than others. This causes computational inefficiency, memory bottlenecks, and degraded hardware utilization. We address these challenges with two coordinated mechanisms: load balancing and token dropping.

Load Balancing. Figure 39 illustrates the load balancing strategies available in Megatron-Core MoE. We provide multiple strategies to encourage uniform token distribution. The primary approach employs an **auxiliary loss** [12, 4], a differentiable penalty term that discourages routing all tokens to a small subset of experts. We also support **expert choice routing** [93], which formulates balanced routing as an optimal transport problem, and **auxiliary-loss-free balancing** via learnable expert bias terms [22] that dynamically adjust routing decisions based on historical load.

Token Dropping. We support two dispatch strategies with different capacity management policies. In *dropless mode* (the default), all routed tokens are processed without capacity constraints, maximizing model

[5] Explore PR#2000 and PR#2959 to start training models with variable-length sequences using Megatron-Core optimizations.

expressiveness at the cost of variable per-expert workload. In *droppable mode*, explicit expert capacity limits are enforced [12, 4]: when tokens assigned to an expert exceed its capacity, excess tokens are dropped and bypassed through residual connections. This provides predictable memory bounds, useful during early training when the router is poorly initialized.

Pad-to-Max for Static Shapes. Droppable mode also enables *pad-to-max* functionality, which pads all expert inputs to the same capacity limit. This converts MoE's inherently dynamic per-expert token counts into static shapes, enabling optimizations like CUDA Graphs (Section 4.3.6) that require fixed tensor dimensions across iterations.

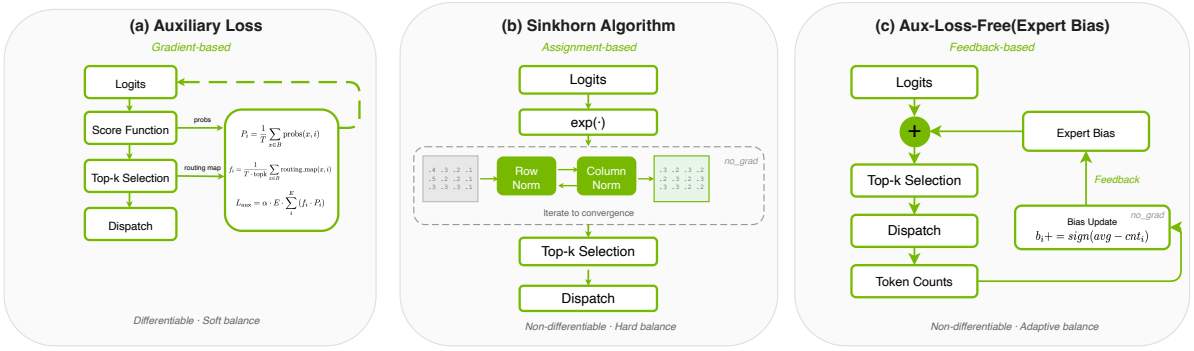


Figure 39: Load balancing strategies in Megatron-Core MoE.

7.2. Shared Experts

Some MoE architectures (DeepSeek-V2/V3 [17, 6], Qwen [32]) include a shared expert that processes all tokens regardless of routing, providing consistent baseline capacity across all tokens (figure 40). When overlap is enabled (`--moe-shared-expert-overlap`), shared expert computation runs in parallel with the all-to-all communication and routed expert computation, hiding its latency behind the dispatch-compute-combine pipeline.

7.3. Latent MoE

In standard MoE, all2all communication dispatches tokens at the full hidden dimension d, and each expert is parameterized by weight matrices in $\mathbb{R}^{m\times d}$ and $\mathbb{R}^{d\times m}$. LatentMoE [19] reduces both costs by inserting a shared down-projection $W_{\downarrow} \in \mathbb{R}^{\ell\times d}$ before dispatch and an up-projection $W_{\uparrow} \in \mathbb{R}^{d\times \ell}$ after combine, where $\ell < d$ is the latent dimension. Routing still operates on the full hidden dimension to preserve routing quality; each routed expert operates entirely in the compressed latent space; shared experts remain at dimension d. The layer output becomes:

$$\text{output}(\mathbf{x}) = W_{\uparrow} \cdot \Big(\sum_{i \in \mathcal{T}_{K,E}} p_i \, E_i(W_{\downarrow} \cdot \mathbf{x};\, \ell) \Big) + \sum_j E_j^{\text{shared}}(\mathbf{x};\, d).$$

The compression ratio $\alpha = d/\ell$ reduces all2all communication volume by a factor of α (tokens are dispatched at dimension ℓ instead of d) and per-expert weight size by the same factor (expert matrices shrink from $\mathbb{R}^{m\times d}$ to $\mathbb{R}^{m\times \ell}$). Elango et al. [19] define two ways to exploit these savings. In ℓ-MoE$_{\text{eff}}$, the total expert count E is scaled by α while top-K is unchanged, preserving baseline accuracy at reduced inference cost. In ℓ-MoE$_{\text{acc}}$ (recommended), both E and top-K are scaled by α, which restores inference cost to the standard-MoE level but exponentially expands the combinatorial space of expert selections ($\binom{\alpha E}{\alpha K} \geq \binom{E}{K}^{\alpha}$), yielding higher accuracy at iso-cost. At scales up to 95B parameters, ℓ-MoE$_{\text{acc}}$ consistently outperforms standard MoE in accuracy per

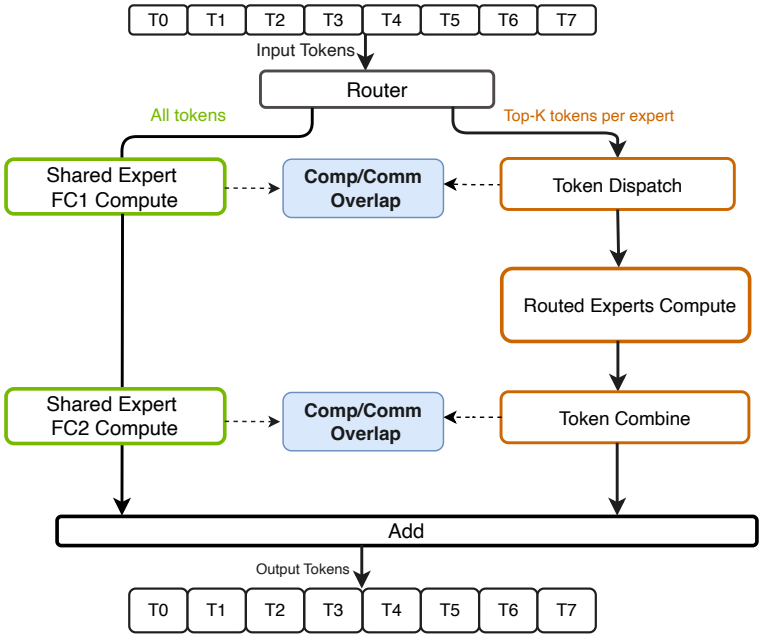


Figure 40: Shared expert architecture in Megatron-Core MoE. The shared expert processes *all* tokens while routed experts process only their assigned tokens. When overlap is enabled, shared expert computation runs in parallel with the token dispatch/combine communication, hiding its latency.

FLOP and per parameter. The architecture has been adopted by NVIDIA's Nemotron-3 Super and Ultra models.

Implementation. LatentMoE is enabled via `--moe-latent-size`, which sets ℓ. The `MoELayer` instantiates two `TELinear` projections: `fc1_latent_proj` (down-projection in the `preprocess` step, after routing but before dispatch) and `fc2_latent_proj` (up-projection in the `postprocess` step, after combine). Expert backends (`TEGroupedMLP`, `SequentialMLP`) automatically adapt their input and output dimensions to ℓ.

7.4. Distributed Checkpoint

Traditional checkpointing tightly couples saved model state with the specific parallelism configuration, requiring complex offline conversion when changing TP, EP, PP, or other parallelism settings. Our distributed checkpoint library solves this through **parallelism-agnostic checkpointing** with automatic resharding.

The core abstraction is the `ShardedTensor` descriptor, which encodes each local tensor's global shape, offset, and sharding pattern. During saving, each rank independently writes its local shard (Fully Parallel Saving), eliminating coordinator bottlenecks. During loading, each rank determines which portions of global tensors it needs based on the *new* sharding specification and reads only those slices.

This enables any-to-any parallelism reconfiguration: a checkpoint saved with TP $= 2$, EP $= 4$ can be loaded with TP $= 4$, EP $= 8$ without offline conversion. The library supports Zarr (default) and PyTorch Distributed [53] storage backends.

7.5. Flexible Asymmetric Virtual Pipeline Parallelism

Interleaved pipeline parallelism [94] divides each physical pipeline stage into multiple virtual stages with interleaved scheduling, mitigating pipeline bubbles. Traditional VPP requires uniform layer distribution (e.g., a 24-layer model with PP $= 4$, VPP $= 2$ distributes layers as $[6, 6, 6, 6]$). However, MoE models exhibit substantial

workload heterogeneity: MoE layers, dense layers, embedding, loss, and specialized layers like MTP have significantly different computational costs.

We introduce **Flexible Asymmetric VPP**, which allows different numbers and types of layers per virtual stage. Consider DeepSeek-V3 with 61 decoder layers and 1 MTP layer at PP $= 16$, VPP $= 2$ (Table 10). The initial rank combines the lightweight embedding with 3 dense decoders (matching the cost of 2 MoE layers). Most ranks hold 2 MoE decoders per stage. The final ranks strategically place the heavy MTP layer and lightweight loss layer to balance workload.

This approach enables VPP for models with arbitrary layer counts, achieves true load balancing by accounting for per-layer computational costs, and provides flexible placement for specialized layers.

Table 10: Layer distribution for DeepSeek-V3 with flexible asymmetric VPP (PP $= 16$, VPP $= 2$).

PP rank	VPP rank 0	VPP rank 1
0	embedding + 3× decoder	2× decoder
1–13	2× decoder	2× decoder
14	2× decoder	MTP
15	2× decoder	loss

This flexible asymmetric approach (figure 41) delivers several key advantages: (1) it enables VPP for models with arbitrary layer counts and compositions, removing artificial constraints on model architecture design; (2) it achieves true computational load balancing by allowing fine-grained control over layer distribution, accounting for the vast differences in computational cost between layer types; and (3) it provides flexible placement strategies for specialized layers (MTP, encoder-decoder structures, etc.), enabling diverse model architectures to fully exploit the latency-hiding benefits of pipeline parallelism. Practitioners can design layouts that distribute computationally expensive layers across ranks to avoid memory pressure and minimize pipeline bubbles.

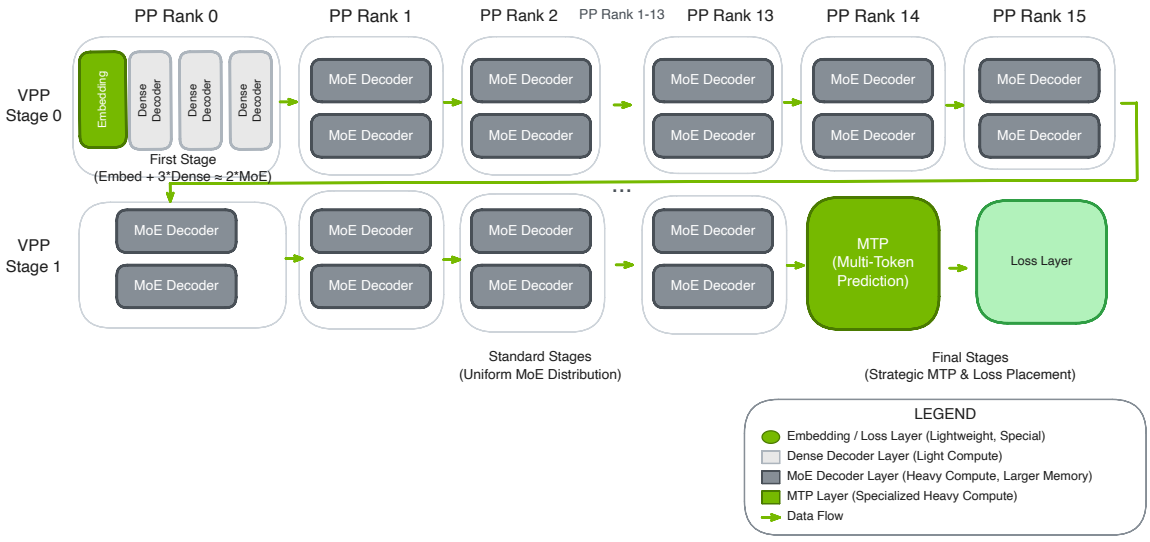


Figure 41: **Flexible Pipeline Parallel Placement.**

7.6. Upcycling

Upcycling converts a pre-trained dense model into a sparse MoE architecture, expanding model capacity without retraining from scratch [95, 96, 97]. We support virtual group initialization and expert weight scaling to enable seamless adaptation of dense checkpoints into fine-grained MoE models. The approach employs softmax-then-topK routing, which routes tokens through a selective subset of experts for increased expressivity at constant or reduced inference cost, as illustrated in figure 42.

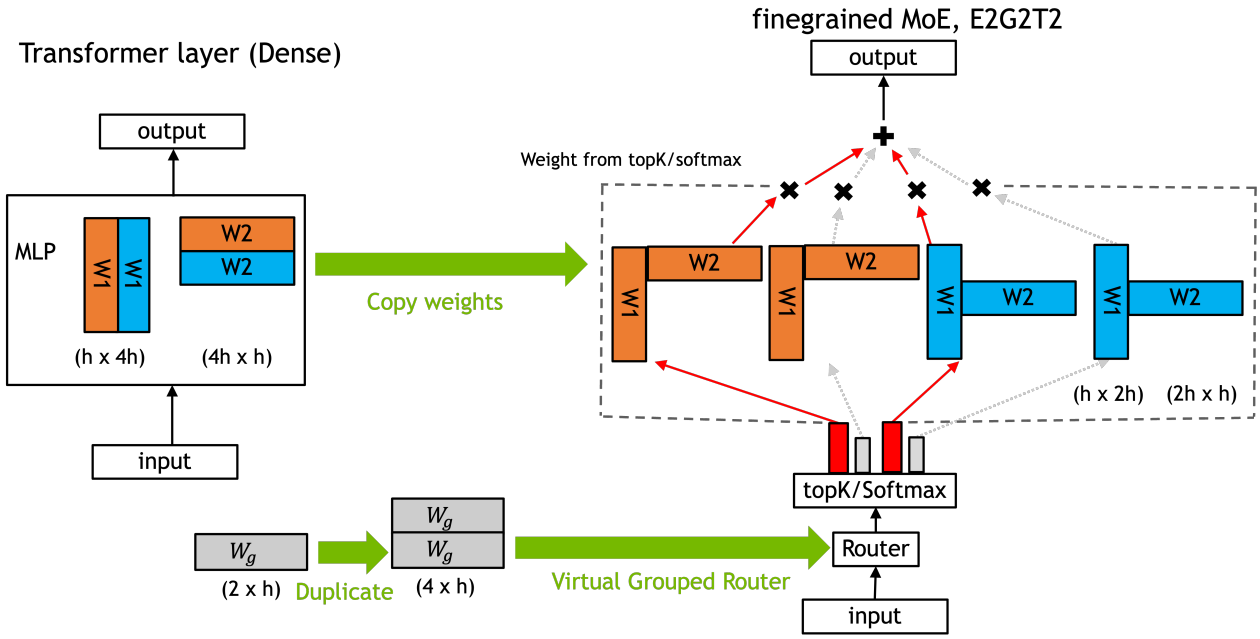


Figure 42: An example of granular upcycling a dense layer into E2G2T2 fine-grained MoE. E2G2T2 denotes 4 experts, top 2, with half intermediate size. (1) We shard MLP weights in the intermediate dimension ($4h \rightarrow 2h$) then duplicate the shards. (2) We initialize half the router weights then duplicate them. This ensures Top2 always selects one of each MLP shard so MoE output is the same as the dense model at the start of training.

7.7. Multi-Token Prediction

Multi-token prediction (MTP) optimizes a model to predict multiple consecutive future tokens at each position, densifying the supervision signal [98, 6]. Unlike parallel independent predictions, MTP maintains causal dependencies between predictions through hidden state transitions, accelerating convergence and improving generation quality. During inference, the model reverts to standard single-token prediction for deployment compatibility.

We integrate MTP with flexible pipeline parallelism, allowing MTP layers to be placed strategically within the VPP layout for balanced workload distribution.

7.8. Muon Optimizer

Unlike traditional optimizers such as AdamW that perform element-wise updates, the Muon optimizer introduces a matrix-aware approach by orthogonalizing entire weight matrices [99]. This improves the conditioning of optimization trajectories and significantly reduces training steps compared to AdamW.

Our integration provides production-ready support for large-scale distributed training with several key advantages: (1) full support for split query-key-value (QKV) weight layouts, enabling efficient orthogonalization even when attention projection matrices are stored as separate tensors; (2) seamless integration with the

distributed optimizer, allowing optimizer states to be sharded across data-parallel ranks while maintaining correct orthogonalization semantics; and (3) CPU offloading for Muon's orthogonalization buffers when GPU memory is constrained.

MuonClip. Training trillion-parameter models introduces stability challenges where query-key dot products can grow unbounded, causing attention explosions [100]. MuonClip addresses this with hardware-accelerated implementations in cuDNN, cudnn-frontend, and Transformer Engine.

8. Performance Evaluation

The preceding sections described Megatron-Core MoE's architecture, parallelism strategies, and optimizations. This section validates these contributions through empirical evaluation, showing the performance of the Megatron-Core MoE stack across diverse model configurations and hardware platforms.

> **Note**
>
> **Living Document.** The performance results in this section represent a point-in-time snapshot based on Megatron-Core **v0.16**, not the upper bound of achievable performance. Active optimization efforts are ongoing, and these numbers will improve in future releases. We also plan to expand coverage to additional model architectures as they emerge in the community. For the latest performance data, please refer to the Megatron-LM and Megatron-Bridge repository.

8.1. Experimental Setup

Benchmark Models. We evaluate Megatron-Core MoE on two state-of-the-art fine-grained MoE architectures that stress-test the framework's capabilities: DeepSeek-V3-685B [6] and Qwen3-235B [32, 87]. Both models feature the fine-grained expert design that amplifies the Three Walls challenges: high expert counts strain memory capacity, top-k routing increases all-to-all communication volume, and smaller per-expert computations reduce GEMM efficiency. These characteristics make them demanding benchmarks for validating our optimization strategies.

Hardware and Software Stack. Benchmarks are conducted on NVIDIA GB200 and H100 GPUs to assess performance across hardware generations. We use the `dev` branch of Megatron-LM[6] with the latest TransformerEngine[7]. The full optimization stack described in Section 4 is enabled, including but not limited to:

- **FP8 training:** MXFP8 on GB200/GB300 (Blackwell) with native Tensor Core support and blockwise FP8 on H100 (Hopper)
- **Memory optimizations:** Selective recomputation and activation & optimizer state offloading to reduce peak memory footprint
- **Optimized token dispatchers:** HybridEP[8] on GB200/GB300 NVL systems exploiting Multi-Node NVLink, and DeepEP[9] on H100 systems
- **Communication overlap:** 1F1B all-to-all overlap with expert computation
- **Kernel optimizations:** Grouped GEMM, router fusion, permutation fusion, and CUDA Graphs

Metrics. We report per-GPU throughput in TFLOPS and tokens processed per second per GPU. These metrics provide two views of computational efficiency and practical training throughput.

[6] https://github.com/NVIDIA/Megatron-LM/tree/dev
[7] https://github.com/NVIDIA/TransformerEngine/tree/main
[8] https://github.com/deepseek-ai/DeepEP/tree/hybrid-ep
[9] https://github.com/deepseek-ai/DeepEP/tree/main

Table 11: Unified throughput benchmarks (per-GPU figures) for two mixture-of-experts models on NVIDIA GB300, GB200, and H100. All configurations use force-balanced routing. The Dtype column specifies the FP8 recipe: FP8-BLK denotes blockwise FP8 on Hopper, and MXFP8 denotes microscaling FP8 on Blackwell.

Model	System	#GPUs	Seqlen	Dtype	Per-GPU TF	Tokens/s/GPU
DeepSeek-V3	GB300	256	4,096	MXFP8	1233	4,730
DeepSeek-V3	GB200	256	4,096	MXFP8	1048	4,020
DeepSeek-V3	GB200	256	4,096	BF16	857	3,298
DeepSeek-V3	H100	1024	4,096	FP8-BLK	368	1,412
Qwen3-235B	GB300	256	4,096	MXFP8	974	6,583
Qwen3-235B	GB200	256	4,096	MXFP8	919	6,212
Qwen3-235B	GB200	256	4,096	BF16	750	5,100
Qwen3-235B	H100	256	4,096	BF16	320	2,132
Qwen3-235B	GB300	128	131,072	MXFP8	1,150	1,556

8.2. Key Performance Results

Overview. Table 11 summarizes per-GPU throughput for representative fine-grained MoE training workloads on NVIDIA GB300, GB200, and H100 systems under the fully enabled optimization stack described in Section 4. All configurations use force-balanced routing to ensure even token distribution across experts.

DeepSeek-V3 and Qwen3-235B (large-scale training). Table 11 reports results for two contemporary, fine-grained MoE training workloads at a standard sequence length of 4,096, including a 1,024-GPU DeepSeek-V3 run on H100. Across these settings, the measurements demonstrate strong per-GPU efficiency and high sustained throughput across all three platforms.

FP8 Training Effectiveness. As shown in Table 11, FP8 training achieves 368 TFLOPS per GPU on H100 for DeepSeek-V3, demonstrating that the blockwise FP8 recipe enables efficient training of large-scale MoE models on Hopper architecture. The optimizations discussed in Section 4.3.5 successfully address the challenges of applying FP8 to MoE models with dynamic expert routing and varying computation patterns, maintaining numerical stability while providing the performance benefits of reduced precision.

GB200 and GB300 Performance. The GB200 and GB300 platforms deliver approximately 3× higher token throughput compared to H100 for both MoE models at comparable or smaller GPU counts. This efficiency gain stems from superior memory bandwidth, higher computational capacity, and native MXFP8 Tensor Core support. Combined with our software optimizations (Sections 4.1 and 4.3), GB200 achieves over 1,048 TFLOPS per GPU for DeepSeek-V3 training, and GB300 further pushes this to 1,233 TFLOPS per GPU.

Scalability. These results demonstrate that Megatron-Core MoE scales efficiently to production workloads with thousands of GPUs and hundreds of billions of parameters. The successful deployment of DeepSeek-V3 at 1,024-GPU scale validates the effectiveness of parallel folding (Section 3.3), optimized dispatchers (Section 4.2), and kernel optimizations (Section 4.3).

Long-context Stress Test). To capture behavior beyond the standard 4,096-token regime, Table 11 includes a long-context Qwen3-235B run at sequence length 131,072 on GB300. Even in this memory- and communication-intensive setting, the system sustains 1,150 TFLOPS per GPU, indicating strong long-context efficiency under the full optimization stack.

Configuration Details. The parallelism configurations (TP, PP, CP, EP, DP, VPP, etc.), batch settings, and other optimization details used for each benchmark entry in Table 11 are provided in Appendix B. Note that these configurations represent our best-found settings through empirical tuning; they may not be globally optimal, as

exhaustively searching the full parallelism and optimization configuration space is infeasible for models of this scale.

Section 8 demonstrated that Megatron-Core MoE achieves state-of-the-art performance across diverse MoE architectures on the latest GPU platforms. We now examine the systematic methodology behind these results and show how the optimizations from Sections 4.1–4.3 translate into deployment decisions through detailed case studies.

9. Performance Best Practices

Sections 3–7 presented a comprehensive optimization arsenal for MoE training: parallel folding, optimized dispatchers, Grouped GEMM, reduced-precision training, CUDA Graphs, and more. However, **having many optimization techniques does not automatically translate into high performance**. Each technique addresses specific bottlenecks but introduces its own trade-offs. Reduced-precision training reduces memory but requires careful precision management. Communication overlap improves throughput but adds memory overhead. CUDA Graphs eliminate host latency but conflict with dynamic tensor shapes. The challenge is not the availability of optimizations; it is knowing *which* optimizations to apply, *when* to apply them, and *how* they interact with each other.

This complexity means that achieving peak performance requires a systematic methodology, not trial-and-error tuning. Through optimization of diverse MoE models (from Mixtral to DeepSeek-V3) on GB200 and H100, we developed a repeatable workflow for identifying bottlenecks and applying targeted solutions. This section distills those lessons into actionable guidance. Section 9.1 presents the methodology under the Three Walls framework with concrete examples for each phase. Section 9.2 then shows how these principles combine for DeepSeek-V3 on GB200 and H100, including why the same model requires different strategies on different hardware.

9.1. A Systematic Optimization Workflow

We present a three-phase workflow for MoE performance optimization. This methodology emerged from tuning Mixtral, DeepSeek-V3, and Qwen3 across GB200 and H100 platforms. The process is inherently **iterative**: solving one bottleneck often exposes the next, requiring continuous profiling and refinement.

9.1.1. Phase 1: Establish Memory-Feasible Parallelism

Memory feasibility is the first constraint. Before optimizing for throughput, the configuration must fit in GPU memory. Table 12 summarizes how each parallelism strategy affects per-GPU memory and communication overhead.

Table 12: Impact of parallelism strategies on memory and communication. d = parallelism degree. †Requires distributed optimizer (`--use-distributed-optimizer`).

Strategy	Peak Activation	Weight Memory	Optimizer States	Comm (Per-Layer)
TP	$1/d$ (with SP)	$1/d$	$1/d$	High
EP	$\sim$1 (load-dependent)	$1/d$ (MoE only)	$1/d$	Medium
PP	1 ($>$1 with VPP)	$1/d$	$1/d$	Medium
CP	$1/d$	1	$1/d^{\dagger}$	Medium
DP	1	1	$1/d^{\dagger}$	Low

Note

Quick feasibility testing. Use `--fake-init-process-group` to emulate distributed training on a single GPU, enabling rapid iteration on parallelism configurations without allocating a full cluster. See the Megatron-LM documentation[a] for detailed usage.
Interactive Memory Estimator. We have developed an interactive memory simulator with a web GUI for quick parallelism tuning against memory footprints. See the blog[101] for details.

[a] https://github.com/NVIDIA/Megatron-LM/pull/2254

Example. For a 685B-parameter fine-grained MoE model, BF16 activations alone can exceed 130 GB per GPU (see Table 3), immediately ruling out baseline configurations on 80 GB devices even with parallelism, and motivating the memory optimizations in Phase 3.

9.1.2. Phase 2: Select Optimal Parallelism Strategy

Once memory-feasible configurations are identified, select the strategy that minimizes communication overhead while maintaining throughput. The optimal choice depends on model architecture, sequence length, and hardware topology.

Guideline 1: Minimize Model Parallelism, Maximize Data Parallelism

- **Goal:** Keep TP/EP/PP/CP as small as possible while avoiding OOM.
- **Why:** Model parallelism introduces communication overhead that hurts performance.
- **How:** Use distributed optimizer (`--use-distributed-optimizer`) to shard optimizer states across DP ranks, freeing memory for larger DP size.

Guideline 2: Keep EP and TP Communication Within NVLink Domain

- **Goal:** Ensure EP×TP fits within the NVLink Domain (typically 8 GPUs in a single node unless Multi-Node NVLink is used).
- **Why:** EP and TP are communication-intensive; NVLink provides much higher bandwidth than cross-node interconnects.
- **Scaling:** When scaling beyond the NVLink Domain, prefer PP over expanding TP/EP across nodes.

Note: For very large MoE models like DeepSeek-V3, EP communication volume may exceed what NVLink can hide. In this case, enable communication-computation overlap (see Section 4.2) to hide EP latency behind computation.

Guideline 3: Use Pipeline Parallelism (PP) for Multi-Node Scaling

- **Goal:** Use PP to distribute layers across nodes while keeping EP×TP within NVLink.
- **VPP:** Enable Virtual Pipeline Parallelism to reduce pipeline bubbles when PP ≥ 2.
- **Config:** Set `--pipeline-model-parallel-layout` to control Pipeline Parallelism settings (see Section 7.5). Larger VPP usually reduces pipeline bubbles but increases P2P communications; a middle value often provides the best balance. The workloads of each VPP rank should be balanced to maximize the throughput.

Guideline 4: Prefer EP over TP for Expert Layers EP offers several advantages over TP for MoE layers:

- **Better GEMM efficiency:** Larger local matrix sizes improve GPU utilization.
- **Lower communication:** EP has less communication overhead than TP for MoE layers.
- **Simpler computation graph:** Easier to overlap communication with computation.

- **Token permutation:** When EP = `num_experts`, local token permutation is eliminated.

Example: For Mixtral-8×7B, EP8×TP1 outperforms EP4×TP2.

Guideline 5: Enable Context Parallelism (CP) for Long Sequences

- **When to use:** Sequence length $\geq$ 8K tokens.
- **When to avoid:** For sequences $<$ 4K tokens, CP overhead typically exceeds benefits.
- **Key factor:** CP efficiency depends on overlapping communication with computation.
- **Config:** Set `--context-parallel-size` to partition sequences across GPUs.

Example. Consider a 256-expert MoE model on an NVL72 system. Applying Guideline 4, Parallel Folding sets expert TP = 1 so that each expert runs on a single GPU, maximizing GEMM efficiency. With Guideline 2, EP64 fits entirely within the NVLink domain. Guideline 1 then drives the remaining choices: the available memory budget determines TP and PP for attention layers, with DP filling the rest.

9.1.3. Phase 3: Profile and Optimize Bottlenecks

With a working parallelism configuration established, profile the training run to identify which wall dominates. Apply targeted optimizations based on the bottleneck type.

Memory Bottleneck (Memory Wall) **Symptom:** Forced to use full recomputation or excessively large parallelism degrees to avoid OOM.

Solutions: Apply memory-saving techniques (Table 13) to reduce memory consumption.

Table 13: Memory bottleneck solutions.

Optimization	Overhead	Config	Reference
FP8 Training	Low	`--fp8-format --fp8-recipe`	§4.1.3
Selective Recomputation	Low	`--recompute-granularity --recompute-modules`	§4.1.4
Precision-Aware Optimizer	Low	`--use-precision-aware-optimizer`	§4.1.6
Activation Offloading	Medium	`--fine-grained-activation-offloading --offload-modules`	§4.1.5
Optimizer Offloading	Medium	`--offload-optimizer-states`	§4.1.6

Communication Bottleneck (Communication Wall) **Symptom:** Profiling shows significant time spent in collective operations.

Solutions: Identify which communication is the bottleneck and apply the corresponding optimization. (Table 14)

Table 14: Communication bottleneck solutions.

Communication Type	Config	Reference
DP gradient reduce and param gather	`--overlap-grad-reduce --overlap-param-gather`	—
TP communication	`--tp-comm-overlap`	—
EP dispatcher	`--moe-token-dispatcher-type`	§4.2.2
EP all-to-all hiding	`--overlap-moe-expert-parallel-comm`	§4.2.3
PP send/recv	`--pipeline-model-parallel-layout`	§7.5

CPU Overhead Bottleneck (Compute Efficiency Wall) The Compute Efficiency Wall manifests in two distinct forms: CPU overhead and computation inefficiency. CPU overhead occurs when the host cannot launch GPU kernels fast enough, creating gaps in GPU execution.

Symptom: Nsight Systems timeline shows gaps between GPU kernels where CPU cannot launch kernels fast enough.

Solutions: Reduce host-side overhead by minimizing kernel launches and enabling CUDA Graphs (Table 15).

Table 15: CPU overhead bottleneck solutions.

Optimization	Config	Reference
Disable Python GC	`--manual-gc --manual-gc-interval 10`	—
Reduce kernel launches	Decrease TP or increase MBS	—
Enable CUDA Graphs	`--cuda-graph-impl transformer_engine`	§4.3.6

Computation Bottleneck (Compute Efficiency Wall) Computation inefficiency occurs when GPU kernels themselves underutilize hardware resources, typically due to small GEMM sizes in fine-grained MoE architectures.

Symptom: GPU SM utilization is low despite no communication or CPU bottlenecks.

Solutions: Improve kernel efficiency through batching, fusion, and lower precision (Table 16).

Table 16: Computation bottleneck solutions.

Optimization	Config	Reference
Grouped GEMM	`--moe-grouped-gemm`	§4.3.2
Kernel fusions	`--moe-router-fusion --moe-permute-fusion`	§4.3.2
FP8 precision	`--fp8-format --fp8-recipe`	§5

Example. On an NVL8 system where EP spans across nodes, profiling may reveal all-to-all communication consuming 30–50% of step time, pointing squarely at the Communication Wall. On an NVL72 system where the same EP degree stays within the NVLink domain, the dominant bottleneck after enabling FP8 often shifts to CPU overhead instead; the faster GPU computation exposes host-side launch latency. The same model on different hardware can require entirely different optimization strategies.

9.1.4. Summary

The ordering of the three phases matters: memory constraints are a hard barrier that must be resolved first (Phase 1), parallelism decisions determine the communication topology (Phase 2), and only then can profiling reveal which wall to attack (Phase 3). Crucially, this process is **iterative**. Memory optimizations may enable smaller parallelism degrees, returning to Phase 1. Some Phase 3 optimizations have their own memory cost (EP communication overlap requires extra buffers, CUDA Graphs consume additional memory), which may require revisiting earlier decisions. Continuous profiling after each round guides the next iteration.

9.2. Case Study: Tuning DeepSeek-V3 on GB200 and H100

DeepSeek-V3 represents an extreme stress test for MoE training: 685B total parameters with Multi-Token Prediction (MTP) and Multi-Latent Attention (MLA). All three walls apply: memory pressure from 256 experts, high all-to-all volume from top-8 routing, and small per-expert GEMMs. The preceding workflow (Section 9.1) showed how to approach each phase individually; this case study examines how those decisions interact to form a coherent optimization stack, and why the same model requires fundamentally different strategies on GB200 and H100.

9.2.1. Final Optimized Configuration and Performance

Table 17 summarizes the final optimized configurations and performance on GB200 and H100 platforms.

Table 17: DeepSeek-V3 final optimized configurations on GB200 and H100. †Parallel Folding is used; TP applies to the non-MoE modules only, expert TP is always 1.

Configuration	GB200	H100
Hardware	256×GB200	1024×H100
Parallelism (TP/PP/EP)†	1/4/64	2/8/64
VPP	4	4
GBS / MBS / SeqLen	8192 / 1 / 4096	8192 / 1 / 4096
Precision	MXFP8	FP8-Blockwise
Dispatcher	HybridEP	DeepEP
Recompute	`mlp`	`mlp`, `mla_up_proj`, `moe_act`, `layernorm`
CUDA Graphs	Enabled	—
EP all-to-all Overlap	—	Enabled
Performance (TFLOPS/GPU)	**1048**	**368**

The GB200 configuration uses HybridEP (optimized for NVL72) and CUDA Graphs to minimize CPU overhead, which becomes the dominant bottleneck in FP8 training on Blackwell. The H100 configuration uses FP8-blockwise precision with DeepEP and EP all-to-all overlap to hide communication latency.

Table 18 summarizes the key optimizations applied on each platform.

Table 18: DeepSeek-V3 optimization summary by platform.

Category	GB200	H100
Parallelism	Parallel Folding Flexible VPP	Parallel Folding Flexible VPP
Precision	MXFP8	FP8-Blockwise
Memory	Memory-efficient permutation Fine-grained recomputation FP8 primary weights Low-precision optimizer states Optimizer states offloading	Memory-efficient permutation Fine-grained recomputation FP8 primary weights Low-precision optimizer states
Communication	HybridEP	DeepEP EP Communication overlap
Compute Efficiency	Kernel fusions CUDA Graphs CPU-side optimizations	Kernel fusions

9.2.2. Anatomy of the Optimized Configuration

Why This Parallelism Layout? Both platforms use Parallel Folding (Guideline 4) to decouple attention and expert parallelism: TP applies only to attention layers for memory reduction, while experts use EP with TP = 1 for better GEMM efficiency. EP64 is chosen so that each GPU holds exactly four of the 256 experts, eliminating local token permutation overhead.

The key platform difference is memory capacity. GB200's 192 GB per GPU (vs. H100's 80 GB) allows TP1/PP4 instead of TP2/PP8, halving the pipeline depth, which reduces pipeline bubbles and simplifies workload balancing (Guideline 1). Flexible VPP then fine-tunes the pipeline: on H100, the layout `Et*3|(tt|)*29m|L`

groups embedding with 3 transformer layers in the first stage, places MTP in a standalone stage, and separates the loss; on GB200, `Et*4|(tttt|)*14tmL` distributes layers across 16 virtual stages for balanced computation across the shorter pipeline. These parallelism configurations establish the foundation for all subsequent optimizations.

Memory Wall FP8 training is the first lever: it halves activation memory on both platforms, freeing GPU memory that would otherwise require aggressive recomputation. On H100, the freed memory budget is critical because it enables EP communication overlap, which requires extra buffers for pipelining dispatch and combine operations (see Communication Wall below). Fine-grained recomputation (`mlp`, `mla_up_proj`, `layernorm`, `moe_act`) trims remaining memory pressure. Memory-efficient permutation eliminates redundant activation storage, and low-precision optimizer states (BF16) provide additional headroom.

On GB200, the memory chain resolves differently. NVL72 keeps EP local, so communication overlap is not critical and its memory budget is freed entirely. GB200's higher C2C bandwidth (NVLink-C2C) makes optimizer state offloading effective: the transfer overhead is low enough that offloading frees substantial GPU memory for activations. This in turn reduces the need for fine-grained recomputation to just `mlp`, far less aggressive than on H100.

Communication Wall On H100/B200 (NVL8), EP64 spans 8 nodes, and cross-node all-to-all latency would consume nearly 50% of step time with the standard all-to-all dispatcher. DeepEP reduces this overhead through fused permutation and optimized collective operations. Even so, EP communication overlap is essential to hide the remaining cross-node latency behind expert computation, and this overlap is only feasible because FP8 freed enough memory for the extra buffers (see Memory Wall above).

On GB200 (NVL72), EP64 stays entirely within the NVLink domain. HybridEP fully utilizes the 1.8 TB/s bidirectional bandwidth without requiring communication overlap. The communication wall is effectively resolved by hardware topology alone, shifting the dominant bottleneck to compute efficiency.

Compute Efficiency Wall FP8 accelerates GEMMs on both platforms (blockwise FP8 on H100, MXFP8 with native Blackwell Tensor Core support on GB200), but this speedup has a side effect: faster GPU computation exposes CPU overhead. On GB200, where NVL72 already eliminates the communication bottleneck, CPU overhead becomes the dominant constraint; the host cannot launch kernels fast enough to keep the GPU saturated.

Partial CUDA Graphs address this by capturing attention, router, and MoE preprocessing into static graphs, while leaving dynamic expert computation ungraphed. Kernel fusions (router fusion, permute fusion, MLA RoPE fusion) reduce kernel launch count. CPU/NUMA binding[10] reduces host-side memory access latency.

The cross-cutting nature of these optimizations is evident throughout: FP8 simultaneously reduces memory (activations halved), and improves compute throughput, but introduces quantization kernels that amplify CPU overhead. The same model arrives at fundamentally different optimization stacks on different hardware: H100's stack centers on hiding communication latency (DeepEP + EP overlap), while GB200's centers on eliminating CPU overhead (CUDA Graphs + kernel fusions). This contrast validates the profile-driven iterative approach from Section 9.1.

9.2.3. Lessons Learned

Four insights generalize beyond DeepSeek-V3:

[10]The `bindpcie` script at https://github.com/NVIDIA/mlperf-common/blob/main/client/bindpcie automatically detects GPU/NUMA topology based on local rank and uses `numactl` to bind each process's CPU and memory to the NUMA node nearest its GPU.

1. **Platform characteristics drive strategy**: GB200's larger memory (192GB vs 80GB) and higher C2C bandwidth enable more aggressive choices: smaller PP degree, less fine-grained recomputation, and effective optimizer offloading. H100's NVL8 requires EP communication overlap to hide cross-node latency, while GB200's NVL72 keeps EP64 within the NVLink domain.
2. **Parallel Folding unlocks flexibility**: Decoupling attention TP from expert EP allows independent optimization of each layer type. Combined with flexible VPP, this enables fine-grained workload balancing across pipeline stages.
3. **FP8 shifts bottlenecks**: FP8 training accelerates GEMMs and reduces memory, but amplifies CPU overhead as the dominant bottleneck. CUDA Graphs, kernel fusions, and CPU/NUMA binding become essential.
4. **Iterative optimization**: The optimization process is inherently cyclical. Memory optimizations (recomputation, offloading) free GPU memory, enabling communication overlap. Communication optimizations expose compute efficiency bottlenecks. Some optimizations have cross-cutting effects: FP8 training reduces both memory and compute time but increases CPU overhead; CUDA Graphs reduce CPU overhead but consume additional memory. These interactions mean that applying one optimization may require revisiting earlier decisions. Continuous profiling after each change guides the next optimization target and helps identify when diminishing returns suggest moving to the next bottleneck.

10. Megatron-Core MoE in Reinforcement Learning

Reinforcement learning (RL) post-training has become a critical paradigm following OpenAI o1 and DeepSeek-R1 [81]. Many leading RL-trained models are MoE architectures (DeepSeek-R1, Kimi-K2, etc.), and RL workloads introduce challenges that amplify the MoE-specific issues discussed throughout this report: highly variable sequence lengths stress the Memory Wall, interleaved inference and training phases demand rapid memory offloading, and routing discrepancies between inference and training engines threaten stability. To achieve high training throughput at scale, RL frameworks run Megatron-Core inside Ray workers as the distributed training backend, and use Megatron-Bridge for bidirectional Hugging Face to Megatron checkpoint conversion.

10.1. Challenges for RL Post-Training

RL post-training differs from pre-training in ways that impose new requirements on the training engine:

1. **Variable-length sequences.** Pre-training typically operates on fixed-length sequences (e.g., 4K tokens). RL workloads, by contrast, produce highly variable sequence lengths: the maximum may reach 128K or even 1M tokens, while the mean within a mini-batch is often one-half to one-quarter of the maximum. This long-tailed distribution makes it difficult to balance compute efficiency against peak memory consumption.
2. **Memory offloading.** RL frameworks typically co-locate a training engine and an inference engine on the same GPUs, offloading one engine's state while the other is active. This requires both engines to release and restore their full memory footprint quickly and completely—a non-trivial requirement when optimizer states, activations, and KV caches must all be managed.
3. **Online weight export.** The inference engine must be updated with the latest parameters after each training step. This requires the training engine to export weights rapidly in a format the inference engine can load, across diverse model architectures.
4. **Training stability.** Standard RL algorithms assume that sampled responses come from the current policy distribution. In practice, however, the inference and training engines use different optimized kernels, producing slightly different token probabilities even with identical parameters—effectively introducing off-policy bias. For MoE models, this problem is compounded: tokens from the same sequence may be routed to different experts in the inference and training engines, amplifying the discrepancy.

10.2. Megatron-Bridge

A typical RL workflow starts from a pretrained Hugging Face model and fine-tunes it in an RL framework. At scale, these frameworks often run Megatron-Core inside Ray workers for distributed training, while rollout commonly uses an inference stack that expects Hugging Face-format checkpoints. This creates two integration needs: mapping model definitions to Megatron-Core modules, and converting checkpoints between Hugging Face and Megatron formats efficiently. Megatron-Bridge addresses the checkpoint interoperability layer, enabling fast HF-to-Megatron conversion for initialization, training, and export. This pattern is used across multiple RL frameworks, including veRL [102], Slime [103], and NeMo RL.

10.3. Megatron-Core Optimization for Reinforcement Learning

Packed Sequence Support. As discussed in Section 6.4.1, we can pack sequences to remove the padding from a batch of variable sequence lengths. Building on packed sequence support, on the RL framework side, we recommend using a packing-aware dynamic batch size, which ensures that every batch has a similar total number of effective tokens. Moreover, we introduce an additional load–balancing strategy that explicitly accounts for the heterogeneous computational cost of the transformer blocks. Because the attention module scales quadratically with sequence length ($O(L^2)$) whereas the feed-forward network (FFN) scales linearly ($O(L)$), a batch that is well balanced in terms of token count can still be highly unbalanced in wall-clock time. To mitigate this issue, we first compute, for every sample in the batch, the square of its sequence length and take the sum within the mini-batch. We then sort micro-batches according to this "attention cost" metric and schedule them in a small-to-large-to-small pattern (i.e., increasing order followed by decreasing order). This serpentine ordering delivers two benefits. (i) For data parallelism, pipeline parallelism, and expert parallelism, it reduces synchronization bubbles because consecutive micro-batches have comparable attention workloads. (ii) Within pipeline parallelism, the warm-up and cool-down phases, where stages are traditionally under-utilized, observe shorter idle periods, as lighter micro-batches arrive earlier and later in the schedule.

Dynamic Context Parallelism. As discussed in Section 6.4.2, conventional training setups must pick a fixed context-parallel (CP) degree that guarantees the longest sequence in the workload will not trigger out-of-memory (OOM) failures. This conservative choice forces the majority of shorter sequences, whose memory footprints are far smaller, to run with an unnecessarily large CP degree, thereby wasting cross-device bandwidth and reducing overall throughput. Dynamic CP removes this one-size-fits-all restriction by selecting the CP degree adaptively on a per-micro-batch basis. At runtime we bucket incoming sequences by length, choose the minimal CP degree that keeps each bucket within the device memory budget, and dispatch micro-batches accordingly. Long sequences receive a higher CP degree to stay memory-safe, whereas short sequences are executed with a lower CP degree that maximizes arithmetic intensity and reduces communication volume.

CPU Optimizer Offloading. To further relieve GPU memory pressure, we offload the optimizer states to host DRAM during the forward and backward passes, loading them back onto the GPU only for the parameter-update step. Because optimizer tensors can occupy multiple times the size of model parameters, temporarily evicting them frees a substantial amount of high-bandwidth GPU memory that can instead be used to cache activations or accommodate longer sequences.

FP16 Training Support. Although bfloat16 (BF16) is widely adopted for pre-training large language models, several studies have observed that, during reinforcement-learning training, half-precision floating-point (FP16) can deliver greater numerical stability under certain hyper-parameter choices. Megatron-Core MoE implementation therefore offers a fully-featured FP16 path, including loss scaling and mixed-precision optimizer kernels, enabling practitioners to select the precision mode that best aligns with their stability requirements without sacrificing throughput.

Router Replay. Recent work [104] shows that capturing the routing decisions produced by the MoE router

during inference and replaying them in subsequent training phases can improve convergence consistency. Thanks to a community-contributed patch, Megatron-Core MoE now supports this feature: the inference engine logs each token's expert assignment, and the training stack can ingest and enforce the same routing pattern. This mechanism decouples routing variability from weight updates, leading to more stable optimization trajectories, especially in RL settings where on-policy data distribution shifts are frequent.

11. Conclusion

This report presented Megatron-Core MoE, an open-source stack for training large-scale Mixture-of-Experts models. MoE sparsity introduces two fundamental challenges: a parameter-compute mismatch that manifests as the Three Walls (Memory, Communication, and Compute Efficiency), and a dense-sparse mismatch that requires decoupled parallelism for attention and MoE layers. By combining integrated solutions across these challenges, Megatron-Core MoE enables efficient trillion-parameter-scale training.

The key technical contributions include:

- **Multi-Dimensional Parallelism and Parallel Folding.** Expert Parallelism integrates seamlessly with tensor, pipeline, context, and data parallelism. MoE Parallel Folding decouples attention and MoE layer configurations, breaking the restrictive EP $\leq$ DP constraint and enabling flexible parallelism mapping tailored to model architecture and hardware topology.
- **Memory Optimization.** Fine-grained activation recomputation, memory-efficient permutation, precision-aware optimizers with BF16 moments, and CPU activation offloading reduce memory footprint from 199.5 GB to under 80 GB per GPU for DeepSeek-V3, making trillion-parameter training feasible on current hardware.
- **Communication Optimization.** High-performance token dispatchers (DeepEP and HybridEP), and communication-computation overlap transform all-to-all from a bottleneck into a background operation hidden behind expert computation.
- **Compute Efficiency.** Grouped GEMM kernels batch expert computations for better hardware utilization, kernel fusion consolidates routing and permutation operations, CUDA Graphs reduce launch overhead, and sync-free execution eliminates host-device synchronization for dropless MoE.
- **FP8 Training.** Full FP8 support with selective precision, protecting numerically sensitive components (router, embeddings, optimizer states) while aggressively quantizing bulk computation, provides benefits across all three walls simultaneously.
- **Long-Context Training.** Context Parallelism and Tensor Parallelism scaling maintain constant subsequence lengths per device, enabling training at 16K to 64K+ token sequences where attention computation dominates.
- **Production Features.** Load balancing strategies and token dropping ensure stable training, distributed checkpointing enables parallelism-agnostic resharding, upcycling initializes MoE models from dense checkpoints, and multi-token prediction integration supports advanced training objectives.
- **Reinforcement Learning Support.** Megatron Core and Megatron-Bridge provide seamless integration with popular RL frameworks, while packed sequence support, dynamic context parallelism, and router replay address the unique challenges of RL post-training workloads.

These optimizations enable Megatron-Core MoE to achieve strong training throughput: DeepSeek-V3 (685B parameters, 256 experts) trains at 1,233/1,048 TFLOPS per GPU on 256 GB300/GB200s and 368 TFLOPS per GPU on 1,024 H100s; Qwen3-235B achieves 974/919 TFLOPS on GB300/GB200 and 320 TFLOPS on H100. The GB300 and GB200 platforms deliver approximately 3$\times$ higher token throughput compared to H100, demonstrating the framework's ability to exploit next-generation hardware.

By open-sourcing these capabilities, Megatron-Core MoE provides researchers and practitioners with production-

grade tools for MoE experimentation and deployment at scale. The modular architecture enables rapid prototyping while the full optimization stack supports training from research prototypes to trillion-parameter production models.

Contributions and Acknowledgments

Core Contributors. Zijie Yan[*§], Hongxiao Bai[*], Xin Yao[*], Dennis Liu[*], Tong Liu, Hongbin Liu, Pingtian Li, Evan Wu, Shiqing Fan, Li Tao, Robin Zhang, Yuzhong Wang, Shifang Xu, Jack Chang, Xuwen Chen, Kunlun Li, Yan Bai, Gao Deng, Nan Zheng, Vijay Anand Korthikanti, Abhinav Khattar, Ethan He, Soham Govande.
[*]Equal contribution. [§]Project lead.

Contributors. Sangkug Lym, Zhongbo Zhu, Tailai Ma, Qi Zhang, Haochen Yuan, Xiaowei Ren, Deyu Fu, Shunkang Zhang, Jiang Shao, Ray Wang, Vasudevan Rengasamy, Rachit Garg, Santosh Bhavani.

Leadership. June Yang[†], Jiajie Yao[†], Xipeng Li, Chandler Zhou, David Wu, Yingcan Wei, Ashwath Aithal, Michael Andersch, Mohammad Shoeybi.
[†]Corresponding authors.

We also acknowledge contributions from other colleagues not listed individually, as well as from the open-source community for co-developed features, valuable feedback, and continued engagement.

References

[1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," 2020. 5

[2] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, *et al.*, "Training compute-optimal large language models," 2022. 5

[3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," *arXiv preprint arXiv:1701.06538*, 2017. 5, 9

[4] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," 2022. 5, 6, 16, 37, 63, 64

[5] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, *et al.*, "Mixtral of experts," 2024. 5

[6] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, *et al.*, "Deepseek-v3 technical report," 2025. 5, 9, 10, 33, 37, 51, 58, 64, 67, 68

[7] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, "A survey on mixture of experts in large language models," *IEEE Transactions on Knowledge and Data Engineering*, p. 1–20, 2025. 5, 6

[8] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2020. 5, 7, 14

[9] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991. 5

[10] D. Eigen, M. Ranzato, and I. Sutskever, "Learning factored representations in a deep mixture of experts," *arXiv preprint arXiv:1312.4314*, 2013. 5

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. 5

[12] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "Gshard: Scaling giant models with conditional computation and automatic sharding," *arXiv preprint arXiv:2006.16668*, 2020. 5, 6, 10, 14, 16, 63, 64

[13] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, *et al.*, "Glam: Efficient scaling of language models with mixture-of-experts," 2022. 5

[14] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Saab, J. Jose, R. Srivatsa, C. Wu, and Y. He, "Tutel: Adaptive mixture-of-experts at scale," in *Proceedings of Machine Learning and Systems*, vol. 5, 2023. 5, 10, 37

[15] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale," in *International Conference on Machine Learning*, pp. 18332–18346, PMLR, 2022. 5, 37

[16] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, *et al.*, "Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024. 5

[17] DeepSeek-AI, A. Liu, B. Feng, B. Wang, B. Wang, *et al.*, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," 2024. 5, 10, 37, 64

[18] NVIDIA, "Nvidia nemotron 3: Efficient and open intelligence," 2025. 5

[19] V. Elango, N. Bhatia, R. Waleffe, R. Shafipour, T. Asida, A. Khattar, N. Assaf, M. Golub, J. Guman, T. Mitra, R. Zhao, R. Borkar, R. Zilberstein, M. Patwary, M. Shoeybi, and B. Rouhani, "Latentmoe: Toward optimal accuracy per flop and parameter in mixture of experts," 2026. 5, 64

[20] J. Krajewski, J. Ludziejewski, K. Adamczewski, M. Pióro, M. Krutul, S. Antoniak, K. Ciebiera, K. Król, T. Odrzygóźdź, P. Sankowski, *et al.*, "Scaling laws for fine-grained mixture of experts," *arXiv preprint arXiv:2402.07871*, 2024. 5

[21] J. Ludziejewski, M. Pióro, J. Krajewski, M. Stefaniak, M. Krutul, J. Małaśnicki, M. Cygan, P. Sankowski, *et al.*, "Joint moe scaling laws: Mixture of experts can be memory efficient," 2025. 5

[22] L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai, "Auxiliary-loss-free load balancing strategy for mixture-of-experts," 2024. 6, 7, 63

[23] C. Zhao, S. Zhou, L. Zhang, C. Deng, Z. Xu, Y. Liu, K. Yu, J. Li, and L. Zhao, "Deepep: an efficient expert-parallel communication library." `https://github.com/deepseek-ai/DeepEP`, 2025. 6, 10, 31

[24] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient Sparse Training with Mixture-of-Experts," *Proceedings of Machine Learning and Systems*, vol. 5, 2023. 6, 10, 36

[25] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," 2019. 7

[26] D. Liu, Z. Yan, X. Yao, T. Liu, V. Korthikanti, E. Wu, S. Fan, G. Deng, *et al.*, "Moe parallel folding: Heterogeneous parallelism mappings for efficient large-scale moe model training with megatron core," 2025. 7, 16

[27] V. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro, "Reducing activation recomputation in large transformer models," 2022. 7, 20, 23

[28] A. A. A, A. S, A. J, J. M, V. Radhakrishnan, S. V, and S. K. P, "Learning (with) distributed optimization," 2023. 7

[29] NVIDIA, "Nvidia megatron core moe user guide," 2024. 7

[30] N. Shazeer, "Glu variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020. 10

[31] N. Corporation, "Transformer engine." https://github.com/NVIDIA/TransformerEngine, 2025. Accessed: 2025-05-25. 10

[32] Qwen, A. Yang, B. Yang, B. Zhang, B. Hui, *et al.*, "Qwen2.5 technical report," 2025. 10, 64, 68

[33] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," 2020. 11, 21, 27

[34] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," 2020. 12

[35] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, *et al.*, "The llama 3 herd of models," 2024. 12

[36] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, *et al.*, "Llama: Open and efficient foundation language models," 2023. 12

[37] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, *et al.*, "Llama 2: Open foundation and fine-tuned chat models," 2023. 12

[38] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. 14, 32

[39] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Granger, P. B. Gibbons, and M. Zaharia, "Pipedream: Generalized pipeline parallelism for dnn training," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pp. 1–15, ACM, 2019. 14, 32

[40] P. Qi, X. Wan, G. Huang, and M. Lin, "Zero bubble pipeline parallelism," 2023. 14

[41] P. Qi, X. Wan, N. Amar, and M. Lin, "Pipeline parallelism with controllable memory," 2024. 14

[42] S. Li and T. Hoefler, "Chimera: Efficiently training large-scale neural networks with bidirectional pipelines," 2021. 14

[43] L. Guan, D. Li, Y. Chen, J. Liang, W. Wang, and X. Lu, "Pipeoptim: Ensuring effective 1f1b schedule with optimizer-dependent weight prediction," 2025. 14

[44] C. J. Shallue, J. Lee, J. Antognini, J. Sohl-Dickstein, R. Frostig, and G. E. Dahl, "Measuring the effects of data parallelism on neural network training," 2019. 14

[45] H. Bai, "Modern distributed data-parallel large-scale pre-training strategies for nlp models," 2022. 14

[46] V. A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro, "Sequence parallelism: Long sequence training from system perspective," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1842–1858, Association for Computational Linguistics, 2023. 14

[47] H. Liu, M. Zaharia, and P. Abbeel, "Ring attention with blockwise transformers for near-infinite context," 2023. 14, 59

[48] S. A. Jacobs, M. Tanaka, C. Zhang, M. Zhang, R. Y. Aminabadi, S. L. Song, S. Rajbhandari, and Y. He, "Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models," 2023. 14, 59

[49] S. Singh, O. Ruwase, A. A. Awan, S. Rajbhandari, Y. He, and A. Bhatele, "A hybrid tensor-expert-data parallelism approach to optimize mixture-of-experts training," in *Proceedings of the 37th International Conference on Supercomputing*, ICS '23, pp. 203–214, ACM, June 2023. 15

[50] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," 2016. 20, 22

[51] O. Beaumont, L. Eyraud-Dubois, J. Herrmann, A. Joly, and A. Shilova, "Optimal checkpointing for heterogeneous chains: How to train deep neural networks with limited memory," 2019. 20

[52] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, "Zero-offload: Democratizing billion-scale model training," 2021. 20

[53] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, *et al.*, "Pytorch fsdp: Experiences on scaling fully sharded data parallel," 2023. 21, 27, 65

[54] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning," 2021. 21

[55] G. Wang, H. Qin, S. A. Jacobs, C. Holmes, S. Rajbhandari, O. Ruwase, F. Yan, L. Yang, and Y. He, "Zero++: Extremely efficient collective communication for giant model training," 2023. 21

[56] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014. 26

[57] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," in *International Conference on Learning Representations*, 2022. 26

[58] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, *et al.*, "Deepseek-v3 technical report," 2024. 26

[59] S. Wang, J. Wei, A. Sabne, A. Davis, R. Illikkal, S. Gangadharan, K. Wang, C. Liu, X. Cai, H. Xu, *et al.*, "Overlap communication with dependent computation via decomposition in large deep learning models," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 93–106, 2022. 32

[60] L.-W. Chang, W. Bao, Q. Hou, C. Jiang, N. Zheng, Y. Zhong, X. Zhang, Z. Song, Z. Jiang, H. Lin, X. Jin, and X. Liu, "Flux: Fast software-based communication overlap on gpus through kernel fusion," 2024. 32

[61] Z. Liu, H. Liu, P. Li, S. Zhang, Z. Yan, and X. Huang, "1f1b-based ep a2a overlapping for moe models." https://zhuanlan.zhihu.com/p/28463368206, Mar. 2025. Accessed: 2026-2-15. 32

[62] D. Wang, X. Xia, Y. Zhang, R. Xiang, J. Gao, T. Liu, F. Jiang, Y. Ma, S. Zhou, X. Huang, Y. Liu, D. Chen, H. Lin, and C. Wu, "Dualpipe: A bidirectional pipeline parallelism with zerobubble on nvidia gpus." https://github.com/deepseek-ai/DualPipe-with-HybridFlow, 2025. GitHub repository. 32

[63] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 578–594, USENIX Association, 2018. 36

[64] NVIDIA, F. Abecassis, A. Agrusa, D. Ahn, J. Alben, S. Alborghetti, M. Andersch, S. Arayandi, *et al.*, "Pretraining large language models with nvfp4," 2025. 36

[65] B. Zoph, I. Bello, S. Kumar, N. Du, Y. Huang, J. Dean, N. Shazeer, and W. Fedus, "St-moe: Designing stable and transferable sparse expert models," 2022. 37

[66] N. Corporation, "CUDA graphs." https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs, 2024. CUDA C++ Programming Guide, Chapter 3.2.8. 39

[67] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burber, G. Chauhan, A. Chourdia, *et al.*, "PyTorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 929–947, ACM, 2024. 39

[68] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *International Conference on Learning Representations*, 2018. 48

[69] H. Peng, K. Wu, Y. Wei, G. Zhao, Y. Yang, Z. Liu, Y. Xiong, Z. Yang, *et al.*, "Fp8-lm: Training fp8 large language models," 2023. 48

[70] H. Xi, H. Cai, S. Xu, Y. Lu, K. Keutzer, J. Chen, and S. Han, "Coat: Compressing optimizer states and activation for memory-efficient fp8 training," 2024. 48

[71] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, *et al.*, "Fp8 formats for deep learning," 2022. 48, 50

[72] M. Fishman, B. Chmiel, R. Banner, and D. Soudry, "Scaling fp8 training to trillion-token llms," 2025. 48

[73] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A White Paper on Neural Network Quantization," 2021. 48

[74] A. Kuzmin, M. van Baalen, Y. Ren, M. Nagel, J. Peters, and T. Blankevoort, "FP8 Quantization: The Power of the Exponent," 2022. 50

[75] NVIDIA, F. Abecassis, A. Agrusa, D. Ahn, J. Alben, S. Alborghetti, M. Andersch, S. Arayandi, A. Bjorlin, A. Blakeman, E. Briones, I. Buck, B. Catanzaro, J. Choi, M. Chrzanowski, E. Chung, V. Cui, S. Dai, B. D. Rouhani, C. del Mundo, D. Donia, B. Eryilmaz, H. Estela, A. Goel, O. Goncharov, Y. Guvvala, R. Hesse, R. Hewett, H. Hum, U. Kapasi, B. Khailany, M. Khona, N. Knight, A. Kondratenko, R. Krashinsky, B. Lanir, S. Layton, M. Lightstone, D. Lo, P. Micikevicius, A. Mishra, T. Moon, D. Narayanan, C. Ni, A. Paithankar, S. Pasumarthi, A. Patel, M. Patwary, A. Poojary, G. Prasad, S. Priyadarshi, Y. Qin, X. Ren, O. Rybakov, C. Sakr, S. Satheesh, S. Sergienko, P. Shamis, K. Shankar, N. Sharma, M. Shoeybi, M. Siu, M. Smelyanskiy, D. Stosic, D. Stosic, B.-Y. Su, F. Sun, N. Tajbakhsh, S. Thomas, P. Tredak, E. Tsykunov, G. Vaithilingam, A. Vavre, R. Venkatesan, R. Waleffe, Q. Wan, H. Wang, M. Wang, L. Wei, H. Wu, E. Wu, K. Wyss, N. Xu, J. Xue, C. Yang, Y. Zhai, R. Zhang, J. Zhu, and Z. Zhu, "Pretraining large language models with nvfp4," 2025. 51, 52

[76] Minimax-AI, "Minimax m2 github repository." https://github.com/MiniMax-AI/MiniMax-M2, 2025. 51

[77] A. Li, B. Liu, B. Hu, B. Li, B. Zeng, B. Ye, C. Tang, C. Tian, C. Huang, C. Zhang, *et al.*, "Every activation boosted: Scaling general reasoner to 1 trillion open language foundation," *arXiv preprint arXiv:2510.22115*, 2025. 51

[78] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, *et al.*, "Microscaling data formats for deep learning," 2023. 51

[79] NVIDIA, "Random number generation using curanddx." https://docs.nvidia.com/cuda/curanddx/get_started/introduction.html, 2025. 56

[80] NVIDIA, "1d block scaling factors layout." https://docs.nvidia.com/cuda/cublas/#d-block-scaling-factors-layout, 2025. 56

[81] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025. 57, 76

[82] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020. 57

[83] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "BigBird: Transformers for longer sequences," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 17283–17297, 2020. 57

[84] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, 2022. 57

[85] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023. 57

[86] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "Flashattention-3: Fast and accurate attention with asynchrony and low-precision," 2024. 57

[87] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, *et al.*, "Qwen3 technical report," 2025. 58, 68

[88] W. Brandon, A. Nrusimha, K. Qian, Z. Ankner, T. Jin, Z. Song, and J. Ragan-Kelley, "Striped attention: Faster ring attention for causal transformers," 2023. 59

[89] M. M. Krell, M. Kosec, S. P. Perez, and A. Fitzgibbon, "Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance," 2021. 59

[90] K. Li, T. Ma, P. Mannan, S. Yang, G. Wu, and C. Wang, "Speeding up variable-length training with dynamic context parallelism and nvidia megatron-core." https://developer.nvidia.com/blog/speeding-up-variable-length-training-with-dynamic-context-parallelism-and-nvidia-megatron-core/, 2025. 60

[91] H. Ge, J. Feng, Q. Huang, F. Fu, X. Nie, L. Zuo, H. Lin, B. Cui, and X. Liu, "Bytescale: Efficient scaling of llm training with a 2048k context length on more than 12,000 gpus," *arXiv preprint arXiv:2502.21231*, 2025. 61

[92] Z. Wang, A. Cai, X. Xie, Z. Pan, Y. Guan, W. Chu, J. Wang, S. Li, J. Huang, C. Cai, *et al.*, "Wlb-llm: Workload-balanced 4d parallelism for large language model training," *arXiv preprint arXiv:2503.17924*, 2025. 61

[93] Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V. Zhao, A. Dai, Z. Chen, *et al.*, "Mixture-of-experts with expert choice routing," 2022. 63

[94] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on gpu clusters using megatron-lm," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021. 65

[95] A. Komatsuzaki, J. Puigcerver, J. Lee-Thorp, C. R. Ruiz, B. Mustafa, J. Ainslie, Y. Tay, M. Dehghani, and N. Houlsby, "Sparse upcycling: Training mixture-of-experts from dense checkpoints," *arXiv preprint arXiv:2212.05055*, 2022. 67

[96] E. He, A. Khattar, R. Prenger, V. Korthikanti, Z. Yan, T. Liu, S. Fan, A. Aithal, M. Shoeybi, and B. Catanzaro, "Upcycling large language models into mixture of experts," 2024. 67

[97] A. Vavre, E. He, D. Liu, Z. Yan, J. Yang, N. Tajbakhsh, and A. Aithal, "Llama 3 meets moe: Efficient upcycling," *arXiv preprint arXiv:2412.09952*, 2024. 67

[98] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve, "Better & faster large language models via multi-token prediction," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. 67

[99] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, *et al.*, "Muon is scalable for llm training," 2025. 67

[100] K. Team, "Kimi k2: Open agentic intelligence," 2025. 68

[101] Y. Bai, "Explore using the megatron-core training framework to improve gpu memory efficiency in large model training." https://developer.nvidia.cn/blog/explore-using-the-megatron-core-training-framework-to-improve-gpu-memory-efficiency-in-large-model-training/, 2025. 71

[102] G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu, "Hybridflow: A flexible and efficient rlhf framework," 2024. 77

[103] Z. Zhu, C. Xie, X. Lv, and slime Contributors, "slime: An llm post-training framework for rl scaling." `https://github.com/THUDM/slime`, 2025. GitHub repository. Corresponding author: Xin Lv. 77

[104] S. Fan *et al.*, "Router replay: Improving sample efficiency in mixture-of-experts reinforcement learning," 2025. 77

A. Notation Reference

Table 19 summarizes the notation used throughout this report.

Table 19: Notation and abbreviations used throughout this report.

Symbol	Description
Model parameters	
E	Number of experts in an MoE layer
K	Top-k routing width (experts activated per token; $K \ll E$)
h	Hidden dimension
N	Total model parameters
N_{active}	Parameters activated per token (scales with K)
N_{total}	Total parameters including all experts (scales with E)
Training dimensions	
T	Local token count per GPU
B	Batch size (tokens per batch)
s	Sequence length
L	Number of MoE layers
Parallelism	
TP	Tensor Parallelism degree
PP	Pipeline Parallelism degree
CP	Context Parallelism degree
DP	Data Parallelism degree
EP	Expert Parallelism degree
ETP	Expert Tensor Parallelism degree (MoE-specific)
EDP	Expert Data Parallelism degree (MoE-specific)
VPP	Virtual Pipeline Parallelism (interleaved pipeline stages)
Batch configuration	
MBS	Micro-batch size
GBS	Global batch size
GA	Gradient accumulation steps
Precision formats	
BF16	BFloat16 (16-bit brain floating point)
FP8-BLK	FP8 with blockwise scaling quantization (Hopper)
MXFP8	Microscaling FP8 with native Tensor support (Blackwell)
Metrics and abbreviations	
MFU	Model FLOP Utilization
GEMM	General Matrix Multiplication
SDPA	Scaled Dot-Product Attention
MLA	Multi-Latent Attention
MTP	Multi-Token Prediction

B. Detailed Benchmark Configurations

This section lists the parallelism configurations and detailed settings for reproducing the performance numbers reported in Table 11. Each configuration is identified by its system, GPU count, and precision format, and the corresponding hyper-parameter string summarizes the parallelism layout and batch configuration.

Table 20: Parallelism and training configuration details for benchmark entries reported in Table 11.

Model	Configuration	Hyper-parameters
DeepSeek-V3	GB300, 256 GPUs, 4k seqlen, MXFP8, 1233 TF	`TP1 PP4 CP1 EP64 VPP4 MBS1 GBS8192`
DeepSeek-V3	GB200, 256 GPUs, 4k seqlen, MXFP8, 1048 TF	`TP1 PP4 CP1 EP64 VPP4 MBS1 GBS8192`
DeepSeek-V3	GB200, 256 GPUs, 4k seqlen, BF16, 857 TF	`TP1 PP8 CP1 EP32 VPP4 MBS1 GBS4096`
DeepSeek-V3	H100, 1,024 GPUs, 4k seqlen, FP8-BLK, 368 TF	`TP2 PP4 CP1 EP64 VPP4 MBS1 GBS8192`
Qwen3-235B	GB300, 256 GPUs, 4k seqlen, MXFP8, 974 TF	`TP1 PP4 CP1 EP64 VPP 6 MBS2 GBS3072`
Qwen3-235B	GB200, 256 GPUs, 4k seqlen, MXFP8, 919 TF	`TP1 PP4 CP1 EP64 VPP 6 MBS3 GBS3072`
Qwen3-235B	GB200, 256 GPUs, 4k seqlen, BF16, 750 TF	`TP1 PP4 CP1 EP32 VPP12 MBS1 GBS8192`
Qwen3-235B	H100, 256 GPUs, 4k seqlen, BF16, 320 TF	`TP2 PP8 CP1 EP32 VPP 4 MBS1 GBS2048`
Qwen3-235B	GB300, 128 GPUs, 128k seqlen, MXFP8, 1,150 TF	`TP4 PP4 CP4 EP32 VPP12 MBS1 GBS1024`

B.1. Configuration Details

Table 20 lists the benchmark configurations corresponding to the throughput results in Table 11. These settings are best-found configurations from empirical tuning at the time of writing and may not be globally optimal.

B.2. Key Optimizations

This section summarizes the configuration of the most performance-critical optimization features used in our benchmark runs. Although the full optimization space is larger, these features consistently have first-order impact on throughput and are configured differently across workloads and platforms.

DeepSeek-V3

- **Dispatcher:** HybridEP on GB300/GB200; DeepEP on H100.
- **Recompute:** None on GB300; `mlp` on GB200; `up_proj, mlp` on H100.
- **1F1B overlap:** ON for GB300 and H100; OFF for GB200.
- **CUDA Graphs:** `attn, moe_router, moe_preprocess` on GB300/GB200; OFF on H100.

These settings are chosen to balance memory headroom, communication overhead, and kernel efficiency for maximum sustained throughput on each platform.

Qwen3-235B

- **Dispatcher:** HybridEP on GB300/GB200; DeepEP on H100.
- **Recompute:** `moe_act, layernorm` on GB200 and H100.
- **1F1B overlap:** OFF for GB300; ON for GB200 and H100.
- **CUDA Graphs:** `attn, moe_router, moe_preprocess`.

The resulting configuration pattern reflects a throughput-first tuning strategy under model- and hardware-specific constraints.

B.3. Reproducibility

The benchmark numbers in Table 11 can be reproduced through two supported workflows. First, users can run end-to-end training with **Megatron-Bridge** [11], which provides a higher-level interface for model, parallelism, and optimization configuration. Second, users can launch training directly with **Megatron-Core (MCore)** using model-specific scripts in **Megatron-MoE-ModelZoo** [12], which exposes the full set of low-level launch

[11] https://github.com/NVIDIA-NeMo/Megatron-Bridge/tree/main/scripts/performance
[12] https://github.com/yanring/Megatron-MoE-ModelZoo/tree/main/best_practice/readme.md

flags for performance tuning. In both workflows, start from Table 20 and adjust only a few cluster-specific runtime settings (for example, hostfile, environment modules, and job scheduler options).